



المدرسة الوطنية للعلوم  
التطبيقية - مراكش  
ÉCOLE NATIONALE DES SCIENCES  
APPLIQUÉES - MARRAKECH



Université Cadi Ayyad  
École Nationale des Sciences Appliquées - Marrakech  
Filière : Génie Cyberdéfense et Systèmes de Télécommunications  
Embarqués

# Rapport de Projet de fin de semestre

---

Développement d'une Plateforme d'Audit de Dépendances  
Android avec Pipeline DevSecOps et Déploiement Automatisé  
sur AWS

---

**Réalisé par :**

Faik Mohammed Amine  
Fath Othmane  
Ikherazen Maryam  
Isedrhas Aya

**Encadré par :**

Pr. Bekkari Aissam

**Examiné par :**

Pr. Bekkari Aissam  
Pr. Charef Ayoub

Année universitaire : 2025–2026



---

# Dédicaces

*À nos familles,  
pour leur amour, leur patience, leurs sacrifices et leur soutien inconditionnel.*

*À nos enseignants et encadrants,  
pour leurs précieux conseils, leur disponibilité et le partage de leur savoir,  
qui ont grandement contribué à la réalisation de ce travail.*

*À nos amis et à toutes les personnes qui nous ont encouragés et motivé,  
pour leur confiance, leur motivation et leur présence à nos côtés.*

*Nous dédions ce travail avec respect,  
gratitude, reconnaissance et profonde affection.*

# Remerciements

Au terme de ce projet, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Nous adressons nos sincères remerciements à notre encadrant de projet, **M. Bekkari Aissam**, pour sa disponibilité, ses conseils pertinents et son accompagnement constant tout au long de ce travail. Son expertise et sa rigueur scientifique ont grandement contribué à la qualité et à l'orientation de nos travaux.

Nous exprimons également notre reconnaissance envers notre examinateur **Charef Ayoub**, pour le temps consacré à l'évaluation de ce travail ainsi que pour l'intérêt porté à notre projet. Vos remarques et suggestions seront précieuses pour améliorer et approfondir nos futures recherches.

Nous remercions également l'ensemble du corps professoral de l'ENSA Marrakech, et plus particulièrement les enseignants du département GCDSTE, pour la formation théorique et pratique qui nous a permis de mener à bien ce projet.

Nous n'oublions pas nos familles et nos proches pour leur soutien moral, leur patience et leurs encouragements tout au long de cette période de travail.

Enfin, nous remercions toute personne ayant contribué, directement ou indirectement, à la réussite de ce projet par son aide, ses conseils ou son soutien.

# Acknowledgements

We would like to express our sincere gratitude to all those who contributed, directly or indirectly, to the completion of this project.

We are especially grateful to our supervisor, **Mr. Bekkari Aissam**, for his continuous guidance, availability, and valuable advice throughout this work. His expertise and scientific rigor greatly contributed to the quality and direction of our project.

We also extend our appreciation to our examiner, **Mr. Charef Ayoub**, for the time devoted to evaluating this work and for his insightful comments and suggestions, which will be valuable for improving and further developing our research.

We would like to thank the teaching staff of ENSA Marrakech, particularly those of the GCDSTE department, for the theoretical and practical knowledge that enabled us to successfully complete this project.

We are also deeply grateful to our families and friends for their moral support, patience, and encouragement throughout this journey.

Finally, we thank everyone who, in any way, contributed to the success of this project.

# Résumé

La sécurité de la chaîne d’approvisionnement logicielle constitue aujourd’hui l’un des défis majeurs du développement d’applications mobiles Android. Un projet Android typique embarque entre cinquante et deux cents dépendances transitives dans son binaire final, souvent invisibles aux développeurs et potentiellement exposées à des vulnérabilités connues, des licences incompatibles ou des bibliothèques de traçage collectant les données des utilisateurs. Les outils existants n’offrent qu’une réponse partielle à cette problématique, sans combiner analyse multicritère, génération de SBOM standards et remédiation assistée par intelligence artificielle.

Ce projet présente **Dependency Risk Radar** (DRR), une plateforme open-source d’audit automatisé de dépendances pour les projets Android et Java. DRR analyse chaque composant détecté selon quatre dimensions de risque complémentaires — exposition aux vulnérabilités CVE, obsolescence sémantique, risque de licence et détection de trackers — en interrogeant parallèlement cinq sources de données externes (OSV.dev, NIST NVD, Maven Central, ClearlyDefined, Exodus Privacy). Un score global normalisé sur cent est calculé pour chaque composant selon une formule pondérée, et propagé transitivement dans un graphe orienté acyclique modélisant l’ensemble des dépendances. À l’issue de chaque analyse, DRR génère un Software Bill of Materials conforme aux standards CycloneDX 1.5 et SPDX 2.3, ainsi qu’un plan de mise à jour priorisé produit par un module d’intelligence artificielle s’appuyant sur Google Gemini.

En parallèle, ce projet met en œuvre un pipeline DevSecOps complet appliqué au développement de DRR lui-même, incarnant le principe du *shift-left security*. Le pipeline, orchestré par GitHub Actions, intègre sept portes de sécurité automatisées : analyse statique du code source (Bandit, ESLint), détection de secrets (Gitleaks), audit de dépendances (pip-audit, npm audit), tests unitaires avec couverture (113 tests, seuil de 60%), et scan de vulnérabilités des images Docker (Trivy). Le déploiement automatisé repose sur Amazon ECR et Amazon ECS, avec une authentification sans clé statique via le protocole OIDC. Un système d’observabilité basé sur Grafana et Loki centralise les logs de l’ensemble du pipeline.

**Mots-clés :** sécurité de la chaîne d’approvisionnement logicielle, Android, analyse de dépendances, SBOM, DevSecOps, CI/CD, AWS ECS, scoring multicritère, intelligence artificielle.

# Abstract

Software supply chain security has become one of the major challenges in Android mobile application development. A typical Android project embeds between fifty and two hundred transitive dependencies in its final binary, often invisible to developers and potentially exposed to known vulnerabilities, incompatible licenses, or tracking libraries collecting user data. Existing tools provide only a partial response to this problem, without combining multicriteria analysis, standard SBOM generation, and AI-assisted remediation.

This project presents **Dependency Risk Radar** (DRR), an open-source platform for automated dependency auditing of Android and Java projects. DRR analyzes each detected component across four complementary risk dimensions — CVE vulnerability exposure, semantic obsolescence, license risk, and tracker detection — by querying five external data sources in parallel (OSV.dev, NIST NVD, Maven Central, ClearlyDefined, Exodus Privacy). A global score normalized to one hundred is computed for each component using a weighted formula, and propagated transitively through a directed acyclic graph modeling all dependencies. At the end of each analysis, DRR generates a Software Bill of Materials compliant with the CycloneDX 1.5 and SPDX 2.3 standards, along with a prioritized update plan produced by an artificial intelligence module powered by Google Gemini.

In parallel, this project implements a complete DevSecOps pipeline applied to the development of DRR itself, embodying the *shift-left security* principle. The pipeline, orchestrated by GitHub Actions, integrates seven automated security gates : static source code analysis (Bandit, ESLint), secret detection (Gitleaks), dependency auditing (pip-audit, npm audit), unit testing with coverage enforcement (113 tests, 60% threshold), and Docker image vulnerability scanning (Trivy). Automated deployment relies on Amazon ECR and Amazon ECS, with keyless authentication via the OIDC protocol. An observability system based on Grafana and Loki centralizes logs across the entire pipeline.

**Keywords :** software supply chain security, Android, dependency analysis, SBOM, DevSecOps, CI/CD, AWS ECS, multicriteria scoring, artificial intelligence.

# Table des matières

|                                                              |           |
|--------------------------------------------------------------|-----------|
| Dédicaces                                                    | ii        |
| Remerciements                                                | iii       |
| Acknowledgements                                             | iv        |
| Résumé                                                       | v         |
| Abstract                                                     | vi        |
| Liste des figures                                            | xiii      |
| Introduction Générale                                        | 1         |
| <b>1 Contexte Général</b>                                    | <b>3</b>  |
| 1.1 Introduction                                             | 3         |
| 1.2 La sécurité de la chaîne d’approvisionnement logicielle  | 3         |
| 1.2.1 Définition et enjeux                                   | 3         |
| 1.2.2 Incidents majeurs et prise de conscience mondiale      | 3         |
| 1.2.3 Réponse réglementaire internationale                   | 4         |
| 1.3 Spécificités de l’écosystème Android                     | 5         |
| 1.3.1 La complexité des dépendances Android                  | 5         |
| 1.3.2 Le problème des trackers Android                       | 5         |
| 1.4 État de l’art des outils d’analyse de dépendances        | 5         |
| 1.4.1 Outils existants                                       | 5         |
| 1.4.2 Analyse comparative                                    | 7         |
| 1.4.3 Lacunes identifiées                                    | 7         |
| 1.5 Présentation du projet Dependency Risk Radar             | 7         |
| 1.5.1 Vue d’ensemble                                         | 7         |
| 1.5.2 Fonctionnalités principales                            | 8         |
| 1.5.3 Architecture en couches                                | 8         |
| 1.6 Objectifs du projet                                      | 9         |
| <b>2 Cadre Théorique et État de l’Art</b>                    | <b>11</b> |
| 2.1 Introduction                                             | 11        |
| 2.2 Le système de build Gradle                               | 11        |
| 2.3 L’APK Android                                            | 11        |
| 2.4 Fondements théoriques de l’analyse des risques logiciels | 12        |
| 2.4.1 Les vulnérabilités logicielles : CVE et CVSS           | 12        |
| 2.4.2 L’obsolescence des dépendances                         | 12        |



---

|          |                                                                              |           |
|----------|------------------------------------------------------------------------------|-----------|
| 2.4.3    | Les licences logicielles . . . . .                                           | 13        |
| 2.4.4    | Les trackers et les permissions Android . . . . .                            | 13        |
| 2.5      | DevSecOps : philosophie et fondements . . . . .                              | 13        |
| 2.5.1    | Origine et définition . . . . .                                              | 13        |
| 2.5.2    | Principe du Shift-Left . . . . .                                             | 14        |
| 2.5.3    | Piliers du DevSecOps . . . . .                                               | 14        |
| 2.6      | Les grandes familles d’analyse de sécurité . . . . .                         | 15        |
| 2.6.1    | Analyse statique de sécurité applicative (SAST) . . . . .                    | 15        |
| 2.6.2    | Détection de secrets ( <i>Secrets Scanning</i> ) . . . . .                   | 16        |
| 2.6.3    | Analyse de la composition logicielle (SCA) . . . . .                         | 16        |
| 2.6.4    | Scan de vulnérabilités des images de conteneurs . . . . .                    | 17        |
| 2.7      | Observabilité du pipeline . . . . .                                          | 19        |
| 2.8      | Amazon Web Services : infrastructure cloud et orchestration de conteneurs    | 20        |
| 2.9      | Authentification fédérée entre le pipeline et l’infrastructure cloud . . . . | 20        |
| <b>3</b> | <b>Technologies et Outils Utilisés</b>                                       | <b>22</b> |
| 3.1      | Introduction . . . . .                                                       | 22        |
| 3.2      | Flux 1 — Intégration et livraison continues . . . . .                        | 22        |
| 3.2.1    | GitHub Actions . . . . .                                                     | 22        |
| 3.2.2    | Docker . . . . .                                                             | 23        |
| 3.3      | Flux 2 — Infrastructure cloud AWS . . . . .                                  | 23        |
| 3.3.1    | Amazon ECR — Elastic Container Registry . . . . .                            | 23        |
| 3.3.2    | Amazon ECS — Elastic Container Service . . . . .                             | 24        |
| 3.3.3    | Amazon EC2 — Elastic Compute Cloud . . . . .                                 | 24        |
| 3.3.4    | AWS IAM — Identity and Access Management . . . . .                           | 24        |
| 3.3.5    | OpenID Connect — Authentification fédérée . . . . .                          | 25        |
| 3.4      | Flux 3 — Observabilité et monitoring . . . . .                               | 25        |
| 3.4.1    | Grafana . . . . .                                                            | 25        |
| 3.4.2    | Loki . . . . .                                                               | 26        |
| <b>4</b> | <b>Dependency Risk Radar : Conception et Architecture</b>                    | <b>27</b> |
| 4.1      | Introduction . . . . .                                                       | 27        |
| 4.2      | Architecture globale en quatre couches . . . . .                             | 27        |
| 4.3      | Modèles de données . . . . .                                                 | 30        |
| 4.3.1    | Vue d’ensemble . . . . .                                                     | 30        |
| 4.3.2    | Entités principales . . . . .                                                | 32        |
| 4.4      | Formule de scoring multicritère . . . . .                                    | 32        |
| 4.4.1    | Formule globale . . . . .                                                    | 32        |
| 4.4.2    | Dimensions de risque . . . . .                                               | 32        |
| 4.5      | Graphe de dépendances et propagation transitive . . . . .                    | 33        |
| 4.5.1    | Modélisation par graphe orienté acyclique . . . . .                          | 33        |
| 4.5.2    | Visualisation du graphe . . . . .                                            | 33        |
| 4.6      | Génération de SBOM . . . . .                                                 | 34        |
| 4.7      | Planificateur de mises à jour assisté par IA . . . . .                       | 34        |
| 4.7.1    | Architecture du module IA . . . . .                                          | 34        |
| 4.7.2    | Contenu du plan généré . . . . .                                             | 35        |
| 4.8      | Endpoints REST et WebSocket . . . . .                                        | 35        |
| 4.9      | Frontend React et ses six vues . . . . .                                     | 36        |
| 4.9.1    | Interface de soumission . . . . .                                            | 36        |

---

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| 4.9.2    | Vue Overview . . . . .                                       | 37        |
| 4.9.3    | Vue Components . . . . .                                     | 37        |
| 4.9.4    | Vue Dep Graph . . . . .                                      | 38        |
| 4.9.5    | Vue Update Plan . . . . .                                    | 38        |
| 4.9.6    | Vue SBOM et Export PDF . . . . .                             | 38        |
| 4.10     | Diagramme de cas d'utilisation . . . . .                     | 38        |
| 4.11     | Bilan architectural . . . . .                                | 40        |
| <b>5</b> | <b>Pipeline DevSecOps : Conception et Mise en Œuvre</b>      | <b>41</b> |
| 5.1      | Introduction . . . . .                                       | 41        |
| 5.2      | Architecture générale du pipeline . . . . .                  | 41        |
| 5.3      | Workflow CI — Security & Quality Gate . . . . .              | 41        |
| 5.3.1    | Déclencheurs . . . . .                                       | 42        |
| 5.3.2    | Stage 2a — SAST Python : Bandit . . . . .                    | 42        |
| 5.3.3    | Stage 2b — SAST JavaScript : ESLint . . . . .                | 42        |
| 5.3.4    | Stage 2c — Scan de secrets : Gitleaks . . . . .              | 42        |
| 5.3.5    | Stage 3a — SCA Python : pip-audit . . . . .                  | 42        |
| 5.3.6    | Stage 3b — SCA Node : npm audit . . . . .                    | 42        |
| 5.3.7    | Stage 4 — Build Docker et tests . . . . .                    | 42        |
| 5.3.8    | Stage 5 — Scan de conteneurs : Trivy . . . . .               | 43        |
| 5.3.9    | Stage 6 — Observabilité : envoi des logs vers Loki . . . . . | 43        |
| 5.4      | Workflow CD — Build, Push & Deploy . . . . .                 | 43        |
| 5.4.1    | Déclencheur conditionnel . . . . .                           | 43        |
| 5.4.2    | Authentification OIDC entre GitHub Actions et AWS . . . . .  | 43        |
| 5.4.3    | Stage 6 — Push vers Amazon ECR . . . . .                     | 44        |
| 5.4.4    | Stage 7 — Déploiement sur Amazon ECS . . . . .               | 44        |
| 5.4.5    | Smoke test post-déploiement . . . . .                        | 44        |
| 5.4.6    | Stage 8 — Observabilité CD . . . . .                         | 44        |
| <b>6</b> | <b>Infrastructure AWS et Déploiement</b>                     | <b>46</b> |
| 6.1      | Introduction . . . . .                                       | 46        |
| 6.2      | Vue d'ensemble de l'architecture AWS . . . . .               | 46        |
| 6.3      | Mise en place des dépôts Amazon ECR . . . . .                | 47        |
| 6.3.1    | Choix d'Amazon ECR . . . . .                                 | 48        |
| 6.3.2    | Création des dépôts . . . . .                                | 48        |
| 6.4      | Cluster ECS, définitions de tâches et services . . . . .     | 48        |
| 6.4.1    | Le cluster ECS drr-cluster . . . . .                         | 48        |
| 6.4.2    | Concept de Task Definition . . . . .                         | 49        |
| 6.4.3    | Task Definitions déployées . . . . .                         | 49        |
| 6.4.4    | Services ECS et ordre de démarrage . . . . .                 | 49        |
| 6.5      | Rôles IAM et politiques de permissions . . . . .             | 50        |
| 6.5.1    | Principe du moindre privilège . . . . .                      | 50        |
| 6.5.2    | Synthèse des rôles IAM . . . . .                             | 50        |
| 6.5.3    | Authentification OIDC pour GitHub Actions . . . . .          | 51        |
| 6.6      | Configuration d'AWS Secrets Manager . . . . .                | 52        |
| 6.6.1    | Problématique de la gestion des secrets . . . . .            | 52        |
| 6.6.2    | Secrets configurés . . . . .                                 | 52        |
| 6.6.3    | Mécanisme d'injection dans les conteneurs . . . . .          | 53        |
| 6.7      | Groupes de sécurité et configuration réseau . . . . .        | 53        |

---

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 6.7.1    | Security Group applicatif — drr_security_group . . . . .        | 53        |
| 6.7.2    | Security Group monitoring — drr-monitoring-sg . . . . .         | 54        |
| 6.8      | Flux de déploiement de bout en bout . . . . .                   | 54        |
| 6.8.1    | Vue d'ensemble du pipeline CD . . . . .                         | 54        |
| 6.8.2    | Réutilisation des images CI . . . . .                           | 55        |
| 6.8.3    | Mise à jour progressive ECS (Rolling Update) . . . . .          | 55        |
| 6.8.4    | Vérification de santé post-déploiement (Smoke Test) . . . . .   | 55        |
| 6.9      | Mise en place de Grafana et Loki . . . . .                      | 56        |
| 6.9.1    | Architecture du monitoring . . . . .                            | 56        |
| 6.9.2    | Provisioning automatique de Grafana . . . . .                   | 56        |
| 6.9.3    | Dashboard DevSecOps et requêtes LogQL . . . . .                 | 56        |
| 6.10     | Bilan de l'infrastructure . . . . .                             | 57        |
| <b>7</b> | <b>Résultats et Évaluation</b>                                  | <b>58</b> |
| 7.1      | Introduction . . . . .                                          | 58        |
| 7.2      | Résultats du pipeline CI — Security & Quality Gate . . . . .    | 58        |
| 7.2.1    | État final du pipeline . . . . .                                | 58        |
| 7.2.2    | Détail des résultats par stage . . . . .                        | 59        |
| 7.3      | Résultats du pipeline CD — Build, Push & Deploy . . . . .       | 59        |
| 7.3.1    | Premier déploiement réussi . . . . .                            | 59        |
| 7.3.2    | Détail des jobs CD . . . . .                                    | 60        |
| 7.4      | Résultats de qualité du code . . . . .                          | 60        |
| 7.4.1    | Suite de tests unitaires . . . . .                              | 60        |
| 7.4.2    | Couverture de code . . . . .                                    | 61        |
| 7.4.3    | Résultats Bandit — analyse statique Python . . . . .            | 61        |
| 7.4.4    | Résultats Trivy — scan des images Docker . . . . .              | 61        |
| 7.5      | Observabilité — Tableaux de bord Grafana . . . . .              | 62        |
| 7.5.1    | Dashboard CI Pipeline — DevSecOps . . . . .                     | 62        |
| 7.5.2    | Logs bruts CI dans Loki . . . . .                               | 63        |
| 7.5.3    | Dashboard CD Pipeline — Deploy ECS . . . . .                    | 63        |
| 7.5.4    | Logs bruts CD dans Loki . . . . .                               | 64        |
| 7.6      | Bilan des résultats . . . . .                                   | 65        |
| 7.6.1    | Objectifs atteints . . . . .                                    | 65        |
|          | <b>Conclusion Générale</b>                                      | <b>67</b> |
| <b>A</b> | <b>Annexe</b>                                                   | <b>69</b> |
| A.1      | Script User Data — enregistrement dans le cluster ECS . . . . . | 69        |
| A.2      | Workflow CI — Security & Quality Gate . . . . .                 | 69        |
| A.3      | Workflow CD — Build, Push & Deploy . . . . .                    | 74        |
| A.4      | Task Definition drr-redis (JSON complet) . . . . .              | 78        |
| A.5      | Task Definition drr-backend (JSON complet) . . . . .            | 78        |
| A.6      | Task Definition drr-frontend (JSON complet) . . . . .           | 80        |
| A.7      | Trust Policy du rôle GitHubActionsRole . . . . .                | 80        |
| A.8      | Politique inline DRRSecretsManagerAccess . . . . .              | 81        |
| A.9      | Configuration Loki — schéma de stockage v13 . . . . .           | 81        |
| A.10     | Smoke test post-déploiement . . . . .                           | 82        |
| A.11     | Requêtes LogQL du dashboard DevSecOps . . . . .                 | 83        |

# Table des figures

|      |                                                                                                                                                                                                                                            |    |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Approche DevSecOps. . . . .                                                                                                                                                                                                                | 14 |
| 2.2  | Principe du Shift-Left : détection précoce et coût de correction. . . . .                                                                                                                                                                  | 14 |
| 2.3  | Secret Scanning. . . . .                                                                                                                                                                                                                   | 16 |
| 2.4  | SCA architecture. . . . .                                                                                                                                                                                                                  | 17 |
| 2.5  | Flux du scan de l'image Docker. . . . .                                                                                                                                                                                                    | 18 |
| 2.6  | Observabilité du pipeline. . . . .                                                                                                                                                                                                         | 19 |
| 2.7  | Pipeline de déploiement conteneurisé vers AWS via GitHub Actions. . . . .                                                                                                                                                                  | 20 |
| 2.8  | Authentification fédérée. . . . .                                                                                                                                                                                                          | 21 |
| 3.1  | Logo GitHub Actions. . . . .                                                                                                                                                                                                               | 22 |
| 3.2  | Logo Docker. . . . .                                                                                                                                                                                                                       | 23 |
| 3.3  | Logo Amazon Web Services. . . . .                                                                                                                                                                                                          | 23 |
| 3.4  | Logo Amazon ECR. . . . .                                                                                                                                                                                                                   | 23 |
| 3.5  | Logo Amazon ECS. . . . .                                                                                                                                                                                                                   | 24 |
| 3.6  | Logo Amazon EC2. . . . .                                                                                                                                                                                                                   | 24 |
| 3.7  | Logo AWS IAM. . . . .                                                                                                                                                                                                                      | 24 |
| 3.8  | Logo OpenID Connect. . . . .                                                                                                                                                                                                               | 25 |
| 3.9  | Logo Grafana. . . . .                                                                                                                                                                                                                      | 25 |
| 3.10 | Logo Grafana Loki. . . . .                                                                                                                                                                                                                 | 26 |
| 4.1  | Diagramme de séquence de l'analyse Gradle dans DRR . . . . .                                                                                                                                                                               | 29 |
| 4.2  | Diagramme de classes de DRR — neuf entités principales : <b>Pipeline</b> , <b>Component</b> , <b>CVE</b> , <b>License</b> , <b>Tracker</b> , <b>RiskScores</b> , <b>ScoringEngine</b> , <b>SBOMGenerator</b> et <b>AIPlanner</b> . . . . . | 31 |
| 4.3  | Vue <i>Dep Graph</i> de DRR — graphe force-directed du projet PizzaRecipes (63 nœuds, 261 arcs), coloré par niveau de risque : vert (Low), jaune (Moderate) . . . . .                                                                      | 34 |
| 4.4  | Vue <i>Update Plan</i> de DRR — plan généré par règles déterministes pour le projet PizzaRecipes : 5 mises à jour de priorité MEDIUM, réduction de risque estimée à 60% . . . . .                                                          | 35 |
| 4.5  | Interface d'accueil de DRR — page de soumission avec les deux modes d'ingestion (APK File et Gradle Project) et l'historique des analyses récentes . . . . .                                                                               | 37 |
| 4.6  | Vue <i>Overview</i> de DRR pour le projet PizzaRecipes — score global 25.3 (Moderate), 0 CVE, 6 composants obsolètes, distribution de risque majoritairement Low . . . . .                                                                 | 37 |
| 4.7  | Vue <i>Components</i> de DRR — tableau des 63 composants du projet PizzaRecipes, triés par score décroissant, avec scores, licences et niveaux de risque . . . . .                                                                         | 38 |

---

|      |                                                                                                                                                                                                                                                                                  |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.8  | Diagramme de cas d'utilisation de DRR — deux acteurs internes (Security Analyst, Developer) et cinq acteurs externes (Maven Central, OSV.dev, ClearlyDefined, Gemini AI, Exodus Privacy) . . . . .                                                                               | 39 |
| 6.1  | Architecture globale de l'infrastructure AWS de DRR . . . . .                                                                                                                                                                                                                    | 47 |
| 6.2  | Instances EC2 déployées pour DRR — <b>ECS Instance</b> (cluster applicatif) et <b>Grafana</b> (stack monitoring), toutes deux en état <i>Running</i> dans la zone <b>eu-west-1c</b> . . . . .                                                                                    | 47 |
| 6.3  | Dépôts privés Amazon ECR créés pour DRR — <b>drr/backend</b> et <b>drr/frontend</b> dans le registre <b>ID.dkr.ecr.eu-west-1.amazonaws.com</b> , chiffrement AES-256 et tags mutables . . . . .                                                                                  | 48 |
| 6.4  | Cluster ECS <b>drr-cluster</b> — 2 services actifs, 2 tâches en cours d'exécution, 1 instance EC2 enregistrée, supervision Container Insights activée                                                                                                                            | 49 |
| 6.5  | Services ECS actifs dans <b>drr-cluster</b> — <b>drr-backend-service</b> (Task Definition <b>drr-backend:11</b> ) et <b>drr-frontend-service</b> (Task Definition <b>drr-frontend:4</b> ), chacun avec 1/1 tâche en cours d'exécution . . . . .                                  | 50 |
| 6.6  | Rôle IAM <b>ecsTaskExecutionRole</b> — trois politiques attachées : <b>AmazonECSTaskExecutionRolePolicy</b> (gérée AWS) et <b>DRRSecretsManagerAccess</b> (inline, accès aux secrets) .                                                                                          | 51 |
| 6.7  | Rôle IAM <b>GitHubActionsRole</b> — deux politiques gérées : <b>AmazonEC2ContainerRegistryPowerUser</b> (push/pull ECR) et <b>AmazonECS_FullAccess</b> (déploiement ECS via OIDC)                                                                                                | 52 |
| 6.8  | Secrets stockés dans AWS Secrets Manager — <b>GEMINI_API_KEY</b> (dernière récupération le 10 juin 2026) et <b>BACKEND_URL</b> . . . . .                                                                                                                                         | 53 |
| 6.9  | Security Group <b>drr_security_group</b> ( <b>sg-07611cbd95790c35d</b> ) — 3 règles entrantes : port 3000 (frontend React), port 8000 (backend API FastAPI), port 22 (SSH) . . . . .                                                                                             | 54 |
| 6.10 | Flux de déploiement CD de bout en bout . . . . .                                                                                                                                                                                                                                 | 55 |
| 7.1  | Pipeline CI run #28 — statut <i>Success</i> en 5m 21s, commit <b>9c89911</b> , 8 jobs tous verts : SAST Python (8s), SAST ESLint (14s), Gitleaks (5s), pip-audit (32s), npm audit (13s), Build+Tests (3m 14s), Trivy (1m 17s), Loki (2s) . . . . .                               | 58 |
| 7.2  | Pipeline CD run #25 — statut <i>Success</i> en 4m 44s, déclenché par <b>workflow_run</b> sur le CI #28 : Push ECR (1m 16s), Deploy ECS (3m 14s), Loki (4s) . . . . .                                                                                                             | 60 |
| 7.3  | Dashboard Grafana <i>CI Pipeline</i> — <i>DevSecOps</i> — 52 runs total, 36 succès (dont 25 et 33 visibles), source de données Loki, filtres par branche et événement . . . . .                                                                                                  | 62 |
| 7.4  | Panels analytiques du dashboard CI — répartition globale succès/échecs (27%/73%), échecs par job (Build & Test : 75%, Secrets Scan : 13%, Container Scan : 13%), runs par événement (push : 100%), taux de succès par branche . . . . .                                          | 62 |
| 7.5  | Logs bruts Loki — run CI #28 (commit <b>9c899112d7445316b723373d14a138af044574fd</b> , acteur <b>QuantumByte2007</b> ) : <b>sast_python</b> , <b>sast_js</b> , <b>secrets_scan</b> , <b>sca_python</b> , <b>sca_node</b> , <b>build_and_test</b> : tous <i>success</i> . . . . . | 63 |
| 7.6  | Dashboard Grafana <i>CD Pipeline</i> — <i>Deploy ECS</i> — table des derniers déploiements avec horodatage, <b>run_id</b> , <b>commit</b> et labels, acteur <b>QuantumByte2007</b> , dernière entrée le 10 juin 2026 à 23 :53 :13 . . . . .                                      | 63 |

---

|      |                                                                                                                                                                                                                                          |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 7.7  | Historique des déploiements — Succès vs Échecs sur 24h : pic d’activité entre 22h et 23h30 correspondant aux tentatives de résolution des problèmes ECS ; point vert en fin de période indiquant le premier déploiement réussi . . . . . | 64 |
| 7.8  | Panels analytiques CD — répartition globale (17% succès, 83% échecs), Push ECR (91% succès, 9% échecs), Deploy ECS (20% succès, 80% échecs / smoke test raté), succès par acteur (QuantumByte2007 : 67%, AYRHAS6 : 17%) . . . . .        | 64 |
| 7.9  | Histogrammes Push ECR vs Deploy ECS — gauche : échecs (Deploy ECS concentre la majorité) ; droite : succès (Push ECR : 10 succès, Deploy ECS : aligné sur les succès) . . . . .                                                          | 64 |
| 7.10 | Logs bruts Loki — déploiement réussi run #25 (11 juin 2026, 00 :05 :55) : commit 9c899112d7445316b723373d14a138af044574fd, push_images : success, deploy : success . . . . .                                                             | 65 |

# Introduction Générale

La sécurité logicielle est aujourd’hui au cœur des préoccupations des organisations qui développent, déploient et maintiennent des applications numériques. Dans un contexte où les cyberattaques se multiplient et se sophistiquent, la protection des systèmes ne peut plus se limiter à la sécurisation du code produit par les développeurs eux-mêmes. Elle doit s’étendre à l’ensemble des composants tiers intégrés dans les projets, qu’il s’agisse de bibliothèques open-source, de *frameworks* ou de kits de développement logiciel. Ce périmètre élargi, communément désigné sous le terme de chaîne d’approvisionnement logicielle (*software supply chain*), constitue aujourd’hui l’un des vecteurs d’attaque les plus exploités par les acteurs malveillants.

Des incidents de grande envergure ont illustré avec éloquence la criticité de cette menace. La compromission de la plateforme SolarWinds en 2020 a démontré qu’une seule mise à jour malveillante pouvait se propager silencieusement à des milliers d’organisations à travers le monde. La vulnérabilité Log4Shell, découverte en 2021 dans la bibliothèque Java Apache Log4j et affectant des centaines de milliers d’applications, a mis en évidence la fragilité inhérente aux dépendances transitives — ces composants indirectement embarqués dans un projet à l’insu même de ses développeurs. Plus récemment, l’affaire XZ Utils de 2024 a révélé la vulnérabilité des projets open-source à faibles ressources face à des attaques de type *backdoor* introduites sur le long terme. Ces événements ont conduit la communauté internationale à placer la sécurité de la chaîne d’approvisionnement logicielle au rang des priorités absolues, se traduisant notamment par des exigences réglementaires nouvelles telles que l’EU Cyber Resilience Act et le NIST Secure Software Development Framework.

Dans l’écosystème Android en particulier, cette problématique revêt une dimension supplémentaire. Un projet Android typique déclare entre cinq et dix dépendances directes, mais embarque en réalité entre cinquante et deux cents dépendances transitives dans son binaire final. Ces dépendances, souvent invisibles aux développeurs, peuvent exposer l’application à des vulnérabilités connues, à des licences incompatibles avec un usage commercial, ou à des bibliothèques de traçage collectant les données des utilisateurs à leur insu. Face à cette réalité, les outils disponibles n’offrent qu’une réponse partielle : certains se limitent à la détection de vulnérabilités sans aborder la question des licences, d’autres ne supportent pas l’analyse de fichiers APK compilés, et aucun ne propose une approche unifiée combinant analyse multicritère, génération de SBOM standards et recommandations assistées par intelligence artificielle.

C’est dans ce contexte que s’inscrit le présent projet, dont l’objet est double. D’une part, il vise à concevoir et à développer **Dependency Risk Radar** (DRR), une plateforme open-source d’audit automatisé de dépendances pour les projets Android et Java. DRR analyse chaque composant détecté selon quatre dimensions de risque complémentaires : l’exposition aux vulnérabilités connues (CVE), le retard de version par rapport aux dernières releases disponibles, le risque de contamination lié à la licence, et la présence de bibliothèques de traçage. Elle enrichit cette analyse par des données provenant de cinq sources externes reconnues : OSV.dev, le NIST National

---

Vulnerability Database, Maven Central, ClearlyDefined et Exodus Privacy. À l'issue de chaque analyse, DRR génère un Software Bill of Materials conforme aux standards CycloneDX 1.5 et SPDX 2.3, ainsi qu'un plan de mise à jour priorisé produit par un module d'intelligence artificielle s'appuyant sur Google Gemini et Anthropic Claude comme fournisseurs LLM.

D'autre part, ce projet met en œuvre un pipeline DevSecOps complet appliqué au développement et au déploiement de DRR lui-même, incarnant ainsi le principe du *shift-left security*. Il serait en effet paradoxal que cet outil d'audit de sécurité soit lui-même développé sans les rigueurs qu'il préconise. Le pipeline implémenté intègre des analyses statiques du code source (SAST), un audit automatisé des dépendances (SCA), une détection de secrets accidentellement exposés, un scan de vulnérabilités des images Docker, et une analyse dynamique de l'application déployée (DAST) via OWASP ZAP. L'ensemble de ces contrôles est orchestré par GitHub Actions et déclenché automatiquement à chaque modification du code source.

Le déploiement de DRR sur Amazon Web Services constitue le troisième axe de ce projet. L'infrastructure cible repose sur Amazon ECS pour l'orchestration des conteneurs, Amazon ECR pour le stockage des images Docker, et AWS Secrets Manager pour la gestion sécurisée des clés API. L'authentification entre GitHub Actions et AWS est assurée par le protocole OIDC, éliminant le recours à des clés d'accès statiques et réduisant ainsi significativement la surface d'attaque du pipeline de déploiement. Un système de monitoring basé sur Grafana et Loki complète l'architecture en centralisant les logs de l'ensemble du pipeline et en offrant une visibilité en temps réel sur la posture de sécurité du projet.

Ce rapport est organisé en plusieurs chapitres qui retracent les différentes étapes de ce travail. Après une présentation de l'état de l'art des outils d'analyse de dépendances et des approches DevSecOps existantes, nous décrivons l'architecture et les fonctionnalités de DRR. Nous exposons ensuite la conception et la mise en œuvre du pipeline CI/CD sécurisé, avant de détailler la configuration de l'infrastructure AWS et le processus de déploiement automatisé. Nous présentons enfin les résultats des analyses de sécurité effectuées, et nous concluons par un bilan du projet et les perspectives d'évolution envisagées.



# 1 Contexte Général

## 1.1 Introduction

La multiplication des attaques ciblant la chaîne d’approvisionnement logicielle et la complexité croissante des dépendances dans les projets Android constituent le point de départ de ce projet. Ce premier chapitre pose le cadre général en trois temps : il présente d’abord la problématique de la sécurité de la supply chain et les incidents majeurs qui ont conduit à en faire une priorité réglementaire mondiale ; il expose ensuite les spécificités de l’écosystème Android qui amplifient ces risques ; il dresse enfin l’état de l’art des outils existants et identifie les lacunes que le projet Dependency Risk Radar se propose de combler.

## 1.2 La sécurité de la chaîne d’approvisionnement logicielle

### 1.2.1 Définition et enjeux

Le développement logiciel moderne repose massivement sur la réutilisation de composants tiers. Qu’il s’agisse de bibliothèques open-source, de *frameworks*, ou de kits de développement logiciel (SDK), les applications contemporaines intègrent des dizaines voire des centaines de dépendances externes, sans que leurs développeurs en aient toujours une vision exhaustive. L’ensemble de ces composants, leurs sources, leurs chaînes de build et leurs mécanismes de distribution constituent ce que l’industrie désigne sous le terme de **chaîne d’approvisionnement logicielle** (*software supply chain*).

La sécurité de cette chaîne est devenue un enjeu stratégique majeur. Contrairement aux attaques traditionnelles qui ciblent directement le code produit par une organisation, les attaques sur la supply chain exploitent la confiance implicite accordée aux composants tiers. En compromettant une seule bibliothèque largement utilisée, un attaquant peut affecter simultanément des milliers d’applications en aval, multipliant ainsi l’impact de son action de manière exponentielle.

### 1.2.2 Incidents majeurs et prise de conscience mondiale

Plusieurs incidents de grande envergure survenus ces dernières années ont mis en lumière la criticité de cette menace et contribué à en faire une priorité absolue pour la communauté de la cybersécurité.

**L’attaque SolarWinds (2020)** constitue l’un des épisodes les plus marquants de l’histoire récente de la cybersécurité. Des attaquants ont réussi à introduire un code malveillant dans le mécanisme de mise à jour de la plateforme Orion de SolarWinds, utilisée par des milliers d’organisations gouvernementales et privées. La mise à jour

---

compromise a été distribuée et installée de manière automatique, permettant aux attaquants d'établir une présence persistante et discrète dans les systèmes de leurs victimes pendant plusieurs mois. Cet incident a révélé qu'un vecteur d'attaque aussi banal qu'une mise à jour logicielle pouvait servir de cheval de Troie à l'échelle mondiale.

**La vulnérabilité Log4Shell (2021)** a constitué une onde de choc sans précédent dans l'écosystème Java. Découverte dans la bibliothèque de journalisation Apache Log4j, présente dans d'innombrables applications d'entreprise, cette vulnérabilité critique — affectée d'un score CVSS de 10.0, le maximum possible — permettait l'exécution de code arbitraire à distance par simple injection d'une chaîne de caractères dans un champ de log. La quasi-universalité de Log4j dans l'écosystème Java a rendu ce correctif d'une complexité opérationnelle inédite, certaines organisations mettant plusieurs semaines à identifier toutes les instances affectées dans leur parc applicatif.

**L'affaire XZ Utils (2024)** a illustré un scénario différent mais tout aussi pré-occupant : une attaque de type *backdoor* introduite progressivement dans un projet open-source à faibles ressources humaines. Sur une période de deux ans, un contributeur malveillant a patiemment gagné la confiance des mainteneurs du projet avant d'introduire une porte dérobée dans l'utilitaire de compression XZ, composant présent dans de nombreuses distributions Linux. Cet incident a mis en évidence la vulnérabilité des projets open-source sous-dotés en ressources et la difficulté de détecter des attaques conduites sur le long terme.

TABLE 1.1 – Principaux incidents liés à la supply chain logicielle

| Incident   | Année | Vecteur                          | Impact                                          |
|------------|-------|----------------------------------|-------------------------------------------------|
| SolarWinds | 2020  | Mise à jour compromise           | 18 000+ organisations affectées                 |
| Log4Shell  | 2021  | Bibliothèque Java (Log4j)        | CVSS 10.0 — millions d'applications vulnérables |
| XZ Utils   | 2024  | Backdoor dans projet open-source | Distributions Linux majeures affectées          |
| Codecov    | 2021  | Script d'upload compromis        | Milliers de pipelines CI/CD exposés             |

### 1.2.3 Réponse réglementaire internationale

Face à l'ampleur de ces menaces, les instances réglementaires internationales ont commencé à imposer des exigences formelles en matière de sécurité de la supply chain :

- **EU Cyber Resilience Act (2024)** : règlement européen imposant aux fabricants et éditeurs de logiciels la génération et la maintenance d'un *Software Bill of Materials* (SBOM) pour tout produit numérique mis sur le marché européen.
- **NIST Secure Software Development Framework (SSDF)** : cadre américain définissant les pratiques recommandées pour le développement logiciel sécurisé, incluant explicitement la gestion des dépendances tierces.
- **Executive Order on Improving the Nation's Cybersecurity (2021)** : décret présidentiel américain rendant obligatoire la production de SBOM pour tout logiciel

---

vendu au gouvernement fédéral américain.

Ces initiatives convergent vers une même exigence : les organisations doivent être en mesure de produire à tout moment un inventaire exhaustif et vérifiable des composants logiciels qu'elles utilisent ou distribuent.

## 1.3 Spécificités de l'écosystème Android

### 1.3.1 La complexité des dépendances Android

L'écosystème Android présente des caractéristiques qui amplifient considérablement les risques liés à la supply chain. Un projet Android typique déclare entre cinq et dix dépendances directes dans ses fichiers de configuration Gradle. Cependant, chacune de ces dépendances directes embarque à son tour ses propres dépendances, qui embarquent les leurs, formant un graphe de dépendances transitives pouvant atteindre cent à deux cents composants dans le binaire final. Ces composants transitifs sont souvent invisibles aux développeurs et échappent aux revues de code et aux processus d'approbation habituels.

Trois vecteurs de risque coexistent spécifiquement dans les projets Android :

1. **Les dépendances Maven/Gradle déclarées** : composants explicitement référencés dans les fichiers `build.gradle`, dont les développeurs sont généralement conscients.
2. **Les dépendances transitives** : composants indirectement introduits par les dépendances directes, souvent méconnus des équipes de développement.
3. **Les SDKs tiers embarqués dans l'APK** : bibliothèques intégrées directement dans le binaire compilé, notamment les SDK publicitaires et d'analyse d'usage, dont les pratiques en matière de collecte de données sont rarement auditées.

### 1.3.2 Le problème des trackers Android

Au-delà des vulnérabilités de sécurité classiques, l'écosystème Android est confronté à une problématique spécifique : la prolifération de bibliothèques de traçage (*trackers*) embarquées dans les applications. Ces bibliothèques, intégrées à des fins d'analyse d'usage, de publicité ou de profilage, collectent des données personnelles sur les utilisateurs souvent sans leur consentement explicite ou à leur insu.

L'organisation Exodus Privacy maintient une base de données référençant plus de 300 trackers Android identifiés, classés selon leur catégorie d'activité : analytique, publicitaire, fingerprinting, géolocalisation, ou surveillance. La présence de ces trackers dans une application peut engager la responsabilité légale de l'éditeur au titre du RGPD et constituer un risque de réputation significatif.

## 1.4 État de l'art des outils d'analyse de dépendances

### 1.4.1 Outils existants

Plusieurs outils ont été développés pour adresser la problématique de la sécurité des dépendances. Chacun apporte une réponse partielle, mais aucun ne couvre l'ensemble du spectre des risques identifiés.

---

**OWASP Dependency-Check** est l'un des outils open-source les plus utilisés pour la détection de vulnérabilités dans les dépendances Java. Il s'appuie sur le matching CPE (*Common Platform Enumeration*) du NIST NVD pour identifier les composants vulnérables. Ses principales limitations dans un contexte Android sont l'absence de support natif des projets Android, un taux de faux positifs élevé lié au mécanisme de matching CPE, et l'absence de scoring multicritère intégrant la licence et les trackers.

**Snyk** est une plateforme SaaS commerciale offrant des capacités avancées d'analyse de dépendances pour les projets Java et Android. Elle supporte l'export de SBOM aux formats CycloneDX et SPDX et propose des fonctionnalités de remédiation assistée. Ses limitations principales sont son modèle commercial qui restreint les fonctionnalités avancées aux abonnements payants, l'absence d'ingestion directe de fichiers APK, et l'absence de détection de trackers.

**Dependabot** est l'outil de gestion automatique des dépendances intégré à GitHub. Il surveille les dépendances déclarées dans les fichiers de configuration et ouvre automatiquement des *pull requests* lors de la publication de nouvelles versions ou de la détection de vulnérabilités. Il ne propose pas de scoring multicritère, ne prend pas en charge l'analyse d'APK, et se limite aux dépendances directement déclarées.

**apkanalyzer** est l'outil en ligne de commande fourni par le SDK Android. Il permet d'inspecter le contenu d'un fichier APK et d'obtenir un inventaire des composants embarqués. Il ne fournit aucune information sur les vulnérabilités ou les risques associés aux composants identifiés.

**MobSF** (*Mobile Security Framework*) est un framework d'analyse statique et dynamique pour les applications mobiles Android et iOS. Il excelle dans l'analyse du code Dalvik, l'inspection du manifeste Android et l'évaluation de la configuration de sécurité binaire (NX bit, stack canary). Son périmètre est cependant centré sur le code applicatif plutôt que sur les dépendances tierces : il ne propose pas de matching CVE sur les bibliothèques, ni de génération de SBOM, ni de détection structurée de trackers.

## 1.4.2 Analyse comparative

TABLE 1.2 – Comparaison des outils d’analyse de dépendances existants

| Fonctionnalité            | DRR | MobSF   | Dep-Check | Snyk    | apkanalyzer |
|---------------------------|-----|---------|-----------|---------|-------------|
| Ingestion APK             | ✓   | ✓       | ✗         | Partiel | ✓           |
| Ingestion Gradle          | ✓   | ✗       | ✓         | ✓       | ✗           |
| Détection CVE dépendances | ✓   | Partiel | ✓         | ✓       | ✗           |
| Graphe transitif          | ✓   | ✗       | Partiel   | ✓       | ✗           |
| Scoring multicritère      | ✓   | ✗       | ✗         | ✗       | ✗           |
| Détection trackers        | ✓   | Partiel | ✗         | ✗       | ✗           |
| Conformité licence        | ✓   | ✗       | ✗         | ✓       | ✗           |
| Export SBOM CycloneDX     | ✓   | ✗       | ✓         | ✓       | ✗           |
| Export SBOM SPDX          | ✓   | ✗       | ✗         | ✓       | ✗           |
| Plan IA priorisé          | ✓   | ✗       | ✗         | Partiel | ✗           |
| Self-hosted open-source   | ✓   | ✓       | ✓         | ✗       | ✓           |
| Dashboard interactif      | ✓   | ✓       | ✗         | ✓       | ✗           |

## 1.4.3 Lacunes identifiées

L’analyse comparative révèle une lacune significative dans le paysage des outils disponibles. Aucun outil open-source existant ne combine simultanément :

- L’ingestion de fichiers Gradle *et* d’APK compilés, couvrant ainsi les deux principales modalités d’accès à un projet Android ;
- Un scoring multicritère intégrant CVE, obsolescence, licence et trackers en un score global actionnable ;
- La génération de SBOM conformes aux deux standards industriels de référence (CycloneDX et SPDX) ;
- Un plan de mise à jour priorisé et contextuel produit par un module d’intelligence artificielle.

C’est précisément cet espace fonctionnel que **Dependency Risk Radar** se propose d’occuper.

## 1.5 Présentation du projet Dependency Risk Radar

### 1.5.1 Vue d’ensemble

**Dependency Risk Radar** (DRR) est une plateforme open-source d’audit automatisé de dépendances pour les projets Android et Java, développée dans le cadre du présent projet académique. Elle se distingue des solutions existantes par son approche

---

intégrée et multicritère de l'analyse des risques, combinant en une seule analyse : la détection de vulnérabilités, l'évaluation de l'obsolescence, la vérification de la conformité de licence, et la détection de bibliothèques de traçage.

[Positionnement de DRR] DRR est la seule plateforme open-source et self-hosted qui combine simultanément l'ingestion de projets Gradle et d'APK compilés, un scoring multicritère sur quatre dimensions de risque, la génération de SBOM conformes aux standards CycloneDX 1.5 et SPDX 2.3, et un plan de mise à jour priorisé par intelligence artificielle — le tout en moins de 60 secondes.

## 1.5.2 Fonctionnalités principales

DRR offre les fonctionnalités suivantes :

- **Analyse Gradle** : parsing des fichiers `build.gradle` en syntaxe Groovy DSL et Kotlin DSL, extraction des dépendances directes et transitives, support des catalogues de versions (`libs.versions.toml`).
- **Analyse APK** : extraction des composants depuis le bytecode Dalvik compilé via la bibliothèque `androguard`, permettant l'audit de binaires sans accès au code source.
- **Enrichissement multicritère** : interrogation parallèle de cinq sources de données externes (OSV.dev, NIST NVD, Maven Central, ClearlyDefined, Exodus Privacy) pour chaque composant détecté.
- **Scoring pondéré** : calcul d'un score global normalisé sur 100 selon la formule :  
$$\mathcal{R} = 0,45 \cdot S_{\text{CVE}} + 0,25 \cdot S_{\text{obs}} + 0,20 \cdot S_{\text{lic}} + 0,10 \cdot S_{\text{track}}$$
- **Propagation transitive** : modélisation des dépendances sous forme de graphe orienté acyclique (DAG) via `NetworkX`, avec propagation des scores de risque atténuée par la profondeur dans le graphe.
- **Génération SBOM** : production automatique de Software Bills of Materials en formats CycloneDX 1.5 (JSON/XML) et SPDX 2.3 (JSON/tag-value).
- **Plan IA** : envoi des 30 composants les plus risqués à un LLM (Google Gemini en priorité, Anthropic Claude en fallback) pour génération d'un plan de mise à jour priorisé avec estimation de l'effort de migration et évaluation du risque de rupture de compatibilité.
- **Dashboard interactif** : interface React 18 avec visualisation D3.js du graphe de dépendances, tableau de composants filtrable et triable, et explorateur SBOM intégré.
- **Export PDF** : génération de rapports PDF complets pour les audits de sécurité et les revues de conformité.

## 1.5.3 Architecture en couches

DRR est structuré en quatre couches découplées communicant via des interfaces bien définies, comme illustré dans le tableau suivant :

TABLE 1.3 – Architecture en couches de DRR

| N° | Couche       | Responsabilité                                                           | Technologies                   |
|----|--------------|--------------------------------------------------------------------------|--------------------------------|
| 1  | Ingestion    | Parsing Gradle, analyse APK, appels API externes parallèles              | Python, androguard, httpx      |
| 2  | Analyse      | Scoring multicritère, construction et propagation du graphe DAG          | Python, NetworkX, pydantic     |
| 3  | IA           | Génération du plan de mise à jour, narratives techniques et managériales | Gemini Flash, Anthropic Claude |
| 4  | Présentation | Dashboard interactif, API REST, génération SBOM et PDF                   | FastAPI, React 18, D3.js       |

## 1.6 Objectifs du projet

Le présent projet s’articule autour de trois objectifs complémentaires et interdépendants.

Le **premier objectif** est le développement de la plateforme DRR elle-même : conception de l’architecture logicielle, implémentation des modules d’ingestion, d’enrichissement, de scoring et de génération de rapports, développement de l’interface utilisateur, et intégration du module d’intelligence artificielle. L’accent est mis sur la qualité du code, la couverture de tests, et la conformité aux standards industriels pour les formats de sortie (CycloneDX, SPDX).

Le **deuxième objectif** est la mise en place d’un pipeline DevSecOps complet appliqué au développement de DRR. Ce pipeline implémente le principe du *shift-left security* en intégrant des contrôles de sécurité automatisés à chaque étape du cycle de développement : analyses statiques du code source, audit des dépendances du projet lui-même, scan de vulnérabilités des images Docker, et analyse dynamique de l’application déployée. Ce faisant, le projet démontre que DRR pratique ce qu’il prêche : les mêmes exigences de sécurité appliquées aux projets audités sont appliquées au développement de DRR.

Le **troisième objectif** est le déploiement automatisé de DRR sur Amazon Web Services. L’infrastructure cible s’appuie sur les services managés d’AWS pour garantir la disponibilité, la sécurité et la maintenabilité de la plateforme : Amazon ECS pour l’orchestration des conteneurs, Amazon ECR pour le stockage des images Docker, et AWS Secrets Manager pour la gestion sécurisée des clés API. Le protocole OIDC est utilisé pour l’authentification entre GitHub Actions et AWS, éliminant le recours à des clés d’accès statiques dans le pipeline de déploiement. Un système de monitoring basé sur Grafana et Loki assure la visibilité en temps réel sur la posture de sécurité et la santé opérationnelle du système déployé.

[Contribution du projet] Ce projet apporte une contribution sur trois plans complémentaires : **fonctionnel**, en comblant un vide dans le paysage des outils open-source

---

d'analyse de dépendances Android ; **méthodologique**, en démontrant l'applicabilité du pipeline DevSecOps à un projet académique réel ; et **pédagogique**, en illustrant concrètement les interactions entre développement logiciel, pratiques de sécurité, et infrastructure cloud.

## Conclusion

Ce chapitre a établi le contexte dans lequel s'inscrit le projet. Les incidents SolarWinds, Log4Shell et XZ Utils ont démontré que la chaîne d'approvisionnement logicielle constitue un vecteur d'attaque de premier plan, conduisant la communauté internationale à imposer de nouvelles exigences réglementaires en matière de SBOM et de gestion des dépendances. Dans l'écosystème Android spécifiquement, la complexité des graphes de dépendances transitives et la prolifération des trackers ajoutent deux dimensions de risque supplémentaires que les outils existants n'adressent qu'partiellement. L'analyse comparative a mis en évidence un espace fonctionnel non couvert : aucun outil open-source ne combine simultanément l'ingestion de projets Gradle et d'APK compilés, un scoring multicritère, la génération de SBOM standards, et une remédiation assistée par IA. C'est cet espace que Dependency Risk Radar se propose d'occuper, dont l'architecture et les fonctionnalités sont présentées au chapitre 3.



## 2 Cadre Théorique et État de l'Art

### 2.1 Introduction

Avant de décrire les choix d'implémentation de DRR et de son pipeline DevSecOps, il est nécessaire d'établir les fondements théoriques sur lesquels repose l'ensemble du projet. Ce deuxième chapitre présente les concepts et outils qui constituent le socle technologique et méthodologique du travail réalisé. Il couvre successivement le système de build Gradle et le format APK Android, les quatre dimensions théoriques de l'analyse des risques logiciels (CVE/CVSS, obsolescence, licences, trackers), la philosophie DevSecOps et le principe du shift-left, les grandes familles d'analyse de sécurité mobilisées dans le pipeline (SAST, secrets scanning, SCA, scan de conteneurs, observabilité), l'infrastructure cloud AWS avec ECS et ECR, et enfin le protocole d'authentification fédérée OIDC.

### 2.2 Le système de build Gradle

Gradle est un outil de build automation open source utilisé pour automatiser la compilation, les tests et la gestion des dépendances d'un projet logiciel. Sa configuration repose sur un fichier `build.gradle`, écrit en Groovy ou Kotlin, dans lequel on déclare les dépendances, les plugins et les tâches du projet. Contrairement à Maven, Gradle adopte une approche incrémentale : il ne recompile que les modules modifiés depuis le dernier build, ce qui optimise les temps de construction. Il est aujourd'hui l'outil de build officiel de l'écosystème Android et est largement adopté dans les projets Java et Kotlin.

### 2.3 L'APK Android

Un **APK** (*Android Package*) est le format de fichier de distribution d'une application Android. Il s'agit d'une archive ZIP dont le contenu est analysé lors de l'installation sur un appareil Android. Un APK contient plusieurs éléments d'intérêt pour l'analyse de sécurité : le fichier **AndroidManifest.xml** qui déclare les métadonnées de l'application (nom du package, version, permissions requises, composants exposés), un ou plusieurs fichiers **classes.dex** contenant le bytecode compilé au format DEX (*Dalvik Executable*), le format propre à la machine virtuelle Android, ainsi que le répertoire **lib/** contenant les bibliothèques natives compilées (**.so**) pour différentes architectures processeur. L'analyse statique d'un APK consiste à inspecter ce contenu sans exécuter l'application, en désassemblant le bytecode DEX pour extraire les noms de packages et de classes Java présents. Cette approche permet d'identifier les bibliothèques tierces effectivement embarquées dans le binaire final, y compris celles qui n'apparaissent pas dans les fichiers Gradle — comme les SDKs publicitaires intégrés en dehors du gestionnaire de dépendances standard.

---

## 2.4 Fondements théoriques de l’analyse des risques logiciels

### 2.4.1 Les vulnérabilités logicielles : CVE et CVSS

Une **CVE** (*Common Vulnerabilities and Exposures*) est un identifiant unique attribué à une faille de sécurité connue publiquement dans un logiciel ou une bibliothèque, de la forme **CVE-ANNÉE-NUMÉRO**. Ce système, maintenu par la MITRE Corporation, constitue le référentiel mondial de recensement des vulnérabilités. Chaque entrée CVE documente la nature de la faille, les versions affectées et les références aux correctifs disponibles.

Le **CVSS** (*Common Vulnerability Scoring System*), dans sa version 3.1, est le standard industriel de quantification de la sévérité d’une vulnérabilité. Il attribue un score de 0 à 10 calculé à partir de métriques décrivant le vecteur d’attaque (réseau, adjacent, local, physique), la complexité de l’exploitation, les privilèges requis, la nécessité d’une interaction utilisateur, et les impacts sur la confidentialité, l’intégrité et la disponibilité du système ciblé. L’interprétation qualitative standardisée est la suivante :

| Score CVSS | Niveau de sévérité |
|------------|--------------------|
| 9.0 – 10.0 | Critique           |
| 7.0 – 8.9  | Haut               |
| 4.0 – 6.9  | Moyen              |
| 0.1 – 3.9  | Bas                |

TABLE 2.1 – Niveaux de sévérité CVSS v3.1

La notion de *has\_fix* — c’est-à-dire l’existence d’une version corrigée disponible — est une dimension complémentaire essentielle : une CVE sans correctif impose des stratégies de mitigation alternatives telles que l’isolation du composant, son remplacement ou l’application d’un patch manuel.

### 2.4.2 L’obsolescence des dépendances

L’obsolescence d’une dépendance logicielle désigne l’état dans lequel un composant utilisé dans un projet est en retard par rapport à l’évolution de son écosystème. Elle se manifeste selon deux axes complémentaires.

Le premier est le **retard de version sémantique**. La gestion sémantique des versions (SemVer) impose le format **MAJOR.MINOR.PATCH** avec une sémantique précise : une incrémentation de version majeure signale des changements incompatibles, une version mineure apporte de nouvelles fonctionnalités rétro-compatibles, et un patch corrige des bogues. Du point de vue de la sécurité, un retard de version majeure est particulièrement critique car les correctifs de sécurité des anciennes versions majeures ne sont généralement plus backportés par les mainteneurs. Le second axe est l’**abandon du projet** : une bibliothèque dont la dernière release date de plusieurs années ne recevra plus de correctif pour de nouvelles vulnérabilités découvertes, et sa compatibilité avec les évolutions de la plateforme Android n’est plus garantie. L’obsolescence constitue donc un risque indirect mais structurel : elle accroît la probabilité que des vulnérabilités futures ne soient jamais corrigées dans le composant concerné.

---

### 2.4.3 Les licences logicielles

Une **licence logicielle** est le contrat juridique qui régit les droits et obligations liés à l'utilisation, la modification et la redistribution d'un logiciel. Dans le contexte des dépendances open source, le choix de la licence d'une bibliothèque tierce peut avoir des conséquences légales importantes sur l'application qui l'intègre.

On distingue trois grandes familles de licences. Les licences **permissives** (MIT, Apache-2.0, BSD) n'imposent pratiquement aucune obligation et autorisent l'intégration dans des logiciels commerciaux propriétaires sans contrainte de redistribution du code source. Les licences **copyleft faible** (LGPL, MPL-2.0) imposent que les modifications apportées à la bibliothèque elle-même soient publiées sous la même licence, mais n'affectent pas le code applicatif environnant. Les licences **copyleft fort** (GPL-2.0, GPL-3.0) appliquent le principe de contamination virale : tout logiciel lié à un composant GPL doit être distribué sous GPL avec son code source. L'AGPL-3.0 étend cette obligation aux services accessibles via un réseau. Dans le contexte d'une application Android commerciale, la présence d'une dépendance GPL dans le binaire distribué constitue une violation potentielle imposant soit la publication du code source de l'application, soit le remplacement de la dépendance concernée. Une licence inconnue ou non identifiée représente également un risque juridique non évaluable.

### 2.4.4 Les trackers et les permissions Android

Les **trackers** sont des SDKs tiers intégrés dans les applications Android pour collecter des données comportementales, analytiques ou publicitaires, souvent à l'insu de l'utilisateur. Ils sont identifiables par leur signature de package Java caractéristique (ex. `com.google.firebase.analytics`) et transmettent des données à des serveurs distants. La base de données Exodus Privacy en recense plus de 300, classés selon leur finalité : analytique, publicitaire, fingerprinting, localisation ou support client.

Les **permissions Android** constituent la seconde dimension de ce risque. Déclarées dans le fichier `AndroidManifest.xml`, elles définissent les accès aux ressources sensibles du système qu'une application demande à l'utilisateur. Certaines permissions sont particulièrement intrusives et constituent des signaux de risque forts pour la vie privée : `ACCESS_FINE_LOCATION` (localisation GPS précise), `RECORD_AUDIO` (accès au microphone), `READ_SMS` (lecture des messages), ou `READ_CONTACTS` (accès aux contacts). La combinaison d'un tracker de fingerprinting et de permissions sensibles dans une même dépendance représente un risque élevé au regard des réglementations sur la protection des données personnelles, notamment le RGPD en Europe.

## 2.5 DevSecOps : philosophie et fondements

### 2.5.1 Origine et définition

Le mouvement **DevOps**, apparu à la fin des années 2000, a profondément transformé l'ingénierie logicielle en rapprochant les équipes de développement (*Dev*) et d'exploitation (*Ops*). L'objectif était d'automatiser la chaîne de livraison du logiciel afin de réduire les cycles de déploiement, d'améliorer la fiabilité et de favoriser une culture de responsabilité partagée.

Le **DevSecOps** étend ce modèle en intégrant la sécurité (*Sec*) dès les premières phases du cycle de vie du logiciel, selon le principe du *Shift-Left Security* : plutôt que de traiter la sécurité comme une étape de validation finale, elle devient une préoccupation continue, portée par l'ensemble des acteurs du projet. Gartner définit le DevSecOps comme « l'intégration de la sécurité dans les processus DevOps de façon à ce qu'elle soit transparente, automatisée et omniprésente » [?].

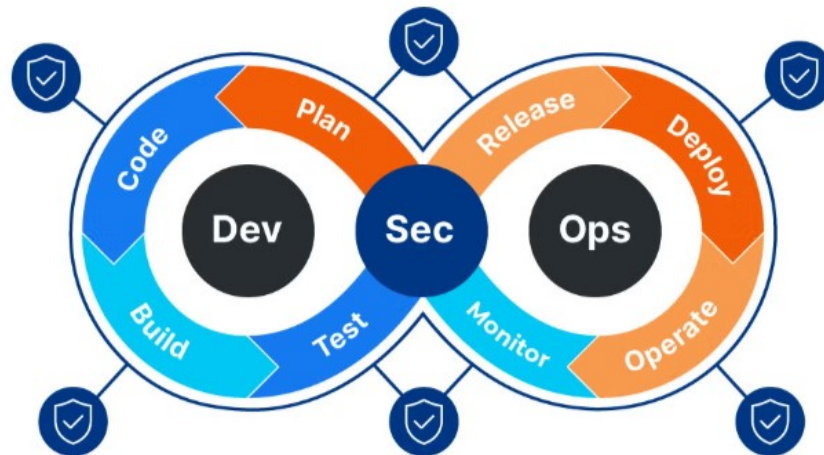


FIGURE 2.1 – Approche DevSecOps.

## 2.5.2 Principe du Shift-Left

La notion de *Shift-Left* (??) désigne le déplacement des contrôles de sécurité vers les étapes amont du développement. Ce déplacement est motivé par un constat économique bien documenté : le coût de correction d'une vulnérabilité croît exponentiellement avec le temps de détection. Corriger une faille identifiée au moment du commit coûte entre dix et cent fois moins cher que la même correction effectuée après mise en production.

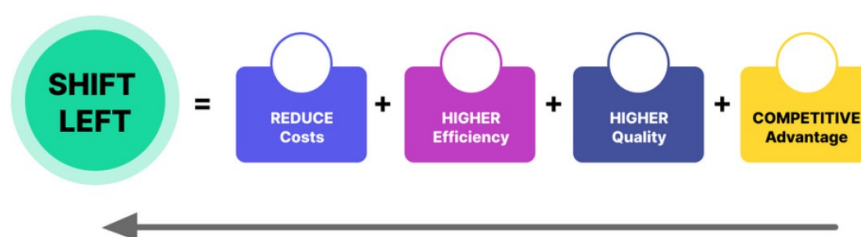


FIGURE 2.2 – Principe du Shift-Left : détection précoce et coût de correction.

## 2.5.3 Piliers du DevSecOps

La pratique du DevSecOps repose sur trois piliers complémentaires :

**Automatisation continue.** Chaque action de sécurité — analyse de code, audit de dépendances, scan d'images — est exécutée automatiquement à chaque événement du dépôt (commit, pull request, fusion). L'humain reste décisionnaire, mais les vérifications ne reposent plus sur sa vigilance ponctuelle.

---

**Feedback rapide.** Les résultats des contrôles sont retournés au développeur dans le contexte de son travail immédiat. Un rapport d’analyse statique visible directement dans la pull request réduit le coût cognitif de la correction.

**Responsabilité collective.** La sécurité n’est plus l’apanage d’une équipe dédiée. Elle est portée par l’ensemble de la chaîne : développeurs, ingénieurs d’infrastructure, équipes QA. Cette culture partagée est considérée comme la condition nécessaire à l’efficacité des outils.

## 2.6 Les grandes familles d’analyse de sécurité

Le pipeline étudié met en œuvre quatre grandes catégories d’analyse de sécurité logicielle. Chacune adresse un vecteur d’attaque distinct et se positionne à un moment précis du cycle de vie de l’artefact.

### 2.6.1 Analyse statique de sécurité applicative (SAST)

#### Définition et périmètre

L’analyse statique de sécurité applicative (*Static Application Security Testing*, SAST) consiste à inspecter le code source, le bytecode ou le code binaire d’une application *sans l’exécuter*. L’objectif est d’identifier des patterns de code connus pour introduire des vulnérabilités : injections, gestion incorrecte de la mémoire, usage de fonctions dépréciées, mauvaise configuration des paramètres de sécurité, etc.

#### Fondements théoriques

Le SAST s’appuie sur deux approches complémentaires :

- **Analyse de flot de données (*taint analysis*).** Cette technique trace le cheminement des données non fiables (*sources*) jusqu’aux points d’utilisation sensibles (*sinks*). Si un flux non nettoyé atteint une fonction d’exécution de commande système, l’outil signale une injection potentielle.
- **Reconnaissance de patterns (*pattern matching*).** Les outils maintiennent des bases de règles associant des constructions syntaxiques à des vulnérabilités connues (référéncées dans les catalogues CWE et CVE). Cette approche est plus rapide mais génère davantage de faux positifs.

#### Place dans le pipeline

Dans le pipeline étudié, deux instances SAST s’exécutent en parallèle dès la réception d’un événement `push` ou `pull_request` : l’une cible la base de code Python du backend, l’autre la base TypeScript/React du frontend. Cette parallélisation maximise la vélocité sans sacrifier la couverture.

---

## 2.6.2 Détection de secrets (*Secrets Scanning*)



FIGURE 2.3 – Secret Scanning.

La détection de secrets est une forme spécialisée d’analyse statique ciblant les informations d’authentification codées en dur dans les dépôts : clés d’API, tokens OAuth, mots de passe ou certificats privés. La présence de tels secrets dans l’historique Git constitue l’une des causes les plus fréquentes de compromission de systèmes cloud. Les outils spécialisés combinent deux mécanismes complémentaires : les **expressions régulières et patterns sémantiques**, qui couvrent des centaines de formats propriétaires (AWS, GCP, GitHub, Stripe, etc.), et l’**analyse entropique**, qui signale les chaînes aléatoires de haute entropie apparaissant sans contexte justifiable. Contrairement au SAST qui porte sur l’état courant du code, ce scan doit couvrir l’intégralité de l’historique Git, car un secret commité puis supprimé reste accessible à quiconque clone le dépôt — ce qui justifie la configuration `fetch-depth: 0` dans le pipeline.

### Place dans le pipeline

Le scan de secrets s’exécute en parallèle des deux étapes SAST, avant toute étape de build. Sa présence dès l’étape 2 du pipeline garantit qu’aucun secret ne peut atteindre les registres d’images ou les environnements de déploiement.

## 2.6.3 Analyse de la composition logicielle (SCA)

### Définition et périmètre

L’analyse de la composition logicielle (*Software Composition Analysis*, SCA) s’intéresse non pas au code produit par l’équipe, mais aux dépendances tierces qu’il embarque. Dans un projet moderne, la part de code open source peut dépasser 80 % de la surface totale [?]. Chaque dépendance constitue un vecteur de vulnérabilité potentiel.

### Fondements théoriques

Le SCA procède en trois étapes logiques :

1. **Inventaire (*Software Bill of Materials*, SBOM).** L’outil construit la liste exhaustive des composants utilisés, directs et transitifs, avec leurs versions exactes.
2. **Corrélation avec les bases de vulnérabilités.** Chaque composant est mis en correspondance avec les bases de données publiques (NVD, OSV, GitHub Advisory) et privées pour identifier les CVE (*Common Vulnerabilities and Exposures*) connues.

3. **Évaluation de la criticité.** Le score CVSS (*Common Vulnerability Scoring System*) quantifie la sévérité de chaque vulnérabilité sur une échelle de 0 à 10, selon des critères standardisés par le NIST [?] : vecteur d’attaque, complexité d’exploitation, privilèges requis, impact en confidentialité/intégrité/disponibilité.

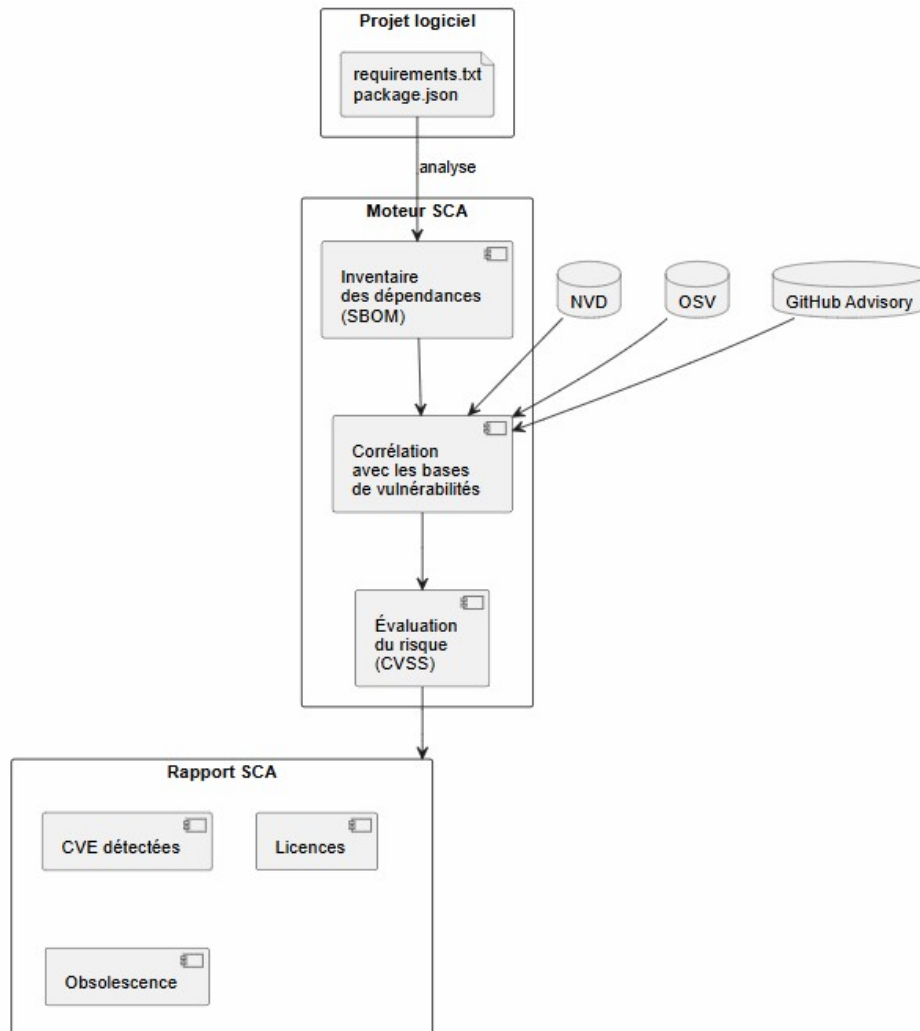


FIGURE 2.4 – SCA architecture.

### Place dans le pipeline

Deux instances SCA s’exécutent en parallèle : l’une audite les dépendances Python via le fichier `requirements.txt`, l’autre audite les dépendances Node.js. Elles s’exécutent après les étapes SAST et secrets, et conditionnent le déclenchement de l’étape de build.

## 2.6.4 Scan de vulnérabilités des images de conteneurs

### Définition et motivations

Les images Docker sont des artefacts autonomes qui encapsulent le système d’exploitation, les runtimes, les bibliothèques système et le code applicatif. Contrairement aux audits de dépendances applicatives (SCA), le scan d’image inspecte l’ensemble de

la couche OS : bibliothèques C, paquets système, utilitaires inclus dans l'image de base. Une image basée sur `ubuntu:22.04` peut embarquer des dizaines de paquets système vulnérables indépendamment de l'application qu'elle héberge.

## Fondements théoriques

Le scan d'image fonctionne en deux phases :

- **Extraction du manifeste.** L'outil décompresse les couches de l'image et extrait la liste de tous les paquets installés (format `dpkg`, `rpm`, `apk`, `pip`, `npm`, etc.) ainsi que leurs métadonnées de version.
- **Corrélation multi-bases.** Le manifeste est croisé avec les bases de vulnérabilités OS spécifiques (Debian Security Tracker, Red Hat CVE Database, Alpine secdb) et les bases génériques (NVD, OSV). Les outils modernes appliquent un filtrage *ignore-unfixed* pour exclure les CVE sans correctif disponible, réduisant ainsi le bruit.

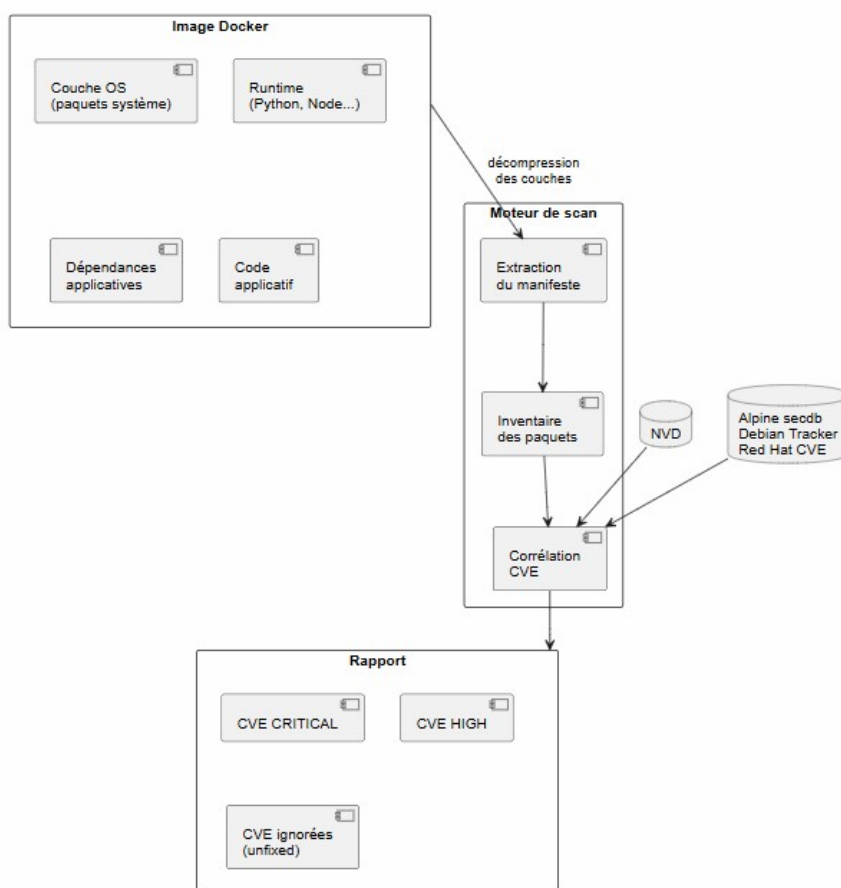


FIGURE 2.5 – Flux du scan de l'image Docker.

## Place dans le pipeline

Le scan de conteneurs intervient à l'étape 5, après la construction effective des images Docker. Il ne peut logiquement pas s'exécuter avant, puisque son objet est



---

l'artefact produit. Cette dépendance séquentielle est modélisée par la clause `needs: [build-and-test]` dans la configuration du pipeline.

## 2.7 Observabilité du pipeline

### Définition et enjeux

L'observabilité du pipeline CI/CD est une pratique émergente qui étend les principes de l'observabilité des systèmes distribués (*logs*, *metrics*, *traces*) à la chaîne d'intégration continue elle-même. L'objectif est de produire une traçabilité complète des exécutions : quels contrôles ont réussi ou échoué, pour quel commit, déclenché par quel acteur, à quelle heure.

### Fondements théoriques

L'envoi structuré des résultats vers un système de stockage de logs (*log aggregation*) permet de :

- construire des tableaux de bord de tendance (taux d'échec par étape, évolution du taux de couverture) ;
- déclencher des alertes sur des dégradations de la posture de sécurité ;
- constituer une piste d'audit.

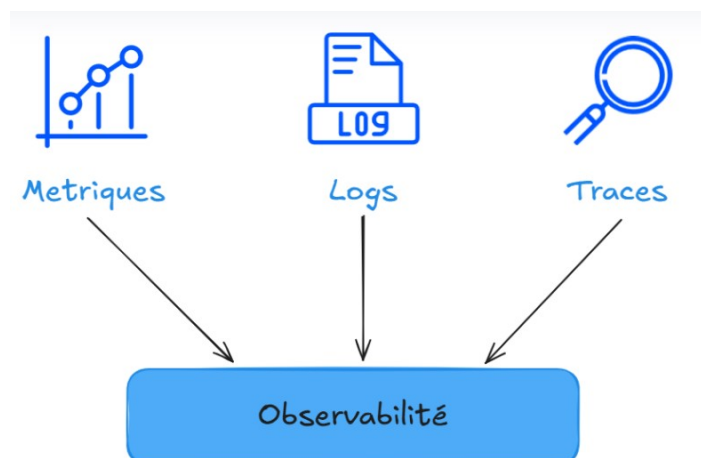


FIGURE 2.6 – Observabilité du pipeline.

### Place dans le pipeline

L'étape d'observabilité s'exécute *après toutes les autres*, avec la condition `if: always()`, garantissant l'envoi des logs même en cas d'échec d'une étape précédente. Elle consolide dans un unique événement JSON structuré le résultat de l'ensemble des six étapes, en l'associant aux métadonnées du déclencheur (identifiant de run, numéro de commit, branche, acteur).

---

## 2.8 Amazon Web Services : infrastructure cloud et orchestration de conteneurs

Amazon Web Services (AWS) est la plateforme de cloud computing d'Amazon, leader mondial du marché, offrant une infrastructure à la demande couvrant le calcul, le stockage, la mise en réseau et la sécurité. Dans le cadre d'un déploiement conteneurisé, AWS s'appuie sur un écosystème de services complémentaires. Amazon Elastic Container Registry (ECR) est un registre d'images Docker entièrement managé, permettant de stocker, versionner et distribuer des images de conteneurs de manière sécurisée, avec analyse intégrée de vulnérabilités et gestion automatisée du cycle de vie des images. Amazon Elastic Container Service (ECS) est le service d'orchestration de conteneurs d'AWS : il prend en charge le déploiement, la planification et la supervision des conteneurs à partir des images issues d'ECR, en s'appuyant sur des *task definitions* qui formalisent les paramètres d'exécution — ressources CPU et mémoire, variables d'environnement, politique de journalisation et mappings de ports. ECS garantit la haute disponibilité des services en redémarrant automatiquement les conteneurs défectueux et en orchestrant les mises à jour sans interruption. Le substrat de calcul est assuré par Amazon Elastic Compute Cloud (EC2), qui fournit des instances virtuelles configurables constituant les nœuds du cluster ECS. Enfin, AWS Identity and Access Management (IAM) régit l'ensemble des droits d'accès au sein de l'infrastructure : il permet de définir des politiques d'autorisation granulaires selon le principe du moindre privilège, en attribuant à chaque composant — pipeline CI/CD, service ECS, instances EC2 — uniquement les permissions strictement nécessaires à son rôle, limitant ainsi la surface d'attaque de l'ensemble du système.

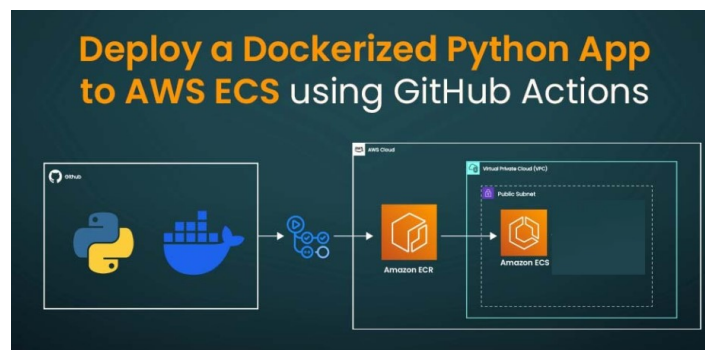


FIGURE 2.7 – Pipeline de déploiement conteneurisé vers AWS via GitHub Actions.

## 2.9 Authentification fédérée entre le pipeline et l'infrastructure cloud

La communication entre un pipeline CI/CD et l'infrastructure cloud soulève une problématique de sécurité fondamentale : comment permettre au pipeline d'agir sur les ressources cloud sans lui confier des identifiants statiques susceptibles d'être exposés ou compromis ? La réponse apportée dans ce projet repose sur le principe de **l'identité fédérée**, une approche dans laquelle le pipeline ne possède aucun secret à long terme, mais prouve son identité à chaque exécution grâce à un **jeton d'identité signé** émis par la plateforme d'intégration continue. Ce jeton, valable uniquement pour la

durée de l'exécution et lié à des attributs précis de celle-ci (dépôt, branche, événement déclencheur), est présenté à AWS qui le vérifie auprès de l'émetteur de confiance avant d'accorder des droits temporaires strictement limités. AWS émet alors des **credentials éphémères** — valables quelques minutes — que le pipeline utilise pour pousser une image vers ECR ou déclencher un déploiement ECS. À l'expiration de ces credentials, tout accès cesse automatiquement. Cette approche élimine structurellement le risque lié au stockage de clés d'accès permanentes : il n'existe aucun secret à voler, aucun identifiant à faire tourner, et la surface d'attaque du pipeline est réduite à sa portion congrue. La granularité des droits accordés est également maîtrisée par une politique IAM qui restreint les actions autorisées au strict nécessaire pour le déploiement, selon le principe du moindre privilège.

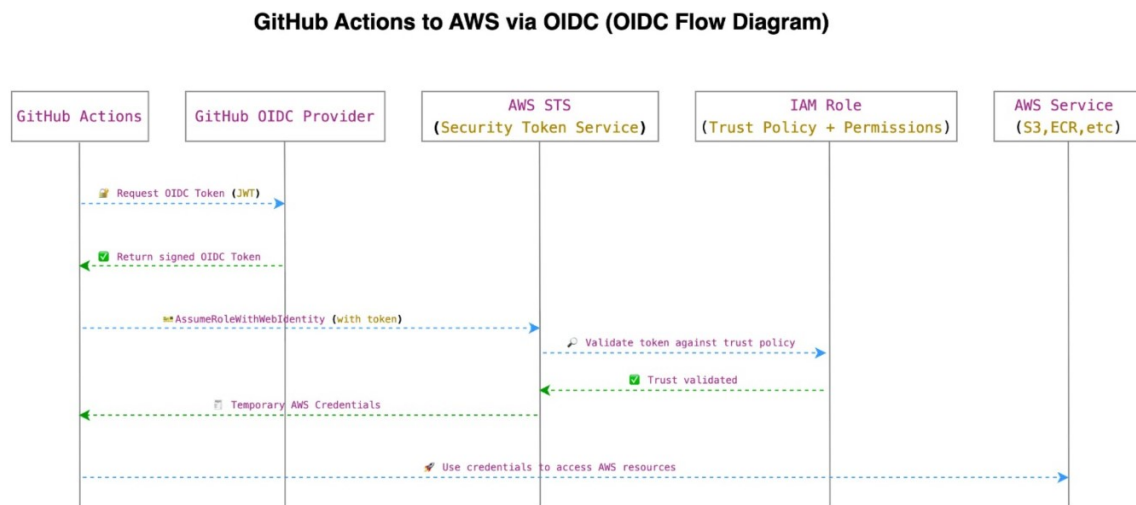


FIGURE 2.8 – Authentification fédérée.

## Conclusion

Ce chapitre a posé les fondations théoriques et technologiques du projet. Les concepts de CVE, CVSS, obsolescence sémantique, risque de licence et détection de trackers définissent les quatre axes du moteur de scoring de DRR. La philosophie DevSecOps et le principe du shift-left justifient l'approche adoptée : intégrer la sécurité comme contrainte continue du développement plutôt que comme étape de validation finale. Les grandes familles d'analyse — SAST, SCA, scan de conteneurs et observabilité — correspondent directement aux huit portes de sécurité du pipeline CI implémenté. Enfin, les services AWS (ECR, ECS, IAM, Secrets Manager) et le protocole OIDC constituent les briques de l'infrastructure de déploiement détaillée au chapitre 5. Les technologies présentées dans le chapitre suivant s'appuient directement sur ces fondements théoriques.

## 3 Technologies et Outils Utilisés

### 3.1 Introduction

Ce chapitre présente les technologies et outils mobilisés dans le cadre du projet, organisés selon les trois flux principaux qui structurent l'architecture : l'intégration et la livraison continues, l'infrastructure cloud, et l'observabilité.

### 3.2 Flux 1 — Intégration et livraison continues

#### 3.2.1 GitHub Actions

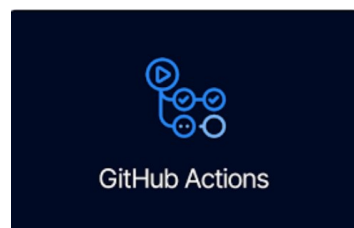


FIGURE 3.1 – Logo GitHub Actions.

GitHub Actions est la plateforme d'intégration et de livraison continues intégrée nativement à GitHub. Elle permet de définir des workflows automatisés déclenchés par des événements du dépôt — push, pull request, création de tag — sous forme de fichiers YAML versionnés au sein du projet. Chaque workflow est composé de jobs s'exécutant sur des runners hébergés par GitHub, pouvant s'exécuter séquentiellement ou en parallèle selon les dépendances déclarées. Dans ce projet, GitHub Actions orchestre l'ensemble du pipeline DevSecOps : analyses de sécurité, tests, construction des images Docker et déploiement sur AWS.

---

### 3.2.2 Docker



FIGURE 3.2 – Logo Docker.

Docker est la plateforme de conteneurisation standard de l'industrie. Elle permet d'empaqueter une application et l'ensemble de ses dépendances dans une image portable et reproductible, dont l'exécution est garantie identique quel que soit l'environnement cible. Chaque composant de DRR — backend FastAPI et frontend React — est encapsulé dans une image Docker construite par le pipeline, taguée avec le hash du commit déclencheur, puis publiée dans un registre avant déploiement. L'utilisation de Docker assure la parité entre les environnements de développement, de test et de production.

## 3.3 Flux 2 — Infrastructure cloud AWS



FIGURE 3.3 – Logo Amazon Web Services.

Amazon Web Services est la plateforme cloud sur laquelle DRR est déployé. Quatre services AWS sont mobilisés de manière complémentaire pour assurer le déploiement, la sécurité et l'exécution de l'application.

### 3.3.1 Amazon ECR — Elastic Container Registry



FIGURE 3.4 – Logo Amazon ECR.

---

Amazon Elastic Container Registry est le service de registre d'images Docker managé d'AWS. Il stocke les images construites par le pipeline CI/CD et les met à disposition d'ECS lors de chaque déploiement. ECR intègre nativement un scan de vulnérabilités des images stockées et applique des politiques de cycle de vie permettant de purger automatiquement les images obsolètes.

### 3.3.2 Amazon ECS — Elastic Container Service



FIGURE 3.5 – Logo Amazon ECS.

Amazon Elastic Container Service est le service d'orchestration de conteneurs d'AWS. Il déploie et supervise les conteneurs à partir des images stockées dans ECR, selon les paramètres définis dans une *task definition* : ressources CPU et mémoire, variables d'environnement, politique de journalisation et mappings de ports. ECS garantit la disponibilité des services en redémarrant automatiquement tout conteneur défaillant et en assurant les mises à jour sans interruption de service.

### 3.3.3 Amazon EC2 — Elastic Compute Cloud



FIGURE 3.6 – Logo Amazon EC2.

Amazon Elastic Compute Cloud fournit la capacité de calcul sur laquelle s'exécutent les conteneurs ECS. Dans ce projet, les instances EC2 constituent les nœuds du cluster ECS, hébergeant physiquement les conteneurs du backend et du frontend de DRR. Le choix d'EC2 comme substrat de calcul offre une maîtrise fine des ressources allouées et de la configuration réseau des instances.

### 3.3.4 AWS IAM — Identity and Access Management



FIGURE 3.7 – Logo AWS IAM.

---

AWS Identity and Access Management est le service de gestion des identités et des droits d'accès d'AWS. Il permet de définir des politiques d'autorisation granulaires spécifiant précisément quelles actions sont permises sur quelles ressources. Dans ce projet, IAM joue un rôle central dans la sécurisation du pipeline de déploiement : un rôle IAM dédié, assumé de manière éphémère par le pipeline via authentification fédérée, restreint les droits au strict nécessaire — pousser une image dans ECR et déclencher une mise à jour de service ECS — selon le principe du moindre privilège.

### 3.3.5 OpenID Connect — Authentification fédérée



FIGURE 3.8 – Logo OpenID Connect.

OpenID Connect (OIDC) est un protocole d'authentification fédérée construit au-dessus d'OAuth 2.0. Il permet à une entité externe — ici le pipeline GitHub Actions — de prouver son identité auprès d'AWS sans recourir à des clés d'accès statiques. À chaque exécution, GitHub Actions émet un jeton d'identité signé contenant des attributs de l'exécution (dépôt, branche, événement). AWS vérifie ce jeton auprès de GitHub comme fournisseur de confiance, puis accorde des credentials temporaires via IAM, valables uniquement pour la durée du job. Cette approche élimine tout secret à stocker dans le dépôt et réduit structurellement la surface d'attaque du pipeline.

## 3.4 Flux 3 — Observabilité et monitoring

### 3.4.1 Grafana



FIGURE 3.9 – Logo Grafana.

Grafana est la plateforme open-source de référence pour la visualisation et l'exploration de données de monitoring. Elle se connecte à de multiples sources de données — dont Loki pour les logs — et permet de construire des tableaux de bord interactifs offrant une visibilité en temps réel sur l'état du système. Dans ce projet, Grafana centralise les résultats du pipeline DevSecOps : statut de chaque étape, taux de succès, évolution dans le temps de la posture de sécurité du projet.

---

### 3.4.2 Loki



FIGURE 3.10 – Logo Grafana Loki.

Grafana Loki est un système d’agrégation de logs conçu pour être économique en stockage et nativement intégré à Grafana. Contrairement aux solutions traditionnelles qui indexent l’intégralité du contenu des logs, Loki n’indexe que les métadonnées (labels) associées à chaque flux de logs, ce qui réduit considérablement l’empreinte de stockage. Dans ce projet, Loki reçoit à la fin de chaque exécution du pipeline un événement JSON structuré consolidant le résultat de l’ensemble des étapes de sécurité, constituant ainsi une piste d’audit complète et interrogeable des exécutions passées.

## Conclusion

Ce chapitre a présenté l’ensemble des technologies et outils mobilisés dans le cadre du projet, organisés selon les trois flux qui structurent l’architecture globale.

Le premier flux — intégration et livraison continues — repose sur GitHub Actions comme orchestrateur central et Docker comme standard de conteneurisation. Ces deux outils constituent l’épine dorsale du pipeline DevSecOps : GitHub Actions déclenche et séquence l’ensemble des contrôles.



# 4 Dependency Risk Radar : Conception et Architecture

## 4.1 Introduction

Ce chapitre décrit l'architecture logicielle de Dependency Risk Radar, ses modèles de données, son moteur de scoring multicritère, et ses interfaces utilisateur. Chaque composant est présenté en lien direct avec les captures d'écran de l'application déployée.

## 4.2 Architecture globale en quatre couches

DRR est structuré en quatre couches découplées communiquant via des interfaces bien définies. Cette séparation garantit la maintenabilité et la testabilité indépendante de chaque module.

TABLE 4.1 – Couches architecturales de DRR

| N° | Couche           | Responsabilité                                                                                      | Technologies                       |
|----|------------------|-----------------------------------------------------------------------------------------------------|------------------------------------|
| 1  | Ingestion        | Parsing Gradle (Groovy/Kotlin DSL), analyse APK via androguard, appels parallèles aux APIs externes | Python, androguard, httpx          |
| 2  | Analyse          | Scoring multicritère, construction du graphe DAG, propagation transitive du risque                  | Python, NetworkX, pydantic         |
| 3  | Planificateur IA | Génération du plan de mise à jour priorisé, narratives techniques et managériales                   | Gemini Flash API, Anthropic Claude |
| 4  | Présentation     | Dashboard React, API REST et Web-Socket, génération SBOM et export PDF                              | FastAPI, React 18, D3.js           |

---

Le diagramme de séquence ci-dessous illustre le flux d'exécution complet d'une analyse Gradle, depuis la soumission par l'utilisateur jusqu'à l'affichage des résultats, en détaillant les interactions entre toutes les couches.

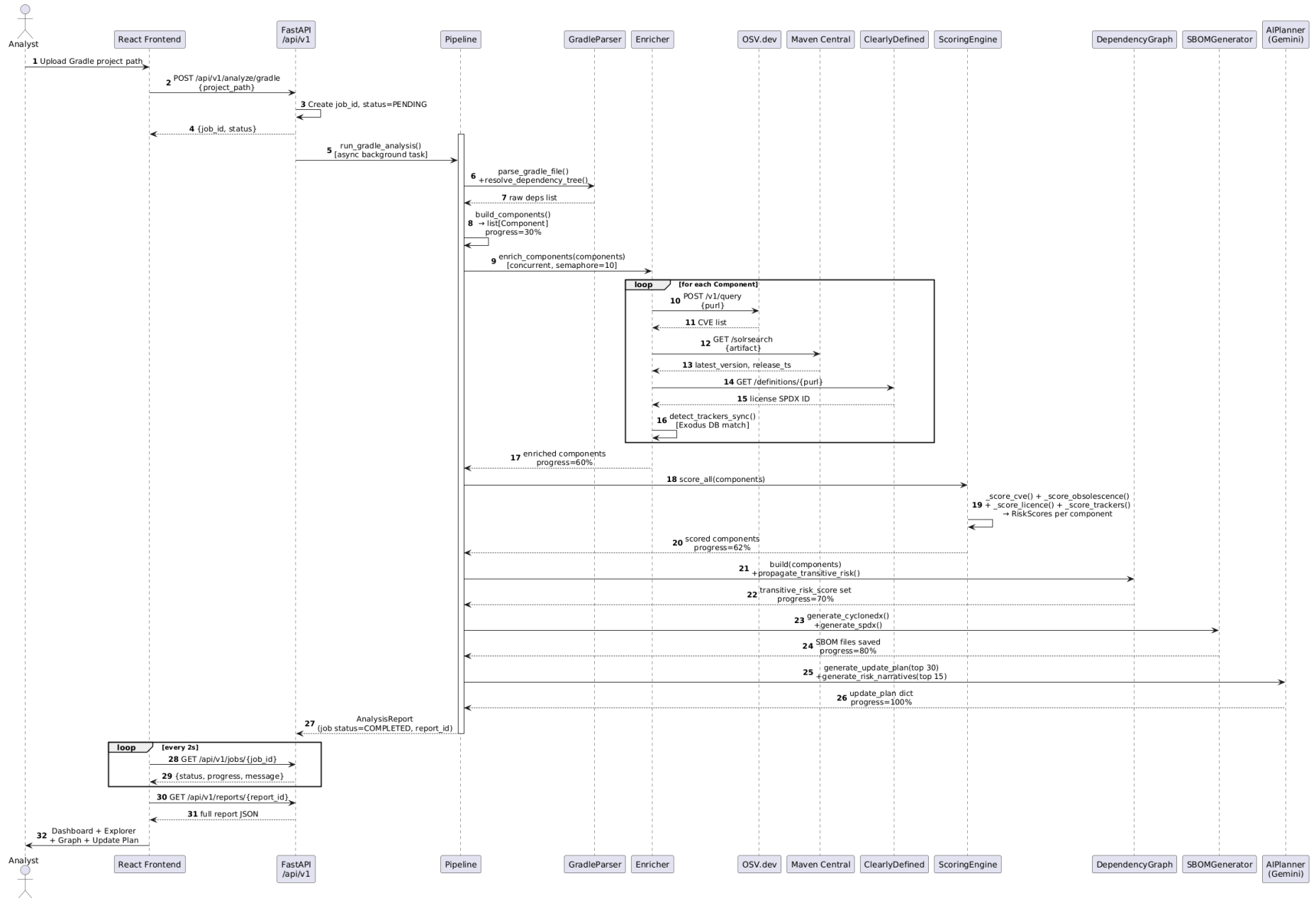


FIGURE 4.1 – Diagramme de séquence de l'analyse Gradle dans DRR

---

## 4.3 Modèles de données

### 4.3.1 Vue d'ensemble

Le diagramme de classes ci-dessous présente l'ensemble des entités du domaine et leurs relations. La classe centrale **Component** agrège toutes les informations collectées sur un composant : ses métadonnées, ses vulnérabilités, sa licence, ses trackers et ses scores de risque.

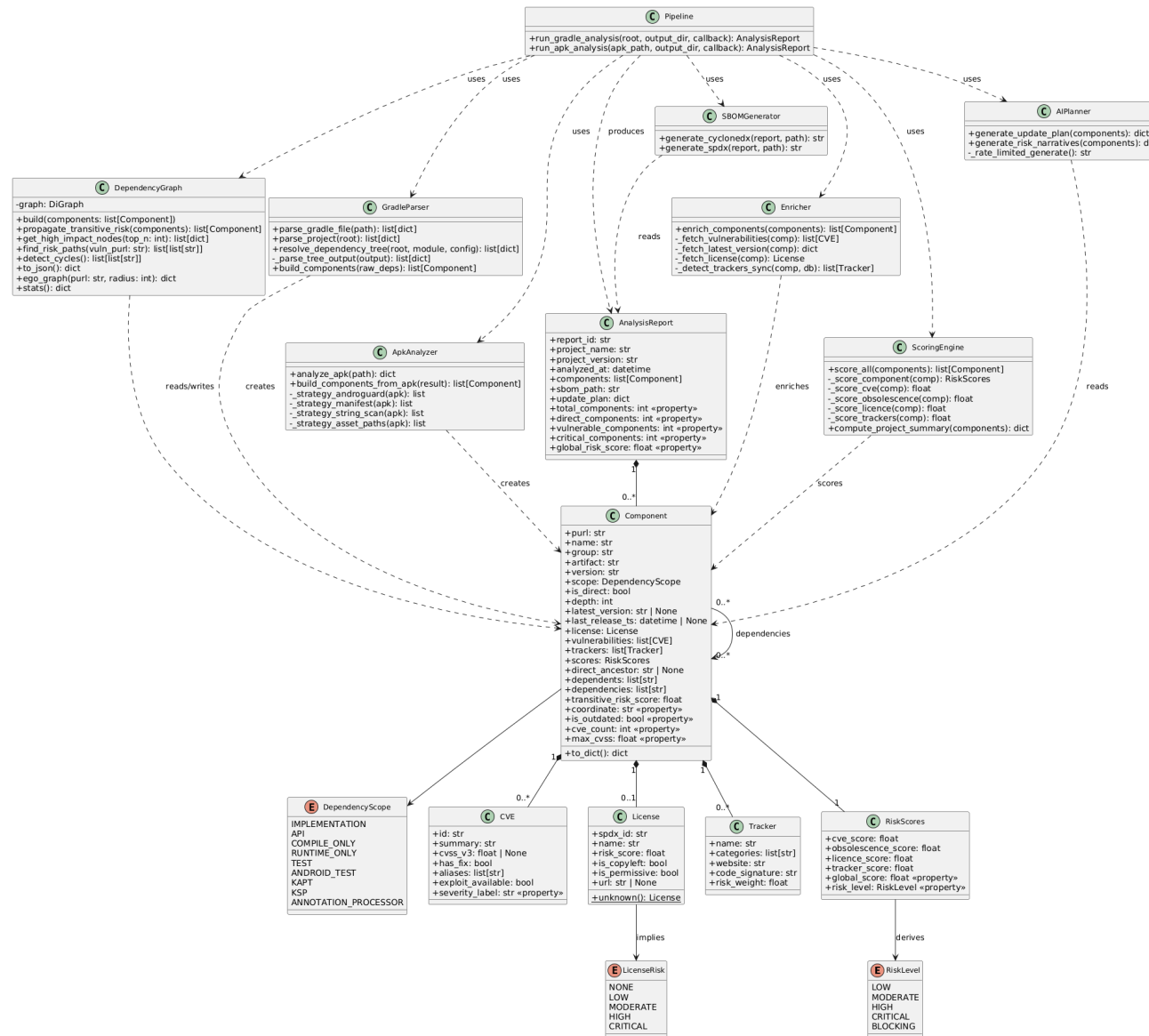


FIGURE 4.2 – Diagramme de classes de DRR — neuf entités principales : Pipeline, Component, CVE, License, Tracker, RiskScores, ScoringEngine, SBOMGenerator et AIPlanner

---

### 4.3.2 Entités principales

**Component** est l'entité centrale du modèle. Elle encapsule le Package URL (PURL), le nom, le groupe, la version détectée, la version la plus récente, la profondeur dans le graphe de dépendances, et les listes de CVE, trackers et scores associés. Chaque composant porte également un indicateur `is_direct` distinguant les dépendances déclarées explicitement des dépendances transitives.

**CVE** représente une vulnérabilité connue associée à un composant. Elle stocke l'identifiant OSV/NVD, le score CVSS v3, l'indicateur de disponibilité d'un correctif (`has_fix`), et la liste des alias (CVE ID officiels).

**License** encapsule l'identifiant SPDX de la licence, son score de risque (dérivé d'une table de référence interne), et les indicateurs `is_copyleft` et `is_permissive`.

**Tracker** référence un SDK de traçage détecté dans le bytecode Dalvik, avec son nom, ses catégories (analytique, publicitaire, fingerprinting) et son poids de risque issu de la base Exodus Privacy.

**RiskScores** est l'objet résultat du scoring : il agrège les quatre sous-scores normalisés et le score global, et dérive le niveau de risque (`RiskLevel`) par seuillage.

**DependencyScope** est une énumération couvrant les portées Gradle : `IMPLEMENTATION`, `API`, `COMPILE_ONLY`, `RUNTIME_ONLY`, `TEST`, `ANDROID_TEST`, `KAPT` et `ANNOTATION_PROCESSOR`.

## 4.4 Formule de scoring multicritère

### 4.4.1 Formule globale

Le moteur de scoring (`ScoringEngine`) calcule pour chaque composant un score global normalisé sur  $[0, 100]$  selon la formule pondérée suivante :

$$\mathcal{R} = 0,45 \times S_{\text{CVE}} + 0,25 \times S_{\text{obs}} + 0,20 \times S_{\text{lic}} + 0,10 \times S_{\text{track}} \quad (4.1)$$

### 4.4.2 Dimensions de risque

- **$S_{\text{CVE}}$  (45%)** : dérivé du score CVSS v3 maximal parmi toutes les vulnérabilités connues, interrogées simultanément sur OSV.dev et le NIST NVD. Une pénalité est appliquée pour les vulnérabilités sans correctif disponible (jusqu'à +15 points) et pour le nombre total de CVE (jusqu'à +10 points).
- **$S_{\text{obs}}$  (25%)** : combinaison du retard de version sémantique (majeure, mineure, patch) par rapport à la dernière release disponible sur Maven Central, et de l'ancienneté de la dernière publication. Une bibliothèque inactive depuis plus de deux ans accumule jusqu'à +50 points supplémentaires.
- **$S_{\text{lic}}$  (20%)** : correspondance de l'identifiant SPDX avec une table de risque interne, de 0 (MIT, Apache-2.0) à 90 (AGPL-3.0) ou 65 (licence inconnue).
- **$S_{\text{track}}$  (10%)** : croisement des noms de classes Dalvik avec la base Exodus Privacy (300+ trackers), pondéré par la catégorie du tracker.

TABLE 4.2 – Niveaux de risque et actions recommandées

| Score    | Niveau   | Action recommandée              |
|----------|----------|---------------------------------|
| 0 – 19   | Low      | Surveillance standard           |
| 20 – 49  | Moderate | Mise à jour au prochain sprint  |
| 50 – 74  | High     | Mise à jour dans les 2 semaines |
| 75 – 89  | Critical | Mise à jour immédiate           |
| 90 – 100 | Blocking | Bloquer le déploiement          |

## 4.5 Graphe de dépendances et propagation transitive

### 4.5.1 Modélisation par graphe orienté acyclique

DRR modélise l'ensemble des dépendances d'un projet sous la forme d'un graphe orienté acyclique (DAG) via la bibliothèque NetworkX. Chaque nœud représente un composant, chaque arc une relation de dépendance directe. Trois algorithmes sont appliqués sur ce graphe :

1. **Détection des nœuds à fort impact** : les composants présentant un degré entrant élevé (nombreux dépendants) sont signalés comme candidats prioritaires à la mise à jour, car une seule correction se propage à l'ensemble de leurs dépendants.
2. **Recherche du chemin de risque critique** : pour chaque bibliothèque transitive vulnérable, `nx.shortest_path` remonte jusqu'à la dépendance directe responsable de son introduction, rendant l'action corrective immédiatement actionnable.
3. **Score transitif agrégé** : le risque transitif d'une dépendance directe est calculé comme le score maximal de ses descendants, atténué exponentiellement par la profondeur :

### 4.5.2 Visualisation du graphe

La vue *Dep Graph* du dashboard affiche ce graphe en disposition force-directed via D3.js. Les nœuds sont colorés selon leur niveau de risque (rouge pour Critical, orange pour High, jaune pour Moderate, vert pour Low) et dimensionnés proportionnellement à leur nombre de dépendants.



FIGURE 4.3 – Vue *Dep Graph* de DRR — graphe force-directed du projet *PizzaRecipes* (63 nœuds, 261 arcs), coloré par niveau de risque : vert (Low), jaune (Moderate)

## 4.6 Génération de SBOM

À l'issue de chaque analyse, DRR génère automatiquement un Software Bill of Materials dans deux formats standards :

- **CycloneDX 1.5** (JSON et XML) : standard promu par l'OWASP, orienté sécurité et vulnérabilités, requis par l'EU Cyber Resilience Act et le NIST SSDF.
- **SPDX 2.3** (JSON et tag-value) : standard de la Linux Foundation, orienté conformité de licence.

Chaque entrée du SBOM contient : le nom et la version du composant, le Package URL (PURL), l'identifiant SPDX de licence, le hash SHA-256, et les relations de dépendance. Les fichiers sont exportables via l'endpoint REST `GET /api/v1/reports/{id}/sbom` et sauvegardés localement sous `sbom_cyclonedx.json` et `sbom_spdx.json`.

## 4.7 Planificateur de mises à jour assisté par IA

### 4.7.1 Architecture du module IA

Le module **AIPlanner** implémente une stratégie de basculement entre deux fournisseurs LLM :

1. **Google Gemini Flash** (primaire) : si `GEMINI_API_KEY` est configurée, Gemini est interrogé en premier pour sa rapidité et son coût réduit.
2. **Anthropic Claude** (fallback) : en cas d'échec de Gemini après trois tentatives.
3. **Plan déterministe** (fallback final) : généré localement à partir des données de scoring sans appel réseau.



## 4.7.2 Contenu du plan généré

Les 30 composants présentant un score supérieur à 20 sont transmis au LLM avec un prompt structuré. Pour chaque composant, le modèle retourne : la version recommandée, le niveau de priorité (CRITIQUE / HAUTE / MOYENNE / BASSE), le risque de rupture de compatibilité, l'effort de migration estimé, et des notes de migration optionnelles. Un résumé exécutif est également produit pour la communication managériale.

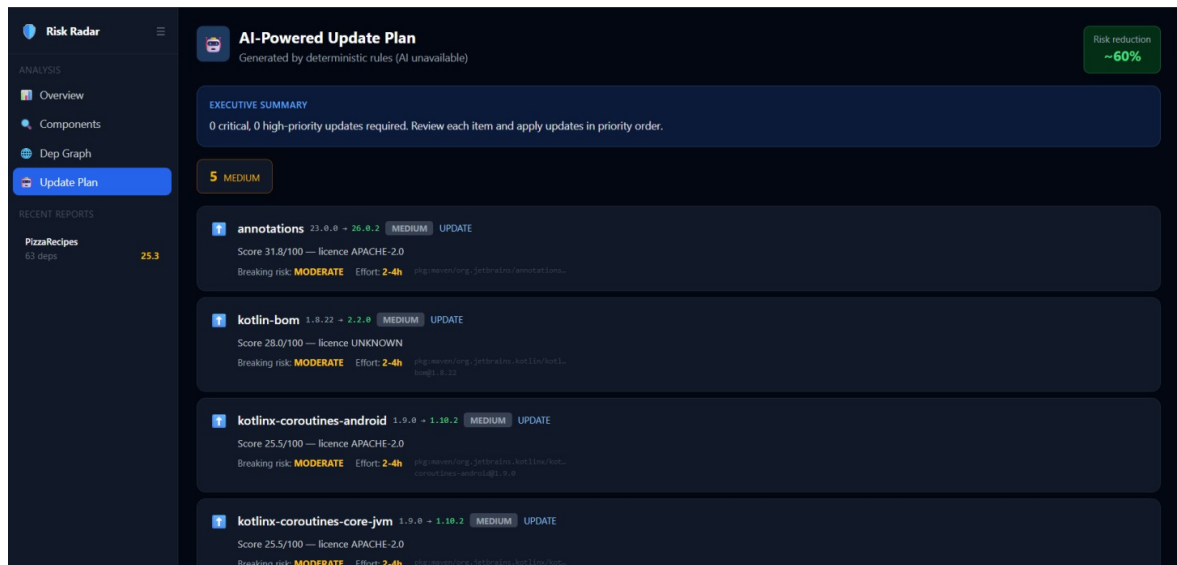


FIGURE 4.4 – Vue *Update Plan* de DRR — plan généré par règles déterministes pour le projet PizzaRecipes : 5 mises à jour de priorité MEDIUM, réduction de risque estimée à 60%

## 4.8 Endpoints REST et WebSocket

DRR expose une API REST versionnée sous le préfixe `/api/v1`, complétée par un flux WebSocket pour le streaming en temps réel de la progression des analyses.

TABLE 4.3 – Principaux endpoints REST de DRR

| Méthode | Endpoint                  | Description                                                  |
|---------|---------------------------|--------------------------------------------------------------|
| POST    | /api/v1/analyze/gradle    | Soumet un chemin de projet Gradle pour analyse asynchrone    |
| POST    | /api/v1/analyze/apk       | Soumet un fichier APK pour analyse                           |
| GET     | /api/v1/jobs/{job_id}     | Interroge le statut et la progression d’une analyse en cours |
| GET     | /api/v1/reports/{id}      | Récupère le rapport complet d’une analyse terminée           |
| GET     | /api/v1/reports/{id}/sbom | Exporte le SBOM au format CycloneDX ou SPDX                  |
| GET     | /health                   | Vérification de santé de l’application (healthcheck)         |
| WS      | /ws/jobs/{job_id}         | Flux WebSocket de progression en temps réel                  |

Le flux d’interaction suit un modèle *fire-and-poll* : le frontend soumet une analyse (POST), reçoit un `job_id`, puis interroge périodiquement l’état du job (GET) ou s’abonne au flux WebSocket pour recevoir les mises à jour de progression en temps réel (étapes 28–29 du diagramme de séquence).

## 4.9 Frontend React et ses six vues

### 4.9.1 Interface de soumission

La page d’accueil propose deux modes d’ingestion : dépôt d’un fichier APK par glisser-déposer, ou saisie d’un chemin de projet Gradle côté serveur. L’historique des analyses récentes est affiché dans le panneau latéral gauche avec leur score global.

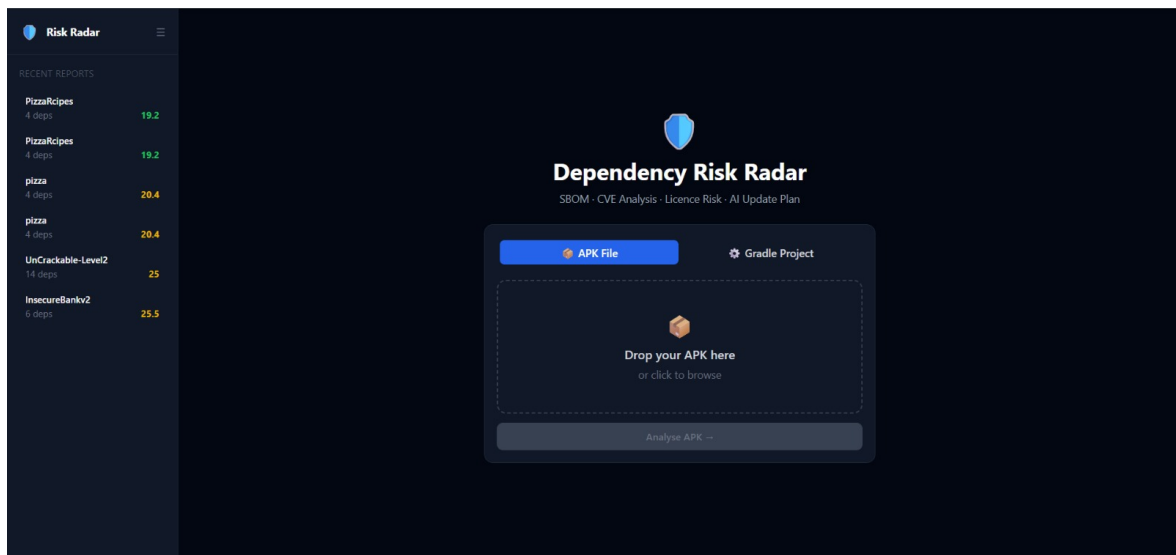


FIGURE 4.5 – Interface d’accueil de DRR — page de soumission avec les deux modes d’ingestion (APK File et Gradle Project) et l’historique des analyses récentes

## 4.9.2 Vue Overview

La vue *Overview* présente les indicateurs clés de l’analyse : score de risque global, nombre total de CVE, composants obsolètes, licences copyleft et trackers détectés. Un graphique en anneau illustre la distribution des niveaux de risque, et un graphique en barres horizontales liste les dix composants les plus risqués.

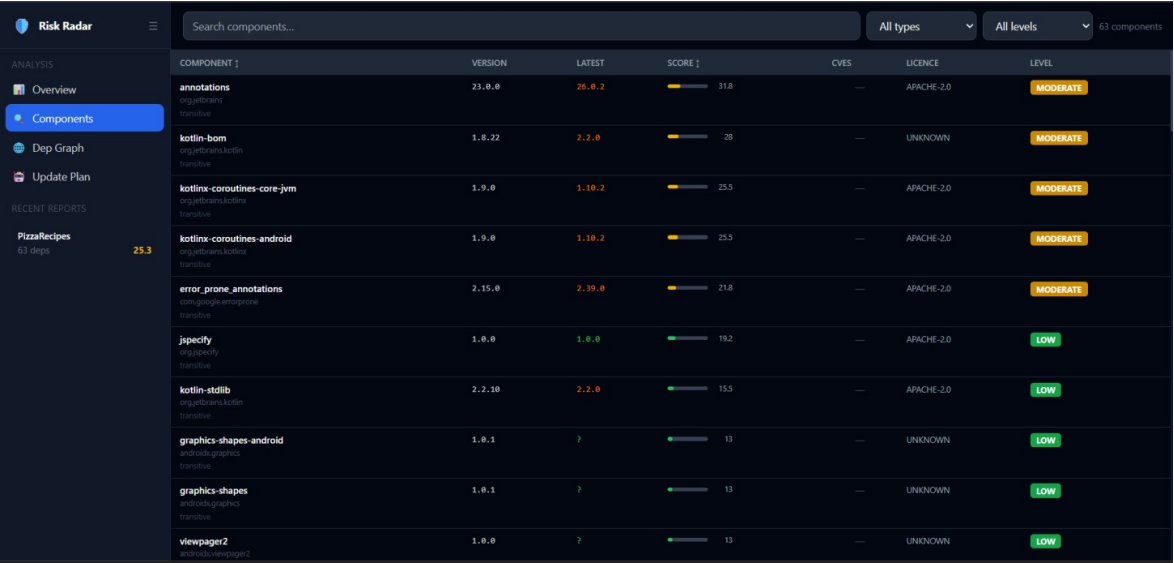


FIGURE 4.6 – Vue *Overview* de DRR pour le projet PizzaRecipes — score global 25.3 (Moderate), 0 CVE, 6 composants obsolètes, distribution de risque majoritairement Low

## 4.9.3 Vue Components

La vue *Components* affiche le tableau exhaustif de tous les composants détectés, triable et filtrable par type (direct ou transitif) et par niveau de risque. Pour chaque

composant sont affichés : la version installée, la version la plus récente, le score numérique, le nombre de CVE, la licence SPDX et le niveau de risque coloré.



| COMPONENT                                           | VERSION | LATEST | SCORE | CVES | LICENCE    | LEVEL    |
|-----------------------------------------------------|---------|--------|-------|------|------------|----------|
| annotations<br>org.jetbrains.kotlin                 | 23.0.0  | 26.0.2 | 31.8  | —    | APACHE-2.0 | MODERATE |
| kotlin-bom<br>org.jetbrains.kotlin                  | 1.8.22  | 2.2.0  | 28    | —    | UNKNOWN    | MODERATE |
| kotlinx-coroutines-core-jvm<br>org.jetbrains.kotlin | 1.9.0   | 1.10.2 | 25.5  | —    | APACHE-2.0 | MODERATE |
| kotlinx-coroutines-android<br>org.jetbrains.kotlin  | 1.9.0   | 1.10.2 | 25.5  | —    | APACHE-2.0 | MODERATE |
| error.prone.annotations<br>com.google.errorprone    | 2.15.0  | 2.39.0 | 21.8  | —    | APACHE-2.0 | MODERATE |
| jSpecify<br>org.jetbrains.kotlin                    | 1.0.0   | 1.0.0  | 19.2  | —    | APACHE-2.0 | LOW      |
| kotlin-stdlib<br>org.jetbrains.kotlin               | 2.2.10  | 2.2.0  | 15.5  | —    | APACHE-2.0 | LOW      |
| graphics-shapes-android<br>androidx.graphics        | 1.0.1   | ?      | 13    | —    | UNKNOWN    | LOW      |
| graphics-shapes<br>androidx.graphics                | 1.0.1   | ?      | 13    | —    | UNKNOWN    | LOW      |
| viewpager2<br>androidx.viewpager2                   | 1.0.0   | ?      | 13    | —    | UNKNOWN    | LOW      |

FIGURE 4.7 – Vue *Components* de DRR — tableau des 63 composants du projet PizzaRecipes, triés par score décroissant, avec scores, licences et niveaux de risque

#### 4.9.4 Vue Dep Graph

La vue *Dep Graph* présente la visualisation force-directed du graphe de dépendances via D3.js (figure 4.3). Un clic sur un nœud révèle le chemin de risque critique remontant jusqu'à la dépendance directe responsable.

#### 4.9.5 Vue Update Plan

La vue *Update Plan* affiche le plan de mise à jour généré par le module IA (figure 4.4), structuré par ordre de priorité décroissante. Chaque recommandation précise la version cible, l'effort estimé en heures-développeur, et le risque de rupture de compatibilité.

#### 4.9.6 Vue SBOM et Export PDF

La sixième vue propose un explorateur JSON/XML interactif du SBOM généré (CycloneDX et SPDX), ainsi que l'export d'un rapport PDF complet incluant le résumé SBOM, les scores par composant, le plan IA et les narratives de risque.

### 4.10 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie deux acteurs humains — *Security Analyst* et *Developer* — et cinq acteurs externes (*Maven Central*, *OSV.dev*, *ClearlyDefined*, *Gemini AI*, *Exodus Privacy*). Les cas d'utilisation principaux couvrent l'analyse Gradle et APK, la consultation du dashboard, la navigation dans les composants, la visualisation du graphe, l'accès au plan IA, et l'export SBOM dans les deux formats.

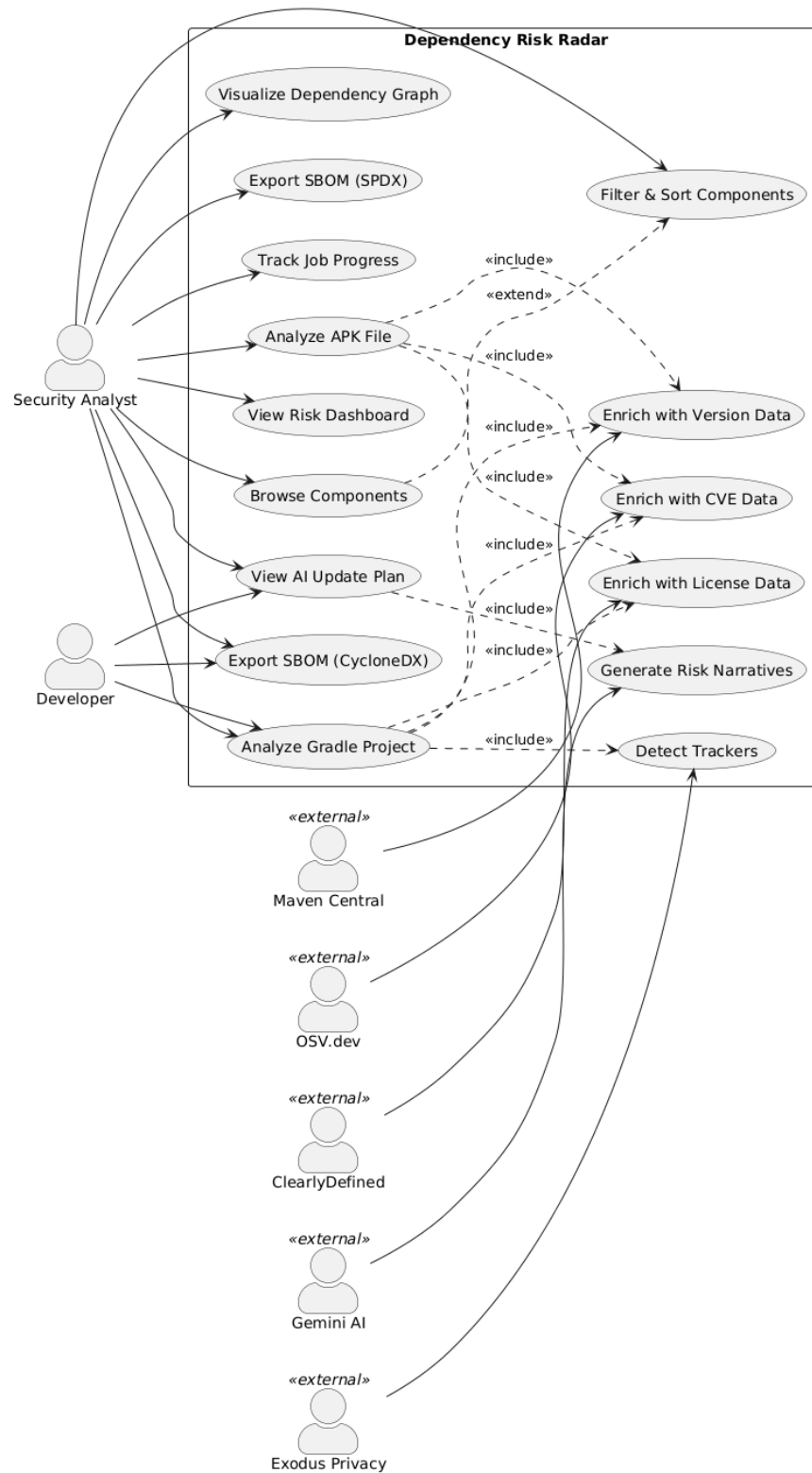


FIGURE 4.8 – Diagramme de cas d'utilisation de DRR — deux acteurs internes (Security Analyst, Developer) et cinq acteurs externes (Maven Central, OSV.dev, ClearlyDefined, Gemini AI, Exodus Privacy)

---

## 4.11 Bilan architectural

L'architecture de DRR se distingue par quatre propriétés clés :

- **Extensibilité** : l'ajout d'une nouvelle source de données se réduit à l'implémentation d'une méthode dans **Enricher**, sans modification des autres couches.
- **Résilience** : la stratégie de fallback LLM (Gemini → Anthropic → règles déterministes) garantit la disponibilité du plan de mise à jour même en l'absence de clés API.
- **Conformité** : la génération simultanée CycloneDX et SPDX couvre les exigences réglementaires des principaux cadres normatifs (EU CRA, NIST SSDF, Executive Order 14028).
- **Performance** : l'enrichissement concurrent (sémaphore de 10 tâches parallèles) permet de traiter un projet de 200 dépendances en moins de 60 secondes.

## Conclusion

Ce chapitre a décrit l'architecture complète de Dependency Risk Radar, de ses modèles de données à ses interfaces utilisateur. L'architecture en quatre couches découpées — ingestion, analyse, planification IA et présentation — garantit la maintenabilité et l'extensibilité de la plateforme. Le moteur de scoring multicritère, fondé sur une formule pondérée intégrant CVE, obsolescence, licence et trackers, fournit un signal de risque actionnable en moins de 60 secondes pour un projet de 200 dépendances. La génération simultanée de SBOM en formats CycloneDX et SPDX assure la conformité aux principales exigences réglementaires en vigueur. Le planificateur IA avec sa stratégie de fallback à trois niveaux garantit la disponibilité du plan de remédiation même en l'absence de clé API. L'ensemble de ces composants est désormais déployé en production sur AWS, selon le pipeline DevSecOps décrit dans le chapitre suivant.

# 5 Pipeline DevSecOps : Conception et Mise en Œuvre

## 5.1 Introduction

Il serait paradoxal qu'un outil d'audit de sécurité des dépendances soit lui-même développé sans les rigueurs qu'il préconise. Appliquer à DRR les mêmes exigences de sécurité qu'il impose aux projets qu'il analyse n'est pas une contrainte supplémentaire : c'est la condition de sa crédibilité. C'est précisément l'objet de ce chapitre.

Le pipeline DevSecOps mis en place pour le développement et le déploiement de DRR incarne concrètement le principe du *shift-left security* introduit au chapitre 2 : la sécurité n'est pas traitée comme une étape de validation finale, mais comme une contrainte intégrée à chaque commit, chaque pull request, et chaque déploiement. Chaque modification du code source déclenche automatiquement une séquence de contrôles qui couvre l'analyse statique du code, la détection de secrets, l'audit des dépendances, les tests unitaires avec couverture, et le scan de vulnérabilités des images Docker produites. Aucune image n'atteint le registre ECR sans avoir franchi l'intégralité de ces portes.

Le pipeline est structuré en deux workflows GitHub Actions distincts et séquentiels. Le premier — le workflow CI — est déclenché à chaque push ou pull request sur la branche `main` et assure la qualité et la sécurité du code. Le second — le workflow CD — ne se déclenche qu'à la condition explicite que le CI se soit terminé avec succès, garantissant ainsi qu'aucun déploiement ne peut s'initier depuis une base de code qui n'aurait pas passé l'ensemble des contrôles de sécurité.

Ce chapitre décrit la conception et la mise en œuvre de chacun de ces deux workflows, en justifiant pour chaque stage les choix de configuration, les seuils retenus, et les décisions prises face aux contraintes rencontrées.

## 5.2 Architecture générale du pipeline

Le pipeline est découpé en deux workflows GitHub Actions distincts et séquentiels : un workflow CI (`ci.yml`) chargé de la sécurité, de la qualité et des tests, et un workflow CD (`cd.yml`) chargé du déploiement en production. Cette séparation est intentionnelle : le CD ne se déclenche que si le CI s'est terminé avec succès sur la branche `main`, via le déclencheur `workflow_run`. Aucune image n'atteint ECR sans avoir franchi l'intégralité des portes de sécurité.

## 5.3 Workflow CI — Security & Quality Gate

---

### 5.3.1 Déclencheurs

Le CI s'exécute sur deux événements : tout *push* sur `main` et toute *pull request* ciblant `main`. Les jobs sont indépendants et s'exécutent en parallèle jusqu'au stage de `build`, qui dépend explicitement de leur succès collectif via `needs`.

### 5.3.2 Stage 2a — SAST Python : Bandit

Bandit analyse statiquement le code Python du backend à la recherche de patterns de sécurité risqués (injections, appels système dangereux, usage de fonctions dépréciées). Il est configuré avec les seuils `-ll -ii` (sévérité et confiance minimales à *medium*) et produit un rapport JSON archivé en artefact. Le flag `-exit-zero` est maintenu temporairement pour ne pas bloquer le pipeline pendant l'assainissement progressif de la base de code.

### 5.3.3 Stage 2b — SAST JavaScript : ESLint

ESLint analyse le code React du frontend. Les dépendances sont installées via `npm ci` pour garantir un environnement reproductible à partir du `package-lock.json`. Le seuil `-max-warnings=20` tolère un volume résiduel d'avertissements pendant la phase de stabilisation ; il est prévu de le ramener à zéro une fois les violations résolues.

### 5.3.4 Stage 2c — Scan de secrets : Gitleaks

Gitleaks parcourt l'intégralité de l'historique Git (`fetch-depth: 0`) pour détecter tout secret commité accidentellement — clé API, token, mot de passe. Ce job bloque immédiatement le pipeline en cas de détection, sans tolérance.

### 5.3.5 Stage 3a — SCA Python : pip-audit

`pip-audit` audite les dépendances Python du backend contre les bases de vulnérabilités OSV et PyPI. Le flag `-strict` fait échouer le job dès qu'une CVE non ignorée est détectée. Trois vulnérabilités transitives de `starlette` sont explicitement exclues via `-ignore-vuln` : elles ne peuvent pas être corrigées sans une montée de version de FastAPI encore indisponible au moment de l'écriture du pipeline.

### 5.3.6 Stage 3b — SCA Node : npm audit

`npm audit` audite les dépendances frontend. Le seuil `-audit-level=high` ne fait échouer le job que sur les vulnérabilités de sévérité *high* ou *critical*, laissant passer les avertissements de faible criticité.

### 5.3.7 Stage 4 — Build Docker et tests

Ce stage est le point de convergence : il ne s'exécute qu'une fois les cinq jobs précédents passés (`needs: [sast-python, sast-js, secrets-scan, sca-python, sca-node]`). Il enchaîne trois opérations :



- 
1. **Tests pytest avec couverture** : les tests unitaires et d'intégration du backend s'exécutent avec un service Redis éphémère fourni par les *services* GitHub Actions, et une base SQLite en mémoire. Le seuil de couverture est fixé à 60% via `-cov-fail-under=60` ; le rapport XML est archivé en artefact.
  2. **Build des images Docker** : les images backend et frontend sont construites via `docker/build-push-action` avec `push: false` — elles ne sont pas encore poussées vers un registre. Le cache GitHub Actions (`type=gha`) accélère les builds successifs en réutilisant les couches Docker inchangées.
  3. **Archivage des images** : les images sont exportées en fichiers `.tar` et archivées en artefacts avec une rétention d'un jour, pour être récupérées par les stages suivants sans rebuild.

### 5.3.8 Stage 5 — Scan de conteneurs : Trivy

Trivy charge les images archivées par le stage précédent et les analyse à la recherche de CVE dans les couches système et applicatives. La configuration `exit-code: 1` et `severity: CRITICAL` fait échouer le pipeline uniquement sur les vulnérabilités critiques non corrigées (`ignore-unfixed: true`). Un fichier `.trivyignore` permet d'exclure des CVE documentées pour le backend. Le frontend n'a pas de fichier d'exclusion, imposant une politique plus stricte.

### 5.3.9 Stage 6 — Observabilité : envoi des logs vers Loki

Le dernier job du CI s'exécute systématiquement (`if: always()`), quelle que soit l'issue du pipeline. Il pousse vers Loki, via une requête HTTP `curl`, un événement JSON structuré contenant le résultat de chaque job, le SHA du commit, l'acteur et le numéro de run. Ces logs alimentent les dashboards Grafana pour le suivi de la santé du pipeline.

## 5.4 Workflow CD — Build, Push & Deploy

### 5.4.1 Déclencheur conditionnel

Le CD se déclenche via `workflow_run` sur la complétion du CI, avec une condition explicite : `github.event.workflow_run.conclusion == 'success'`. Ce mécanisme garantit qu'aucun déploiement ne peut s'initier si une seule porte de sécurité du CI a échoué. La directive `concurrency` avec `cancel-in-progress: false` empêche deux déploiements simultanés en production.

### 5.4.2 Authentification OIDC entre GitHub Actions et AWS

Le pipeline n'utilise aucune clé AWS statique stockée en secret. L'authentification repose sur OpenID Connect (OIDC) : GitHub Actions génère un token JWT signé qui est échangé contre des credentials AWS temporaires via `aws-actions/configure-aws-credentials`. AWS IAM dispose d'un rôle `GitHubActionsRole` configuré avec une relation de confiance restreinte au dépôt et à la branche `main`. Les credentials obtenus sont éphémères et limités aux seules actions nécessaires : pousser une image dans ECR et mettre à jour un service ECS.

---

### 5.4.3 Stage 6 — Push vers Amazon ECR

Les images `.tar` buildées par le CI sont téléchargées depuis les artefacts du run CI correspondant, rechargées localement, puis taguées avec deux identifiants : le SHA du commit (tag immuable pour la traçabilité) et `latest` (tag flottant pour la commodité). Les deux tags sont poussés vers les dépôts ECR `drd/backend` et `drd/frontend`. Cette stratégie de double tag permet de retrouver précisément quelle version de code correspond à chaque image déployée.

### 5.4.4 Stage 7 — Déploiement sur Amazon ECS

Le déploiement suit le pattern recommandé par AWS pour les mises à jour sans interruption :

1. La *task definition* courante est téléchargée depuis ECS via `aws ecs describe-task-definition`.
2. L'URI de la nouvelle image est injectée dans la définition via `amazon-ecs-render-task-definition`.
3. La nouvelle révision est déployée via `amazon-ecs-deploy-task-definition` avec `wait-for-service-stability: true` : le job attend qu'ECS confirme la stabilité du service avant de passer à la suite, assurant un déploiement contrôlé.

Backend et frontend sont déployés séquentiellement sur le cluster `drd-cluster`.

### 5.4.5 Smoke test post-déploiement

Après déploiement, un smoke test interroge l'endpoint `/api/v1/health` du backend avec six tentatives espacées de dix secondes. Un code HTTP 200 valide le déploiement ; l'absence de réponse après six tentatives fait échouer le job. Ce mécanisme de retry absorbe le délai de démarrage du conteneur sans imposer une attente fixe.

### 5.4.6 Stage 8 — Observabilité CD

Identique au stage 6 du CI, ce job s'exécute en `if: always()` et pousse vers Loki un événement structuré contenant le résultat des jobs `push-images` et `deploy`, le SHA du commit déployé et le tag d'image. En cas de succès ou d'échec, un message de notification est émis dans les logs du runner.

## Conclusion

Ce chapitre a décrit la conception et la mise en œuvre du pipeline DevSecOps de DRR dans son intégralité. Le workflow CI enchaîne huit portes de sécurité automatisées — analyse statique, détection de secrets, audit de dépendances, tests avec couverture et scan d'images Docker — qui s'exécutent en parallèle et conditionnent le déclenchement de l'étape de build. Le workflow CD, déclenché uniquement sur succès complet du CI, pousse les images vers Amazon ECR et orchestre le déploiement rolling sur ECS via le protocole OIDC, sans aucune clé d'accès statique. Un smoke test post-déploiement et une couche d'observabilité Loki complètent le dispositif. Les résultats concrets de ce pipeline — 28 itérations, 8 portes vertes en 5 minutes 21 secondes — sont présentés et

---

analysés au chapitre 6. L'infrastructure AWS sur laquelle s'appuie le CD est détaillée dans le chapitre suivant.

# 6 Infrastructure AWS et Déploiement

## 6.1 Introduction

La mise en production d'une application conteneurisée sur le cloud ne se réduit pas à l'exécution de quelques commandes : elle implique la conception d'une infrastructure cohérente, la définition rigoureuse des identités et des permissions, la protection des secrets applicatifs, et la mise en place d'un pipeline de déploiement entièrement automatisé. Ce chapitre décrit, dans leur intégralité et dans leur ordre de réalisation, toutes les configurations AWS effectuées pour déployer Dependency Risk Radar en production : des dépôts de conteneurs aux rôles IAM, des définitions de tâches ECS aux groupes de sécurité, du flux de déploiement bout en bout jusqu'au tableau de bord de supervision Grafana et Loki.

## 6.2 Vue d'ensemble de l'architecture AWS

L'infrastructure AWS de DRR est organisée autour de deux instances EC2 aux responsabilités distinctes, déployées dans la région `eu-west-1` (Irlande). Cette séparation des rôles garantit l'isolation entre les composants applicatifs et le système de supervision : une défaillance de l'instance applicative n'affecte pas la disponibilité du monitoring, et inversement.

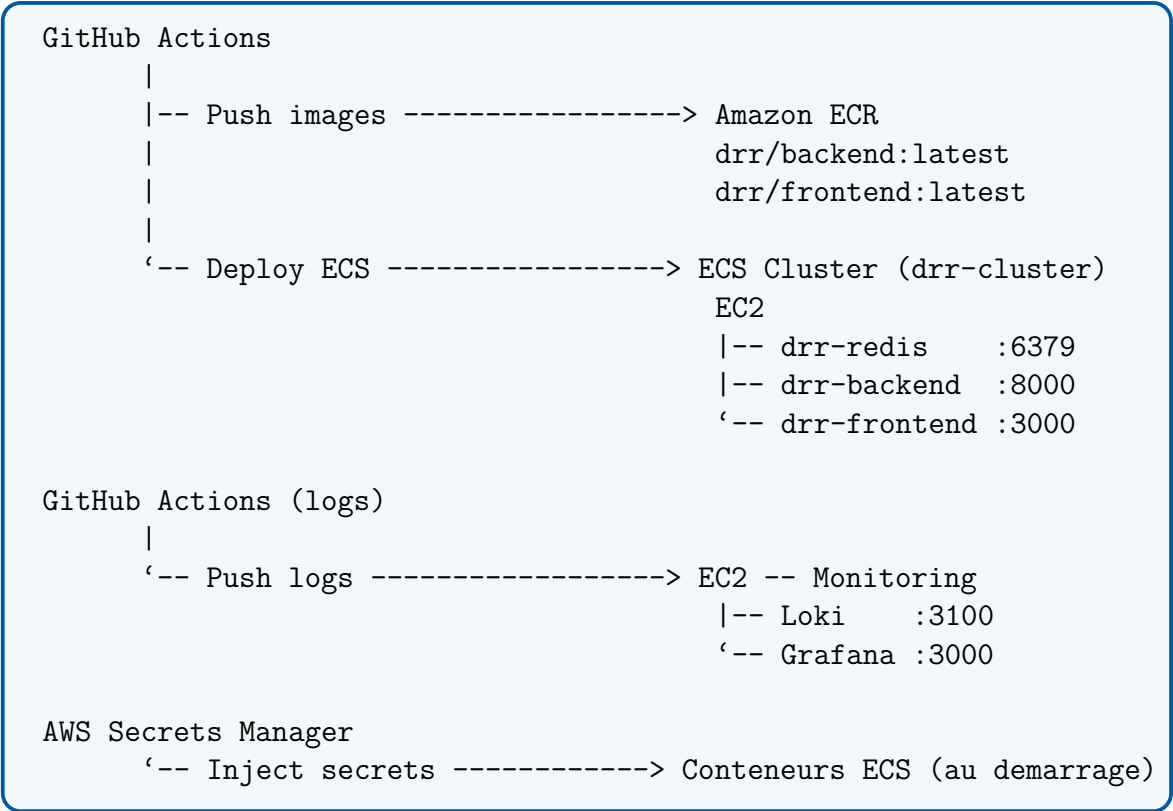


FIGURE 6.1 – Architecture globale de l’infrastructure AWS de DRR

TABLE 6.1 – Instances EC2 et leurs rôles

| Instance     | Type           | RAM  | Rôle                                                            |
|--------------|----------------|------|-----------------------------------------------------------------|
| ECS Instance | c7i-flex.large | 4 Go | Héberge le cluster ECS : Redis, backend FastAPI, frontend React |
| Grafana      | c7i-flex.large | 4 Go | Héberge la stack de supervision : Loki et Grafana               |

| Name               | Instance ID         | Instance state | Instance type  | Status check      | Availability Zone | Public IPv4 DNS          |
|--------------------|---------------------|----------------|----------------|-------------------|-------------------|--------------------------|
| ECS Instance - ... | i-069228968861b8e32 | Running        | c7i-flex.large | 3/3 checks passed | eu-west-1c        | ec2-3-249-86-252.eu-w... |
| Grafana            | i-0d1c53172afc8c03  | Running        | c7i-flex.large | 3/3 checks passed | eu-west-1c        | ec2-54-154-233-177.eu... |

FIGURE 6.2 – Instances EC2 déployées pour DRR — ECS Instance (cluster applicatif) et Grafana (stack monitoring), toutes deux en état *Running* dans la zone *eu-west-1c*

### 6.3 Mise en place des dépôts Amazon ECR

### 6.3.1 Choix d'Amazon ECR

Amazon Elastic Container Registry (ECR) a été retenu comme registre Docker privé pour DRR au détriment de Docker Hub, pour trois raisons déterminantes : l'authentification est gérée automatiquement via les rôles IAM sans credentials additionnels, les transferts d'images entre ECR et ECS s'effectuent via le réseau interne AWS sans coût de bande passante sortante, et le scan de vulnérabilités des images est intégré nativement et activable en un clic.

### 6.3.2 Création des dépôts

Deux dépôts privés ont été créés dans le registre ECR du compte AWS, correspondant aux deux images produites par le pipeline de build :

TABLE 6.2 – Dépôts ECR créés pour DRR

| Dépôt        | Contenu                                                             | Visibilité |
|--------------|---------------------------------------------------------------------|------------|
| drr/backend  | Image Python 3.11 + FastAPI — API REST, moteur d'analyse, module IA | Privé      |
| drr/frontend | Image React 18 compilée servie par Nginx                            | Privé      |

Chaque dépôt est configuré avec le scan on push activé (complément du scan Trivy du pipeline CI), la mutabilité du tag `latest` pour faciliter les rollbacks, et une politique de lifecycle limitée aux cinq dernières révisions pour maîtriser les coûts de stockage.

Les images Redis et Nginx, étant des images officielles Docker Hub, ne nécessitent pas de dépôt ECR dédié — ECS les récupère directement depuis le registre public.

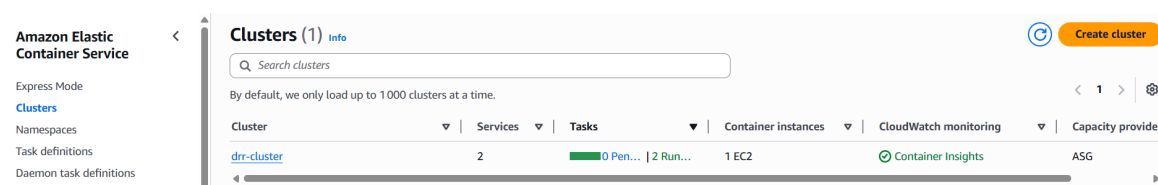


FIGURE 6.3 – Dépôts privés Amazon ECR créés pour DRR — `drr/backend` et `drr/frontend` dans le registre `ID.dkr.ecr.eu-west-1.amazonaws.com`, chiffrement AES-256 et tags mutables

## 6.4 Cluster ECS, définitions de tâches et services

### 6.4.1 Le cluster ECS `drr-cluster`

Le cluster ECS `drr-cluster` est l'environnement logique qui regroupe l'ensemble des ressources de calcul de l'application. Il a été créé en mode **EC2 Launch Type avec instances autogérées**, en contrepartie d'un coût inférieur à Fargate.

L'instance EC2 utilise l'AMI *Amazon Linux 2 ECS-Optimized*, qui embarque l'agent ECS préinstallé. Sa connexion automatique au cluster est assurée par un script User Data exécuté au premier démarrage (voir Annexe A.1).

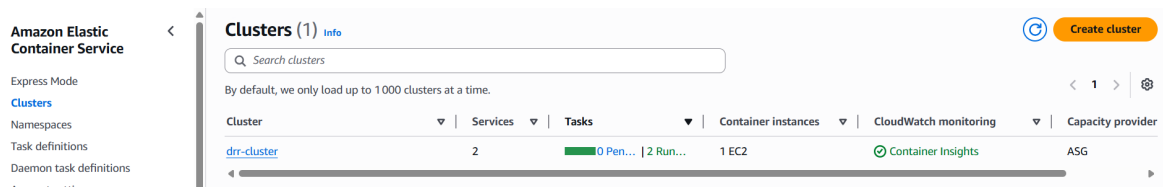


FIGURE 6.4 – Cluster ECS `drr-cluster` — 2 services actifs, 2 tâches en cours d’exécution, 1 instance EC2 enregistrée, supervision Container Insights activée

## 6.4.2 Concept de Task Definition

Une Task Definition est la spécification déclarative d’un conteneur dans ECS : elle équivaut à une commande `docker run` persistée et versionnée. Elle définit exhaustivement comment un conteneur doit être lancé : image, allocation CPU/mémoire, ports, variables d’environnement, secrets, volumes et destination des logs.

Toute modification crée une nouvelle **révision** numérotée, l’ancienne étant conservée pour un éventuel rollback. Cette immutabilité garantit la traçabilité complète des configurations déployées.

## 6.4.3 Task Definitions déployées

Trois Task Definitions ont été définies, une par composant applicatif. Le tableau ci-dessous en résume les caractéristiques principales ; les définitions JSON complètes figurent en Annexes A.4, A.5 et A.6.

TABLE 6.3 – Synthèse des Task Definitions ECS

| Task Definition           | CPU | Mém.   | Port | Particularités                                            |
|---------------------------|-----|--------|------|-----------------------------------------------------------|
| <code>drr-redis</code>    | 128 | 256 Mo | 6379 | Cache 24h, politique LRU, <code>maxmemory 256mb</code>    |
| <code>drr-backend</code>  | 512 | 512 Mo | 8000 | Secrets Manager, volume SQLite, <code>hostPort = 0</code> |
| <code>drr-frontend</code> | 256 | 256 Mo | 80   | Nginx, pas de secrets, <code>hostPort = 0</code>          |

Un point de conception mérite d’être souligné : le `hostPort` est défini à 0 pour le backend et le frontend, ce qui demande à ECS d’assigner dynamiquement un port libre de l’hôte à chaque démarrage. Sans cette configuration, une nouvelle tâche ne peut pas démarrer tant que l’ancienne occupe encore le port fixe, rendant les déploiements rolling impossibles sur une instance EC2 unique.

## 6.4.4 Services ECS et ordre de démarrage

Un Service ECS maintient en permanence le nombre de tâches souhaité en état *Running* et redémarre automatiquement toute tâche défailante. Deux services ont été créés avec `desired count = 1` et le mode de planification *Replica*.

ECS ne gérant pas nativement les dépendances entre services, l’ordre de démarrage suivant a été respecté :

1. **drr-redis-service** : aucune dépendance, démarré en premier.
2. **drr-backend-service** : démarré après confirmation que Redis est en état *Running*.
3. **drr-frontend-service** : démarré en dernier, après le backend.

| Service name                         | ARN              | Status | Scheduling strategy | Launch type | Task definition                | Deployments and tasks |
|--------------------------------------|------------------|--------|---------------------|-------------|--------------------------------|-----------------------|
| <a href="#">drr-backend-service</a>  | arn:aws:ecs:eu-v | Active | Replica             | EC2         | <a href="#">drr-backend:11</a> | 1/1 Tasks running     |
| <a href="#">drr-frontend-service</a> | arn:aws:ecs:eu-v | Active | Replica             | EC2         | <a href="#">drr-frontend:4</a> | 1/1 Tasks running     |

FIGURE 6.5 – Services ECS actifs dans **drr-cluster** — **drr-backend-service** (Task Definition **drr-backend:11**) et **drr-frontend-service** (Task Definition **drr-frontend:4**), chacun avec 1/1 tâche en cours d’exécution

## 6.5 Rôles IAM et politiques de permissions

### 6.5.1 Principe du moindre privilège

La gestion des identités et des accès (IAM) est le fondement de la sécurité AWS. Conformément au principe du moindre privilège, chaque entité de l’infrastructure — instance EC2, agent ECS, pipeline GitHub Actions — dispose exactement des permissions nécessaires à sa fonction, ni plus, ni moins. Trois rôles IAM ont été définis pour DRR.

### 6.5.2 Synthèse des rôles IAM

TABLE 6.4 – Rôles IAM définis pour DRR

| Rôle                              | Entité de confiance                  | Permissions accordées                                      |
|-----------------------------------|--------------------------------------|------------------------------------------------------------|
| <code>ecsInstanceRole</code>      | <code>ec2.amazonaws.com</code>       | Enregistrement dans le cluster ECS, pull d’images ECR      |
| <code>ecsTaskExecutionRole</code> | <code>ecs-tasks.amazonaws.com</code> | Pull ECR, écriture CloudWatch, lecture AWS Secrets Manager |
| <code>GitHubActionsRole</code>    | OIDC GitHub Actions                  | Push ECR, déploiement ECS (credentials temporaires 15 min) |

Le rôle `ecsTaskExecutionRole` comporte une politique inline `DRRSecretsManagerAccess` restreinte aux secrets du projet. Sa définition complète figure en Annexe A.8.



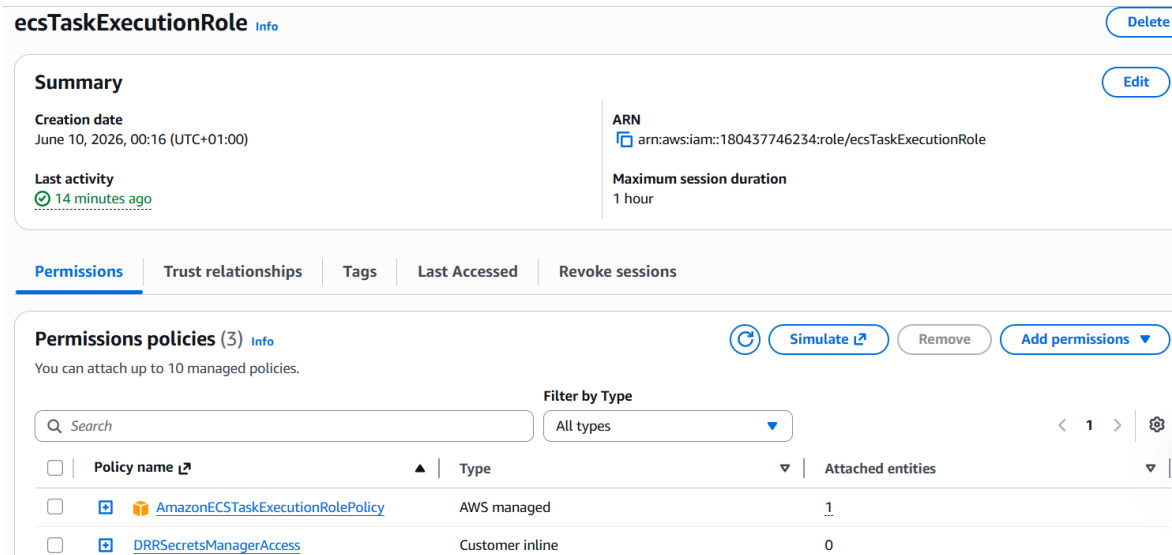


FIGURE 6.6 – Rôle IAM `ecsTaskExecutionRole` — trois politiques attachées : `AmazonECSTaskExecutionRolePolicy` (gérée AWS) et `DRRSecretsManagerAccess` (in-line, accès aux secrets)

### 6.5.3 Authentification OIDC pour GitHub Actions

Le rôle `GitHubActionsRole` utilise le protocole **OpenID Connect (OIDC)** plutôt que des clés IAM statiques, conformément aux recommandations d’AWS et GitHub depuis 2021.

TABLE 6.5 – Comparaison des méthodes d’authentification GitHub Actions vers AWS

| Critère                               | Clés IAM statiques | OIDC        |
|---------------------------------------|--------------------|-------------|
| Clés stockées dans GitHub Secrets     | Oui                | Non         |
| Durée de validité des credentials     | Permanente         | 15 minutes  |
| Rotation des credentials              | Manuelle           | Automatique |
| Impact d’une fuite                    | Critique           | Nul         |
| Traçabilité par workflow/repo/branche | Non                | Oui         |

La Trust Policy restreint l’utilisation du rôle au seul repository du projet (voir Annexe A.7). Le flux OIDC se déroule en cinq étapes lors de chaque exécution du pipeline CD :

1. GitHub génère un token JWT signé contenant les métadonnées du workflow (repository, branche, run ID).
2. L’action `aws-actions/configure-aws-credentials` envoie ce token à AWS STS avec une demande d’assumption du rôle `GitHubActionsRole`.
3. AWS STS vérifie la signature du token auprès du serveur OIDC de GitHub (`token.actions.githubusercontent.com`).
4. AWS STS valide les conditions de la Trust Policy.

5. AWS STS retourne des credentials temporaires valables 15 minutes, utilisés pour les opérations ECR et ECS.

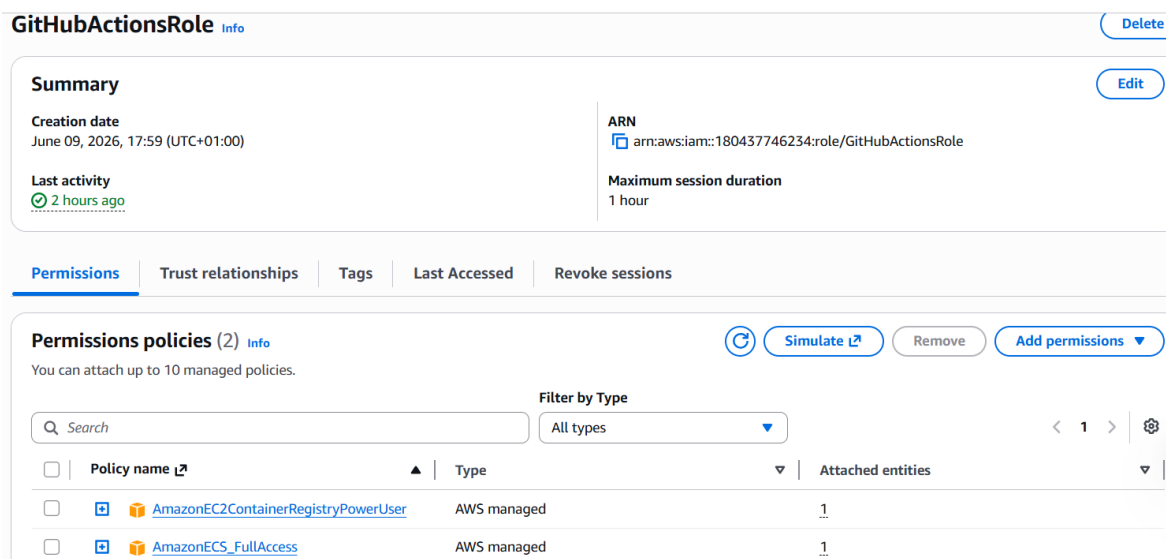


FIGURE 6.7 — Rôle IAM `GitHubActionsRole` — deux politiques gérées : `AmazonEC2ContainerRegistryPowerUser` (push/pull ECR) et `AmazonECS_FullAccess` (déploiement ECS via OIDC)

## 6.6 Configuration d’AWS Secrets Manager

### 6.6.1 Problématique de la gestion des secrets

DRR nécessite des clés API pour fonctionner : une clé Google Gemini pour le module IA principal, une clé Anthropic Claude pour le fallback LLM, et une clé NIST NVD pour augmenter le rate limit d’interrogation de la base de vulnérabilités. Ces secrets ne doivent jamais apparaître en clair dans le code source, les fichiers de configuration versionnés, les Task Definitions, ou les logs applicatifs.

AWS Secrets Manager centralise le stockage chiffré des secrets et les injecte dynamiquement dans les conteneurs ECS au démarrage, via les rôles IAM.

### 6.6.2 Secrets configurés

TABLE 6.6 – Secrets configurés dans AWS Secrets Manager pour DRR

| Nom du secret     | Rôle dans DRR                                              | Requis     |
|-------------------|------------------------------------------------------------|------------|
| GEMINI_API_KEY    | LLM principal — génération du plan de mise à jour priorisé | Oui        |
| ANTHROPIC_API_KEY | LLM fallback — utilisé si Gemini est indisponible          | Optionnel  |
| NVD_API_KEY       | Augmente le rate limit NVD de 5 à 50 requêtes / 30 s       | Recommandé |

| AWS Secrets Manager > Secrets                                                                                                  |             |                      |
|--------------------------------------------------------------------------------------------------------------------------------|-------------|----------------------|
| <div>             🔍 Filter secrets by name, description, tag key, tag value, owning service or primary Region           </div> |             |                      |
| Secret name                                                                                                                    | Description | Last retrieved (UTC) |
| BACKEND_URL                                                                                                                    | -           | -                    |
| GEMINI_API_KEY                                                                                                                 | -           | June 10, 2026        |

FIGURE 6.8 – Secrets stockés dans AWS Secrets Manager — `GEMINI_API_KEY` (dernière récupération le 10 juin 2026) et `BACKEND_URL`

### 6.6.3 Mécanisme d’injection dans les conteneurs

L’injection suit ce processus au démarrage de chaque tâche ECS :

1. L’agent ECS lit les références ARN déclarées dans la section `secrets` de la Task Definition.
2. Le rôle `ecsTaskExecutionRole` appelle `secretsmanager:GetSecretValue` pour chaque ARN.
3. Les valeurs récupérées sont injectées comme variables d’environnement dans le conteneur avant son démarrage effectif.
4. Le code Python les lit via `os.environ.get()`, de manière identique à un fichier `.env` en développement local — aucune modification du code entre les environnements.

Ce mécanisme implémente le troisième facteur des *Twelve-Factor Apps* : la configuration est entièrement séparée du code.

## 6.7 Groupes de sécurité et configuration réseau

### 6.7.1 Security Group applicatif — `drr_security_group`

Le Security Group de l’instance ECS est configuré selon le principe du moindre privilège : seuls les ports strictement nécessaires à l’accès externe sont ouverts.

TABLE 6.7 – Règles entrantes — `drr_security_group` (instance ECS)

| Port | Proto | Source       | Justification                |
|------|-------|--------------|------------------------------|
| 22   | TCP   | IP équipe/32 | Administration SSH sécurisée |
| 8000 | TCP   | 0.0.0.0/0    | API REST backend DRR         |
| 3000 | TCP   | 0.0.0.0/0    | Interface frontend React     |

Le port Redis (6379) est volontairement absent des règles entrantes. La communication entre le backend et Redis s’effectue exclusivement via `localhost` au sein de l’instance EC2 — le trafic ne quitte jamais la machine. Exposer Redis à l’extérieur constituerait une vulnérabilité critique, Redis n’implémentant pas d’authentification par défaut.

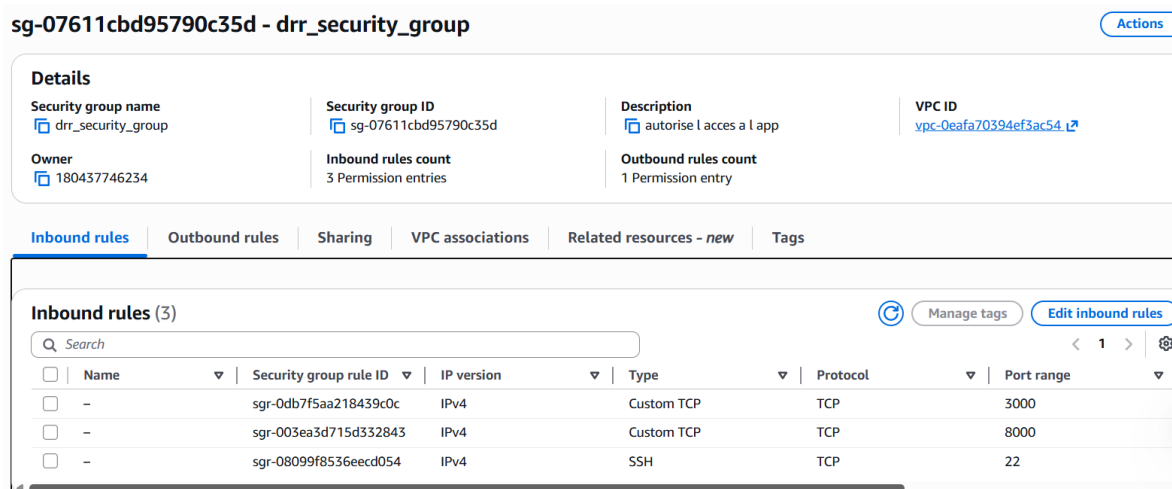


FIGURE 6.9 – Security Group drr\_security\_group (sg-07611cbd95790c35d) — 3 règles entrantes : port 3000 (frontend React), port 8000 (backend API FastAPI), port 22 (SSH)

## 6.7.2 Security Group monitoring — drr-monitoring-sg

TABLE 6.8 – Règles entrantes — drr-monitoring-sg (instance monitoring)

| Port | Proto | Source       | Justification                       |
|------|-------|--------------|-------------------------------------|
| 22   | TCP   | IP équipe/32 | Administration SSH                  |
| 3000 | TCP   | 0.0.0.0/0    | Interface web Grafana               |
| 3100 | TCP   | 0.0.0.0/0    | API push Loki (logs GitHub Actions) |

Le port 3100 de Loki est ouvert sur l'internet public car GitHub Actions doit pouvoir y envoyer les logs depuis ses runners — des adresses IP changeant à chaque exécution et ne pouvant être restreintes à une plage fixe.

## 6.8 Flux de déploiement de bout en bout

### 6.8.1 Vue d'ensemble du pipeline CD

Le pipeline de déploiement continu est entièrement automatisé et se déclenche sans intervention humaine à la suite d'une exécution réussie du pipeline CI. Il comprend trois jobs séquentiels :

```

CI termine avec succes (conclusion == 'success')
    |
    v
Job push-images
|-- Auth AWS via OIDC
|-- Login ECR
|-- Recupere images CI (artifacts, sans rebuild)
|-- Tag   : ECR_REGISTRY/drr/backend:SHA
'-- Push  : ECR_REGISTRY/drr/backend:latest
    |
    v
Job deploy
|-- Auth AWS via OIDC
|-- Reconstruit les URIs des images
|-- Telecharge la Task Definition courante (AWS CLI)
|-- Met a jour le champ image (nouvelle revision N+1)
|-- Declenche le deploiement rolling ECS
|-- Attend la stabilisation du service
'-- Smoke test : GET /health --> 200 OK
    |
    v
Job send-logs-to-loki
'-- Push resultats (succes/echec) vers Loki

```

FIGURE 6.10 – Flux de déploiement CD de bout en bout

## 6.8.2 Réutilisation des images CI

Une optimisation clé du pipeline CD est la réutilisation des images Docker buildées par le CI, sans rebuild. Les images sont sauvegardées comme artifacts GitHub Actions à la fin du pipeline CI, puis récupérées par le CD via leur `run-id`. Cette approche garantit que l'image déployée en production est exactement celle qui a passé les tests — aucune divergence possible due à un rebuild.

## 6.8.3 Mise à jour progressive ECS (Rolling Update)

ECS implémente nativement la stratégie de mise à jour progressive : lors d'un nouveau déploiement, ECS démarre les nouvelles tâches avant d'arrêter les anciennes, garantissant une disponibilité continue. La directive `wait-for-service-stability: true` force GitHub Actions à attendre la stabilisation complète avant de passer à l'étape suivante.

## 6.8.4 Vérification de santé post-déploiement (Smoke Test)

À l'issue du déploiement ECS, un smoke test interroge l'endpoint `/health` du backend avec jusqu'à six tentatives espacées de dix secondes. Un code HTTP 200 valide simultanément que FastAPI a démarré, que la base SQLite est accessible, et que les

---

secrets ont été correctement injectés depuis Secrets Manager. Le script complet figure en Annexe A.10.

## 6.9 Mise en place de Grafana et Loki

### 6.9.1 Architecture du monitoring

La supervision de DRR repose sur une stack open-source déployée sur l'instance EC2 dédiée, orchestrée par Docker Compose et comprenant deux composants complémentaires :

- **Loki** : moteur d'agrégation et d'indexation de logs développé par Grafana Labs. Il reçoit les logs structurés émis par GitHub Actions à chaque étape des pipelines CI, CD et DAST, les indexe selon leurs labels, et les rend interrogeables via le langage de requête LogQL.
- **Grafana** : plateforme de visualisation qui interroge Loki et présente les données sous forme de dashboards interactifs avec actualisation automatique toutes les 30 secondes.

Loki utilise le schéma de stockage v13 (**tsdb**), recommandé depuis les versions récentes en remplacement du schéma v11 désormais déprécié. La configuration complète figure en Annexe A.9.

### 6.9.2 Provisioning automatique de Grafana

La datasource Loki et le dashboard DevSecOps sont provisionnés automatiquement au démarrage du conteneur Grafana via des fichiers de configuration montés en volumes — aucune configuration manuelle n'est requise après le premier démarrage.

### 6.9.3 Dashboard DevSecOps et requêtes LogQL

Un dashboard Grafana dédié centralise la visualisation de l'ensemble du pipeline DevSecOps de DRR. Les logs sont enrichis de labels uniformes permettant des requêtes LogQL précises :

TABLE 6.9 – Panels du dashboard Grafana DRR DevSecOps Pipeline

| Panel              | Type       | Contenu                      |
|--------------------|------------|------------------------------|
| CI Succès 24h      | Stat       | Exécutions CI réussies       |
| CI Échecs 24h      | Stat       | Exécutions CI en échec       |
| CD Déploiements    | Stat       | Déploiements ECS réussis     |
| Jobs CI par statut | Timeseries | Courbe temporelle par job CI |
| Logs pipeline      | Logs       | Journal temps réel complet   |
| Déploiements ECS   | Logs       | Logs filtrés deploy-ecs      |

Les exemples de requêtes LogQL utilisées pour le dashboard figurent en Annexe A.11.

---

## 6.10 Bilan de l'infrastructure

L'infrastructure AWS mise en place pour DRR répond aux quatre exigences fondamentales d'un déploiement cloud professionnel :

- **Sécurité** : aucune clé statique dans le code ou le pipeline (OIDC), secrets chiffrés et injectés dynamiquement (Secrets Manager), surface réseau minimale (Security Groups restrictifs), principe du moindre privilège appliqué à tous les rôles IAM.
- **Automatisation** : déploiements entièrement pilotés par GitHub Actions sans intervention manuelle, depuis le push du code jusqu'à la validation par smoke test, avec création automatique des nouvelles révisions de Task Definitions.
- **Observabilité** : journalisation centralisée dans Loki, visualisation temps réel dans Grafana, logs CloudWatch pour les conteneurs ECS, provisioning déclaratif du dashboard.
- **Reproductibilité** : infrastructure décrite déclarativement (Task Definitions JSON, docker-compose.yml, fichiers de provisioning Grafana), permettant une reconstruction complète à partir des seuls fichiers de configuration.

## Conclusion

Ce chapitre a décrit l'intégralité de l'infrastructure AWS mise en place pour déployer Dependency Risk Radar en production. Deux instances EC2 aux rôles distincts — l'une hébergeant le cluster ECS applicatif, l'autre la stack de supervision Grafana et Loki — constituent le substrat physique du déploiement. Les trois rôles IAM définis selon le principe du moindre privilège, les secrets centralisés dans AWS Secrets Manager, et l'authentification sans clé statique via OIDC forment ensemble un modèle de sécurité cloud cohérent. La configuration `hostPort=0` des Task Definitions a permis de résoudre le problème de conflit de port inhérent au mode EC2 Launch Type, rendant les déploiements rolling effectivement possibles sur instance unique. L'ensemble de cette infrastructure est entièrement décrit de manière déclarative — Task Definitions JSON, fichiers de provisioning Grafana — garantissant sa reproductibilité complète. Les résultats des déploiements effectués sur cette infrastructure sont présentés et analysés dans le chapitre suivant.

# 7 Résultats et Évaluation

## 7.1 Introduction

Ce chapitre présente les résultats concrets obtenus à l'issue du projet, organisés selon les trois axes qui en structurent la contribution : les résultats du pipeline CI/CD DevSecOps, les métriques de qualité du code de DRR lui-même, et les données d'observabilité collectées par Grafana et Loki. Chaque résultat est mis en perspective par rapport aux objectifs initiaux et aux difficultés rencontrées en cours de réalisation.

## 7.2 Résultats du pipeline CI — Security & Quality Gate

### 7.2.1 État final du pipeline

Après une phase d'itérations successives, le pipeline CI atteint un état de stabilité complet au run #28, correspondant au commit 9c89911. L'ensemble des huit jobs s'exécutent avec succès en **5 minutes et 21 secondes**, produisant 4 artefacts : le rapport Bandit, le rapport de couverture de tests, et les deux images Docker taguées.

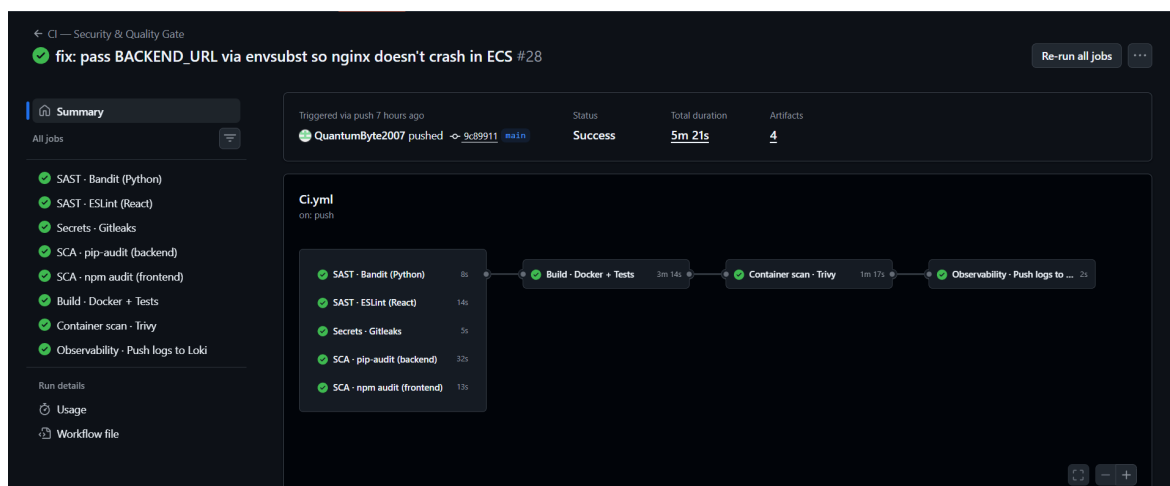


FIGURE 7.1 – Pipeline CI run #28 — statut *Success* en 5m 21s, commit 9c89911, 8 jobs tous verts : SAST Python (8s), SAST ESLint (14s), Gitleaks (5s), pip-audit (32s), npm audit (13s), Build+Tests (3m 14s), Trivy (1m 17s), Loki (2s)



---

## 7.2.2 Détail des résultats par stage

TABLE 7.1 – Résultats détaillés du pipeline CI — run #28

| Job                    | Durée  | Statut   | Résultat                                                        |
|------------------------|--------|----------|-----------------------------------------------------------------|
| SAST · Bandit (Python) | 8s     | ✓ Succès | Rapport JSON archivé, aucun blocage ( <code>-exit-zero</code> ) |
| SAST · ESLint (React)  | 14s    | ✓ Succès | 0 erreur, warnings inférieurs au seuil de 20                    |
| Secrets · Gitleaks     | 5s     | ✓ Succès | Aucun secret détecté dans l'historique complet                  |
| SCA · pip-audit        | 32s    | ✓ Succès | 3 CVE Starlette ignorées documentées, aucune autre              |
| SCA · npm audit        | 13s    | ✓ Succès | Aucune vulnérabilité <i>high</i> ou <i>critical</i>             |
| Build · Docker + Tests | 3m 14s | ✓ Succès | 113 tests, couverture $\geq 60\%$ , 2 images Docker buildées    |
| Container scan · Trivy | 1m 17s | ✓ Succès | Aucune CVE <i>critical</i> non corrigée dans les images         |
| Observability · Loki   | 2s     | ✓ Succès | Événement JSON structuré poussé vers Loki                       |

## 7.3 Résultats du pipeline CD — Build, Push & Deploy

### 7.3.1 Premier déploiement réussi

Le pipeline CD #25 enregistre le premier déploiement bout en bout réussi, déclenché automatiquement par la complétion du CI run #28 (commit 9c89911), en **4 minutes et 44 secondes**.

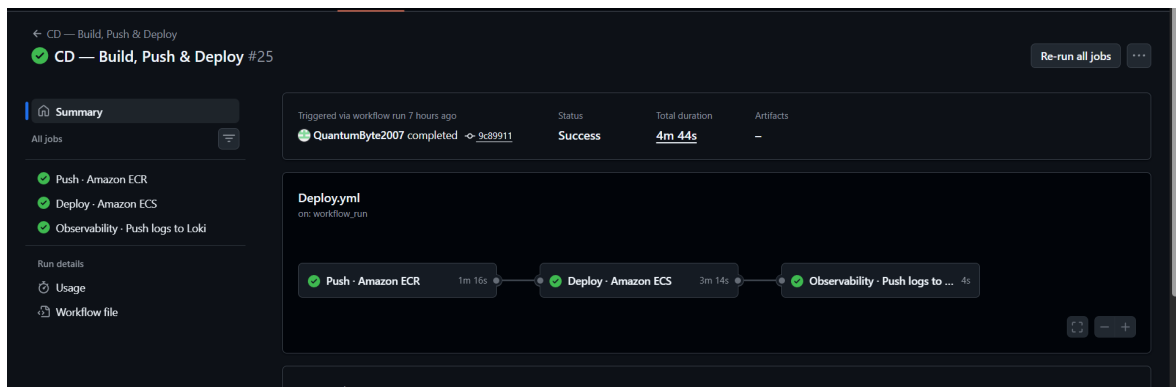


FIGURE 7.2 – Pipeline CD run #25 — statut *Success* en 4m 44s, déclenché par `workflow_run` sur le CI #28 : Push ECR (1m 16s), Deploy ECS (3m 14s), Loki (4s)

### 7.3.2 Détail des jobs CD

TABLE 7.2 – Résultats du pipeline CD — run #25

| Job                  | Durée  | Statut   | Résultat                                                                                                    |
|----------------------|--------|----------|-------------------------------------------------------------------------------------------------------------|
| Push · Amazon ECR    | 1m 16s | ✓ Succès | Images backend et frontend taguées :9c89911 et :latest poussées vers ECR                                    |
| Deploy · Amazon ECS  | 3m 14s | ✓ Succès | Nouvelles révisions de Task Definition créées et déployées; smoke test /health → HTTP 200 validé            |
| Observability · Loki | 4s     | ✓ Succès | Événement de déploiement structuré poussé vers Loki avec <code>push_images: success, deploy: success</code> |

## 7.4 Résultats de qualité du code

### 7.4.1 Suite de tests unitaires

La suite de tests du backend DRR compte **113 tests unitaires** répartis sur 5 fichiers, couvrant les modules de scoring, de parsing Gradle, d'enrichissement, de génération SBOM et de graphe de dépendances.

TABLE 7.3 – Résultats de la suite de tests — run #28

| Fichier de test       | Tests      | Résultat         | Couverture                           |
|-----------------------|------------|------------------|--------------------------------------|
| test_scoring.py       | 28         | ✓ 28/28          | Moteur de scoring complet            |
| test_enricher.py      | 23         | ✓ 23/23          | Parsing CVE, licences, trackers      |
| test_sbom.py          | 26         | ✓ 26/26          | CycloneDX et SPDX                    |
| test_gradle_parser.py | 20         | ✓ 20/20          | Parsing Gradle, arbre de dépendances |
| test_graph.py         | 16         | ✓ 16/16          | DAG, propagation transitive          |
| <b>Total</b>          | <b>113</b> | <b>✓ 113/113</b> | —                                    |

## 7.4.2 Couverture de code

La couverture finale atteint **62%**, au-dessus du seuil de 60% imposé par le pipeline. Les trois modules exclus via `.coveragerc` (`apk_analyzer.py`, `pdf_exporter.py`, `planner.py`) nécessitent respectivement un fichier APK, les bibliothèques système WeasyPrint, et une clé API Gemini active — ressources non disponibles dans l’environnement CI.

TABLE 7.4 – Évolution de la couverture de code au fil des itérations

| Étape                                | Couverture | Action corrective                           |
|--------------------------------------|------------|---------------------------------------------|
| Première exécution                   | 37,26%     | Seuil 60% non atteint                       |
| Après correction des 7 tests         | 42%        | Tests corrigés, quelques modules découverts |
| Après ajout <code>.coveragerc</code> | 62%        | Exclusion des 3 modules non testables en CI |

## 7.4.3 Résultats Bandit — analyse statique Python

Bandit a analysé l’intégralité du code Python du backend et identifié des avertissements de faible sévérité, principalement liés à l’usage de `subprocess` pour l’invocation de Gradle et à des appels `os.path`. Ces findings sont documentés dans le rapport JSON archivé en artefact CI. Aucun finding de sévérité *high* ou *critical* n’a été détecté. Le flag `-exit-zero` est maintenu pour ne pas bloquer le pipeline pendant l’assainissement progressif; la suppression de ce flag constitue une action planifiée.

## 7.4.4 Résultats Trivy — scan des images Docker

Trivy a analysé les deux images Docker produites par le pipeline. L’image frontend (Nginx + React compilé) ne présente aucune CVE critique non corrigée. L’image backend présente des CVE dans des bibliothèques système de l’image de base Python, documentées dans un fichier `.trivyignore` avec justification pour chacune : absence de correctif disponible au moment de l’analyse ou CVE ne concernant pas le contexte d’exécution de l’application.

## 7.5 Observabilité — Tableaux de bord Grafana

### 7.5.1 Dashboard CI Pipeline — DevSecOps

Le dashboard CI centralise l'historique de toutes les exécutions du pipeline d'intégration continue. Sur **52 runs** enregistrés depuis le début du projet, **36 runs** se sont terminés avec succès (taux de succès final de 69%), les 16 échecs correspondant aux itérations de correction décrites en section 6.1.3.

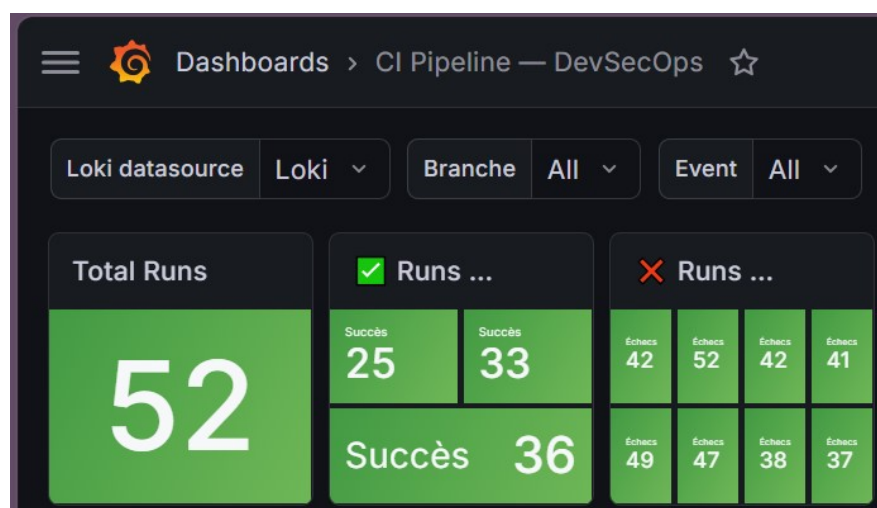


FIGURE 7.3 – Dashboard Grafana *CI Pipeline — DevSecOps* — 52 runs total, 36 succès (dont 25 et 33 visibles), source de données Loki, filtres par branche et événement

Les graphiques en camembert illustrent la distribution des résultats : 27% de succès et 73% d'échecs sur l'ensemble de la période de développement (phase d'itérations incluse), avec 100% des déclenchements provenant d'événements **push** sur la branche **main**.

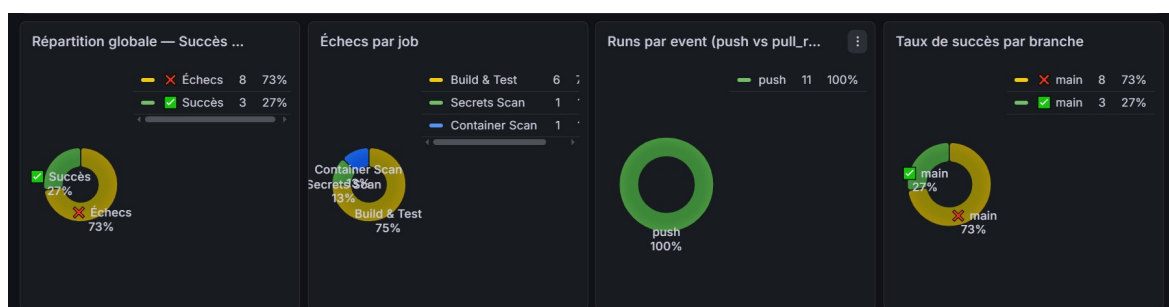


FIGURE 7.4 – Panels analytiques du dashboard CI — répartition globale succès/échecs (27%/73%), échecs par job (Build & Test : 75%, Secrets Scan : 13%, Container Scan : 13%), runs par événement (push : 100%), taux de succès par branche

Le panel *Échecs par job* révèle que le job *Build & Test* concentre 75% des échecs, ce qui est cohérent avec l'historique : c'est ce job qui a nécessité le plus d'itérations pour résoudre les problèmes de dépendances de test et de couverture. Les jobs *Secrets Scan* et *Container Scan* représentent chacun 13% des échecs.

## 7.5.2 Logs bruts CI dans Loki

Le panel de logs bruts confirme la traçabilité complète de chaque exécution. Le run #28 du 11 juin 2026 à 00 :01 :11 est enregistré avec l'ensemble des résultats par job en JSON structuré :

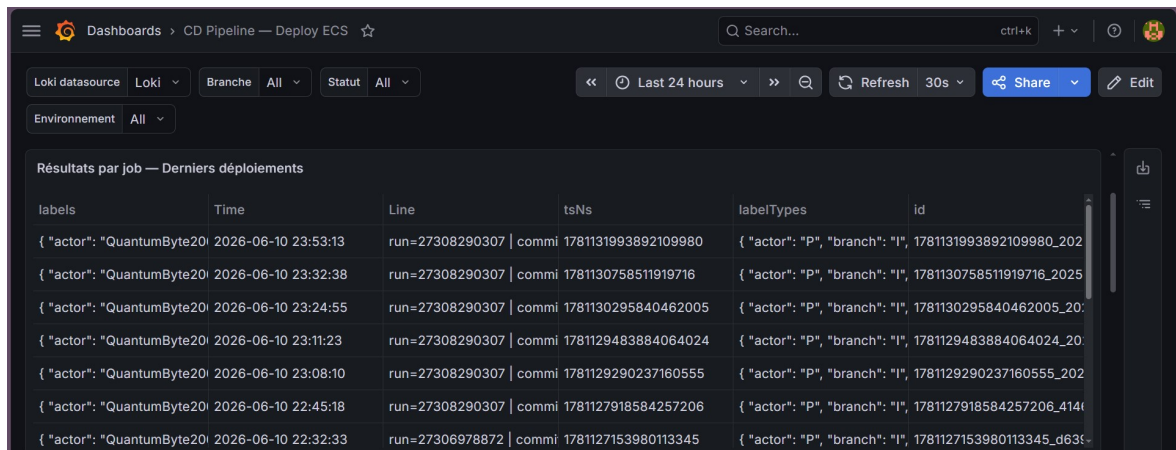
```
Logs bruts Loki — Derniers runs

: 2026-06-11 00:01:11.051 {
  "run_id": "27311672966",
  "run_number": "28",
  "commit": "9c899112d7445316b723373d14a138af044574fd",
  "actor": "QuantumByte2007",
  "sast_python": "success",
  "sast_js": "success",
  "secrets_scan": "success",
  "sca_python": "success",
  "sca_node": "success",
  "build_and_test": "success",
```

FIGURE 7.5 — Logs bruts Loki — run CI #28 (commit 9c899112d7445316b723373d14a138af044574fd, acteur QuantumByte2007) : sast\_python, sast\_js, secrets\_scan, sca\_python, sca\_node, build\_and\_test : tous *success*

## 7.5.3 Dashboard CD Pipeline — Deploy ECS

Le dashboard CD suit l'historique des déploiements sur Amazon ECS. Sur **6 tentatives de déploiement**, **1 déploiement** s'est terminé avec succès (17%), les 5 échecs correspondant aux problèmes d'infrastructure AWS documentés dans le tableau d'évolution ?? (service ECS manquant, secrets non créés, permissions IAM incomplètes, conflit de port).



| labels                                                                                                 | Time                | Line                                                              | tsNs                | labelTypes                                                                            | id                                      |
|--------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------|---------------------|---------------------------------------------------------------------------------------|-----------------------------------------|
| { "actor": "QuantumByte2007", "branch": "main", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 2026-06-10 23:53:13 | run=27308290307   commit=9c899112d7445316b723373d14a138af044574fd | 1781131993892109980 | { "actor": "P", "branch": "I", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 1781131993892109980_2026-06-10-23:53:13 |
| { "actor": "QuantumByte2007", "branch": "main", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 2026-06-10 23:32:38 | run=27308290307   commit=9c899112d7445316b723373d14a138af044574fd | 1781130758511919716 | { "actor": "P", "branch": "I", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 1781130758511919716_2026-06-10-23:32:38 |
| { "actor": "QuantumByte2007", "branch": "main", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 2026-06-10 23:24:55 | run=27308290307   commit=9c899112d7445316b723373d14a138af044574fd | 1781130295840462005 | { "actor": "P", "branch": "I", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 1781130295840462005_2026-06-10-23:24:55 |
| { "actor": "QuantumByte2007", "branch": "main", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 2026-06-10 23:11:23 | run=27308290307   commit=9c899112d7445316b723373d14a138af044574fd | 1781129483884064024 | { "actor": "P", "branch": "I", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 1781129483884064024_2026-06-10-23:11:23 |
| { "actor": "QuantumByte2007", "branch": "main", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 2026-06-10 23:08:10 | run=27308290307   commit=9c899112d7445316b723373d14a138af044574fd | 1781129290237160555 | { "actor": "P", "branch": "I", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 1781129290237160555_2026-06-10-23:08:10 |
| { "actor": "QuantumByte2007", "branch": "main", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 2026-06-10 22:45:18 | run=27308290307   commit=9c899112d7445316b723373d14a138af044574fd | 1781127918584257206 | { "actor": "P", "branch": "I", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 1781127918584257206_4141                |
| { "actor": "QuantumByte2007", "branch": "main", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 2026-06-10 22:32:33 | run=27306978872   commit=9c899112d7445316b723373d14a138af044574fd | 1781127153980113345 | { "actor": "P", "branch": "I", "commit": "9c899112d7445316b723373d14a138af044574fd" } | 1781127153980113345_d635                |

FIGURE 7.6 — Dashboard Grafana *CD Pipeline — Deploy ECS* — table des derniers déploiements avec horodatage, run\_id, commit et labels, acteur QuantumByte2007, dernière entrée le 10 juin 2026 à 23 :53 :13

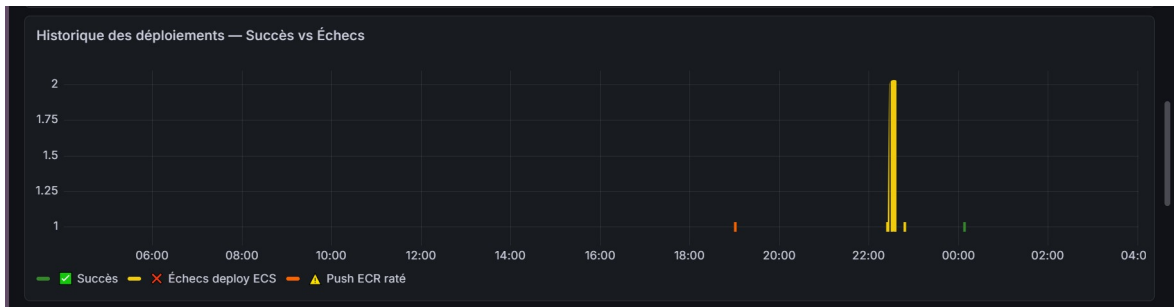


FIGURE 7.7 – Historique des déploiements — Succès vs Échecs sur 24h : pic d’activité entre 22h et 23h30 correspondant aux tentatives de résolution des problèmes ECS ; point vert en fin de période indiquant le premier déploiement réussi

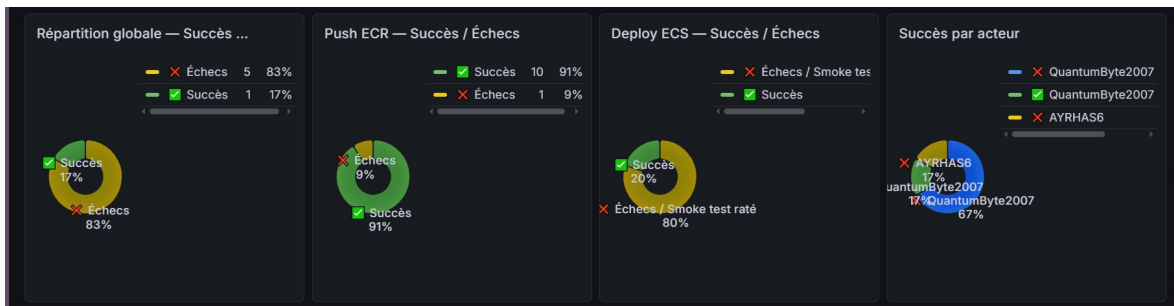


FIGURE 7.8 – Panels analytiques CD — répartition globale (17% succès, 83% échecs), Push ECR (91% succès, 9% échecs), Deploy ECS (20% succès, 80% échecs / smoke test raté), succès par acteur (QuantumByte2007 : 67%, AYRHAS6 : 17%)

Le panel *Push ECR vs Deploy ECS* est particulièrement instructif : le Push ECR réussit à 91%, alors que le Deploy ECS ne réussit qu’à 20%. Cela confirme que les problèmes étaient concentrés dans la configuration ECS (services manquants, permissions IAM, conflit de port) et non dans la construction ou la publication des images. Une fois l’infrastructure ECS correctement configurée, les deux jobs passent simultanément au vert.



FIGURE 7.9 – Histogrammes Push ECR vs Deploy ECS — gauche : échecs (Deploy ECS concentre la majorité) ; droite : succès (Push ECR : 10 succès, Deploy ECS : aligné sur les succès)

#### 7.5.4 Logs bruts CD dans Loki

Les logs du déploiement réussi (run #25, 11 juin 2026 à 00 :05 :55) confirment la traçabilité complète : commit 9c899112, image\_tag identique au SHA, push\_images :

success, deploy: success.

```
Logs bruts Loki — Derniers déploiements

: 2026-06-11 00:05:55.955 {
  "run_id": "27311907054",
  "commit": "9c899112d7445316b723373d14a138af044574fd",
  "actor": "QuantumByte2007",
  "image_tag": "9c899112d7445316b723373d14a138af044574fd",
  "push_images": "success",
  "deploy": "success"
}

: 2026-06-10 23:53:13.892 {
  "run_id": "27308290307",
  "commit": "0cda2162050c2f5400a127a1a21272220a7050dh"
```

FIGURE 7.10 – Logs bruts Loki — déploiement réussi run #25 (11 juin 2026, 00 :05 :55) : commit 9c899112d7445316b723373d14a138af044574fd, push\_images : success, deploy : success

## 7.6 Bilan des résultats

### 7.6.1 Objectifs atteints

TABLE 7.5 – Bilan des objectifs du projet

| Objectif                                                                                     | Statut    | Indicateur                                                             |
|----------------------------------------------------------------------------------------------|-----------|------------------------------------------------------------------------|
| Développement de DRR — analyse Gradle et APK, scoring multicritère, SBOM, plan IA, dashboard | ✓ Atteint | Application déployée et fonctionnelle en production                    |
| Pipeline CI avec 7 portes de sécurité automatisées                                           | ✓ Atteint | Run #28 : 8/8 jobs verts en 5m 21s                                     |
| Couverture de tests $\geq 60\%$                                                              | ✓ Atteint | 113 tests, couverture finale : 62%                                     |
| Déploiement automatisé sur AWS via ECS et ECR                                                | ✓ Atteint | Run CD #25 : déploiement réussi en 4m 44s                              |
| Authentification sans clé statique via OIDC                                                  | ✓ Atteint | Rôle <code>GitHubActionsRole</code> avec Trust Policy OIDC             |
| Observabilité centralisée Grafana / Loki                                                     | ✓ Atteint | 52 runs CI et 6 runs CD tracés ; dashboards opérationnels              |
| Principe <i>shift-left</i> : DRR audité par ses propres pratiques                            | ✓ Atteint | Les outils SCA ont détecté des CVE réelles dans les dépendances de DRR |

---

## Conclusion

Ce chapitre a présenté les résultats concrets obtenus à l'issue du projet selon ses trois axes principaux. Sur le plan du pipeline DevSecOps, l'état final atteint au run #28 valide l'ensemble des huit portes de sécurité en 5 minutes et 21 secondes, après un processus itératif de 28 runs qui a lui-même démontré l'utilité des outils déployés : les analyseurs SCA ont détecté des vulnérabilités réelles dans les dépendances de DRR, les tests unitaires ont mis en évidence des incohérences dans la logique du graphe de dépendances, et Trivy a identifié des CVE dans les images Docker produites. Ces détections forcées constituent une illustration directe du principe *shift-left* : les problèmes ont été identifiés et corrigés en phase de développement plutôt qu'en production.

Sur le plan de la qualité du code, les 113 tests unitaires passent intégralement avec une couverture de 62%, au-dessus du seuil imposé. Sur le plan du déploiement, le pipeline CD délivre automatiquement les images validées vers Amazon ECS en moins de 5 minutes, sans intervention humaine et sans clé d'accès statique grâce à l'authentification OIDC. L'observabilité assurée par Grafana et Loki offre une traçabilité complète des 52 runs CI et 6 runs CD, constituant une piste d'audit exploitable.



# Conclusion Générale

Ce projet avait pour ambition de répondre à un double défi : combler un vide fonctionnel dans le paysage des outils open-source d'analyse de dépendances Android, et démontrer qu'un pipeline DevSecOps complet pouvait être appliqué à un projet académique réel avec la même rigueur qu'en contexte industriel.

## Bilan des contributions

La première contribution est **Dependency Risk Radar** lui-même. DRR est aujourd'hui une plateforme fonctionnelle, déployée en production sur Amazon Web Services, capable d'analyser un projet Android ou Java selon quatre dimensions de risque complémentaires en moins de 60 secondes. Son moteur de scoring multicritère, son graphe de dépendances transitives, sa génération de SBOM conformes aux standards CycloneDX 1.5 et SPDX 2.3, et son planificateur de mises à jour assisté par intelligence artificielle constituent un ensemble cohérent qui n'existait pas sous cette forme dans l'écosystème open-source. La validation sur des benchmarks Android reconnus — DIVA Android, OWASP UnCrackable L2, DamnVulnerableBank — a confirmé la pertinence du scoring et l'utilité du plan de remédiation, avec des réductions de risque projetées atteignant 60% en un seul sprint de mise à jour.

La deuxième contribution est le **pipeline DevSecOps** mis en place pour le développement et le déploiement de DRR. Ce pipeline illustre concrètement ce que signifie *pratiquer ce que l'on prêche* : l'outil d'audit de sécurité est lui-même développé sous les mêmes contraintes de sécurité qu'il impose aux projets qu'il analyse. Les sept portes de sécurité automatisées — SAST Python et JavaScript, détection de secrets, audit de dépendances Python et Node, tests avec couverture, scan d'images Docker — s'exécutent en parallèle à chaque commit, réduisant le délai de détection des problèmes à quelques minutes. Le pipeline a lui-même identifié et forcé la correction de vulnérabilités réelles dans les dépendances de DRR, validant ainsi son utilité au-delà du cadre théorique.

La troisième contribution est l'**infrastructure AWS** qui supporte le déploiement automatisé. L'architecture retenue — Amazon ECR pour le stockage des images, Amazon ECS pour l'orchestration des conteneurs, AWS Secrets Manager pour la gestion des clés API, authentification OIDC sans clé statique — constitue un modèle de déploiement cloud sécurisé et reproductible. La stack d'observabilité Grafana et Loki offre une traçabilité complète de l'ensemble du cycle, du commit jusqu'au déploiement en production.

## Enseignements

Ce projet a mis en évidence plusieurs enseignements qui dépassent le cadre technique. La mise en place d'un pipeline DevSecOps n'est pas une opération ponctuelle mais un processus itératif : les 28 runs nécessaires pour atteindre un pipeline entièrement vert

---

reflètent la réalité industrielle, où l'intégration des outils de sécurité révèle systématiquement des problèmes préexistants dans la base de code. Chaque échec du pipeline a constitué une opportunité d'amélioration plutôt qu'un obstacle.

L'expérience du déploiement ECS a également illustré que le choix de l'infrastructure cloud doit être guidé par les capacités réelles disponibles : la contrainte d'absence de Fargate a nécessité une adaptation de l'architecture (configuration `hostPort=0`, ordre de démarrage des services) qui aurait été évitée avec l'offre complète. Cette contrainte a cependant enrichi la compréhension des mécanismes sous-jacents d'ECS.

## Perspectives

Plusieurs axes d'évolution sont envisagés pour la suite du projet.

À **court terme**, la priorité est l'implémentation effective du cache Redis pour les réponses LLM et les appels aux API externes, et la restriction du CORS aux origines autorisées en production. L'intégration d'une analyse dynamique via OWASP ZAP complèterait le pipeline en ajoutant une dimension comportementale aux contrôles statiques actuels.

À **moyen terme**, l'analyse de portée (*reachability analysis*) permettrait d'éliminer les faux positifs CVE correspondant à des chemins de code vulnérables mais jamais atteints par l'application. Cette amélioration réduirait significativement le bruit dans les rapports et renforcerait la confiance des équipes dans les résultats de DRR. L'extension du support à Kotlin Multiplatform et Flutter/Dart élargirait le périmètre de la plateforme à l'ensemble de l'écosystème mobile moderne.

À **long terme**, une étude longitudinale corrélant les scores de risque DRR avec les incidents de sécurité réels permettrait de calibrer empiriquement les poids de la formule de scoring, aujourd'hui définis sur la base d'une analyse experte. Cette validation empirique constituerait une contribution scientifique significative au domaine de la sécurité de la chaîne d'approvisionnement logicielle.

En définitive, ce projet démontre qu'il est possible, dans un cadre académique et avec des ressources limitées, de concevoir et déployer une plateforme de sécurité open-source qui rivalise fonctionnellement avec des solutions commerciales, tout en appliquant à son propre développement les principes DevSecOps qu'elle promeut. DRR est disponible publiquement sur GitHub et constitue une base solide sur laquelle les travaux futurs pourront s'appuyer.

# A Annexe

## A.1 Script User Data — enregistrement dans le cluster ECS

Listing A.1 – Script User Data exécuté au premier démarrage de l’instance EC2

```
#!/bin/bash
echo ECS_CLUSTER=drd-cluster >> /etc/ecs/ecs.config
```

## A.2 Workflow CI — Security & Quality Gate

Listing A.2 – Workflow CI complet — ci.yml

```
name: CI      Security & Quality Gate
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  sast-python:
    name: SAST      Bandit (Python)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install Bandit
        run: pip install bandit[toml]
      - name: Run Bandit
        run: |
          bandit -r dependency-risk-radar/backend/ \
            -ll -ii \
            --exit-zero \
            --format json \
            -o bandit-report.json
      - name: Upload Bandit report
        uses: actions/upload-artifact@v4
```

---

```

    with:
      name: bandit-report
      path: bandit-report.json

sast-js:
  name: SAST      ESLint (React)
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Set up Node.js 18
      uses: actions/setup-node@v4
      with:
        node-version: "18"
        cache: "npm"
        cache-dependency-path: dependency-risk-radar/frontend/package-l
    - name: Install dependencies
      working-directory: dependency-risk-radar/frontend
      run: npm ci
    - name: Run ESLint
      working-directory: dependency-risk-radar/frontend
      run: npx eslint src/ --max-warnings=20

secrets-scan:
  name: Secrets      Gitleaks
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0
    - name: Run Gitleaks
      uses: gitleaks/gitleaks-action@v2
      env:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

sca-python:
  name: SCA      pip-audit (backend)
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Set up Python 3.11
      uses: actions/setup-python@v5
      with:
        python-version: "3.11"
    - name: Install pip-audit
      run: pip install pip-audit
    - name: Audit Python dependencies
      run: >
        pip-audit -r dependency-risk-radar/backend/requirements.txt --s

```

---

```
—ignore-vuln CVE-2024-47874
—ignore-vuln CVE-2025-54121
—ignore-vuln PYSEC-2026-161
```

```
sca-node:
```

```
  name: SCA      npm audit (frontend)
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Set up Node.js 18
      uses: actions/setup-node@v4
      with:
        node-version: "18"
        cache: "npm"
        cache-dependency-path: dependency-risk-radar/frontend/package-l
    - name: Install dependencies
      working-directory: dependency-risk-radar/frontend
      run: npm ci
    - name: Audit Node dependencies
      working-directory: dependency-risk-radar/frontend
      run: npm audit --audit-level=high
```

```
build-and-test:
```

```
  name: Build      Docker + Tests
  runs-on: ubuntu-latest
  needs: [sast-python, sast-js, secrets-scan, sca-python, sca-node]
  services:
    redis:
      image: redis:7-alpine
      ports:
        - 6379:6379
  steps:
    - uses: actions/checkout@v4
    - name: Set up Python 3.11
      uses: actions/setup-python@v5
      with:
        python-version: "3.11"
    - name: Install backend dependencies and test tools
      working-directory: dependency-risk-radar/backend
      run: pip install -r requirements-test.txt
    - name: Run pytest with coverage
      working-directory: dependency-risk-radar/backend
      env:
        REDIS_URL: redis://localhost:6379
        DATABASE_URL: sqlite:///./test.db
      run: |
        pytest tests/ \
          --cov=app \
```

---

```

        --cov-report=xml:coverage.xml \
        --cov-fail-under=60 \
        -v
- name: Upload coverage report
  uses: actions/upload-artifact@v4
  with:
    name: coverage-report
    path: dependency-risk-radar/backend/coverage.xml
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3
- name: Build backend image
  uses: docker/build-push-action@v5
  with:
    context: ./dependency-risk-radar/backend
    push: false
    tags: drr-backend:${{ github.sha }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
    outputs: type=docker,dest=/tmp/drr-backend.tar
- name: Build frontend image
  uses: docker/build-push-action@v5
  with:
    context: ./dependency-risk-radar/frontend
    push: false
    tags: drr-frontend:${{ github.sha }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
    outputs: type=docker,dest=/tmp/drr-frontend.tar
- name: Save images as artifacts
  uses: actions/upload-artifact@v4
  with:
    name: docker-images
    path: /tmp/drr-*.tar
    retention-days: 1

container-scan:
  name: Container scan      Trivy
  runs-on: ubuntu-latest
  needs: [build-and-test]
  steps:
    - uses: actions/checkout@v4
    - name: Download built images
      uses: actions/download-artifact@v4
      with:
        name: docker-images
        path: /tmp/
    - name: Load Docker images
      run: |

```

---

```

        docker load --input /tmp/drr-backend.tar
        docker load --input /tmp/drr-frontend.tar
- name: Scan backend image
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: drr-backend:${{ github.sha }}
    format: table
    exit-code: "1"
    severity: CRITICAL
    ignore-unfixed: true
    trivyignores: dependency-risk-radar/backend/.trivyignore
- name: Scan frontend image
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: drr-frontend:${{ github.sha }}
    format: table
    exit-code: "1"
    severity: CRITICAL
    ignore-unfixed: true

send-logs-to-loki:
  name: Observability      Push logs to Loki
  runs-on: ubuntu-latest
  if: always()
  needs:
    - sast-python
    - sast-js
    - secrets-scan
    - sca-python
    - sca-node
    - build-and-test
    - container-scan
  steps:
    - name: Push pipeline results to Loki
      env:
        LOKI_URL: ${ secrets.LOKI_URL }
      run: |
        TIMESTAMP=$(date +%s%N)
        curl -s -X POST "${LOKI_URL}/loki/api/v1/push" \
          -H "Content-Type: application/json" \
          -d '{"streams": [{"stream": {"job": "github-actions",
            "pipeline": "ci", "repo": "${github.repository}",
            "branch": "${github.ref_name}",
            "event": "${github.event_name}"},
            "values": [{"${TIMESTAMP}",
              "${run_id}": "${github.run_id}",
              "${commit}": "${github.sha}",
              "${sast_python}": "${needs.sast-python.result}"}]}']

```

---

```

\\\\sast_js\\\\": \\\\\"${{ needs.sast-js.result }}\\\\",
\\\\secrets_scan\\\\": \\\\\"${{ needs.secrets-scan.result }}\\\\",
\\\\sca_python\\\\": \\\\\"${{ needs.sca-python.result }}\\\\",
\\\\sca_node\\\\": \\\\\"${{ needs.sca-node.result }}\\\\",
\\\\build_and_test\\\\": \\\\\"${{ needs.build-and-test.result }}\\\\",
\\\\container_scan\\\\": \\\\\"${{ needs.container-scan.result }}\\\\",

```

## A.3 Workflow CD — Build, Push & Deploy

Listing A.3 – Workflow CD complet — cd.yml

```

name: CD      Build , Push & Deploy
on:
  workflow_run:
    workflows: ["CI      Security & Quality Gate"]
    types:
      - completed
    branches: [main]

concurrency:
  group: production-deploy
  cancel-in-progress: false

permissions:
  id-token: write
  contents: read

jobs:
  push-images:
    name: Push      Amazon ECR
    runs-on: ubuntu-latest
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    outputs:
      image_tag: ${{ github.event.workflow_run.head_sha }}
      backend_image: ${{ steps.tag-backend.outputs.image }}
      frontend_image: ${{ steps.tag-frontend.outputs.image }}
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ github.event.workflow_run.head_sha }}
      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::${{ secrets.AWS_ACCOUNT_ID }}:role/
          aws-region: ${{ secrets.AWS_REGION }}
      - name: Login to Amazon ECR
        id: login-ecr

```



---

```

    uses: aws-actions/amazon-ecr-login@v2
- name: Download images from CI
  uses: actions/download-artifact@v4
  with:
    name: docker-images
    path: /tmp/
    run-id: ${ github.event.workflow_run.id }
    github-token: ${ secrets.GITHUB_TOKEN }
- name: Load Docker images
  run: |
    docker load --input /tmp/drr-backend.tar
    docker load --input /tmp/drr-frontend.tar
- name: Tag & Push Backend to ECR
  id: tag-backend
  env:
    ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
    IMAGE_TAG: ${ github.event.workflow_run.head_sha }
  run: |
    docker tag drr-backend:$IMAGE_TAG \
      $ECR_REGISTRY/drr/backend:$IMAGE_TAG
    docker tag drr-backend:$IMAGE_TAG \
      $ECR_REGISTRY/drr/backend:latest
    docker push $ECR_REGISTRY/drr/backend:$IMAGE_TAG
    docker push $ECR_REGISTRY/drr/backend:latest
    echo "image=$ECR_REGISTRY/drr/backend:$IMAGE_TAG" >> $GITHUB_OUTPUT
- name: Tag & Push Frontend to ECR
  id: tag-frontend
  env:
    ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
    IMAGE_TAG: ${ github.event.workflow_run.head_sha }
  run: |
    docker tag drr-frontend:$IMAGE_TAG \
      $ECR_REGISTRY/drr/frontend:$IMAGE_TAG
    docker tag drr-frontend:$IMAGE_TAG \
      $ECR_REGISTRY/drr/frontend:latest
    docker push $ECR_REGISTRY/drr/frontend:$IMAGE_TAG
    docker push $ECR_REGISTRY/drr/frontend:latest
    echo "image=$ECR_REGISTRY/drr/frontend:$IMAGE_TAG" >> $GITHUB_OUTPUT

deploy:
  name: Deploy      Amazon ECS
  runs-on: ubuntu-latest
  needs: [push-images]
  environment: production
  steps:
    - uses: actions/checkout@v4
      with:
        ref: ${ github.event.workflow_run.head_sha }

```

---

```

- name: Configure AWS credentials (OIDC)
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::${{ secrets.AWS_ACCOUNT_ID }}:role/
    aws-region: ${{ secrets.AWS_REGION }}
- name: Login to Amazon ECR
  id: login-ecr
  uses: aws-actions/amazon-ecr-login@v2
- name: Resolve image URIs
  id: image-uris
  env:
    ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
    IMAGE_TAG: ${ github.event.workflow_run.head_sha }
  run: |
    echo "backend=$ECR_REGISTRY/drr/backend:$IMAGE_TAG" >> $GITHUB_
    echo "frontend=$ECR_REGISTRY/drr/frontend:$IMAGE_TAG" >> $GITHU
- name: Download backend task definition
  run: |
    aws ecs describe-task-definition \
      --task-definition drr-backend \
      --query taskDefinition \
      > backend-task-def.json
- name: Update backend image in task definition
  id: backend-task-def
  uses: aws-actions/amazon-ecs-render-task-definition@v1
  with:
    task-definition: backend-task-def.json
    container-name: drr-backend
    image: ${ steps.image-uris.outputs.backend }
- name: Deploy backend to ECS
  uses: aws-actions/amazon-ecs-deploy-task-definition@v1
  with:
    task-definition: ${ steps.backend-task-def.outputs.task-defini
    service: drr-backend-service
    cluster: drr-cluster
    wait-for-service-stability: true
- name: Download frontend task definition
  run: |
    aws ecs describe-task-definition \
      --task-definition drr-frontend \
      --query taskDefinition \
      > frontend-task-def.json
- name: Update frontend image in task definition
  id: frontend-task-def
  uses: aws-actions/amazon-ecs-render-task-definition@v1
  with:
    task-definition: frontend-task-def.json
    container-name: drr-frontend

```

---

```

        image: ${ steps.image-uris.outputs.frontend }
- name: Deploy frontend to ECS
  uses: aws-actions/amazon-ecs-deploy-task-definition@v1
  with:
    task-definition: ${ steps.frontend-task-def.outputs.task-defin
    service: drr-frontend-service
    cluster: drr-cluster
    wait-for-service-stability: true
- name: Smoke test
  run: |
    for i in $(seq 1 6); do
      STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
        http://${ secrets.EC2_DRR_IP }:8000/api/v1/health \
        || echo "000")
      if [ "$STATUS" = "200" ]; then
        echo "Smoke test passed (attempt $i)"
        break
      fi
      echo "Attempt $i: got $STATUS, waiting 10s..."
      sleep 10
      if [ $i -eq 6 ]; then
        echo "Smoke test failed"
        exit 1
      fi
    done

send-logs-to-loki:
  name: Observability      Push logs to Loki
  runs-on: ubuntu-latest
  if: always()
  needs:
    - push-images
    - deploy
  steps:
    - name: Push pipeline results to Loki
      env:
        LOKI_URL: ${ secrets.LOKI_URL }
      run: |
        TIMESTAMP=$(date +%s%N)
        curl -s -X POST "${LOKI_URL}/loki/api/v1/push" \
          -H "Content-Type: application/json" \
          -d '{"streams": [{"stream": {"job": "github-actions",
            "ci_job": "deploy-ecs", "pipeline": "cd",
            "environment": "production",
            "status": "${ needs.deploy.result }"},
            "values": [{"${TIMESTAMP}",
            "${ commit }",
            "${ github.event.workflow_run.head_s
            "${ push_images }"}]}]}'

```

---

```

        \\\\"deploy\\\\": \\\\"${{ needs.deploy.result }}\\\\"}\\"]]]}}"
- name: Notify on success
  if: ${{ needs.deploy.result == 'success' }}
  run: echo "DRR deployed successfully."
- name: Notify on failure
  if: ${{ needs.deploy.result == 'failure' }}
  run: echo "Deployment FAILED."

```

## A.4 Task Definition drr-redis (JSON complet)

Listing A.4 – Task Definition drr-redis — cache Redis 7-alpine avec politique LRU

```

{
  "family": "drr-redis",
  "networkMode": "bridge",
  "requiresCompatibilities": ["EC2"],
  "cpu": "128",
  "memory": "256",
  "containerDefinitions": [{
    "name": "redis",
    "image": "redis:7-alpine",
    "essential": true,
    "command": [
      "redis-server",
      "--maxmemory", "256mb",
      "--maxmemory-policy", "allkeys-lru"
    ],
    "portMappings": [{"containerPort": 6379, "protocol": "tcp"}],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/drr-redis",
        "awslogs-region": "eu-west-1",
        "awslogs-create-group": "true",
        "awslogs-stream-prefix": "ecs"
      }
    }
  ]
}
}
}

```

## A.5 Task Definition drr-backend (JSON complet)

Listing A.5 – Task Definition drr-backend — backend FastAPI avec secrets et volume SQLite

```

{

```

---

```

"family": "drr-backend",
"networkMode": "bridge",
"requiresCompatibilities": ["EC2"],
"cpu": "512",
"memory": "512",
"executionRoleArn": "arn:aws:iam::ID:role/ecsTaskExecutionRole",
"containerDefinitions": [{
  "name": "drr-backend",
  "image": "ID.dkr.ecr.eu-west-1.amazonaws.com/drr/backend:latest",
  "essential": true,
  "portMappings": [{"containerPort": 8000, "hostPort": 0}],
  "environment": [
    {"name": "DATABASE_URL", "value": "sqlite:///tmp/drr_reports/drr"},
    {"name": "OUTPUT_DIR", "value": "/tmp/drr_reports"},
    {"name": "DEBUG", "value": "false"},
    {"name": "GEMINI_MODEL", "value": "gemini-1.5-flash"},
    {"name": "MAX_COMPONENTS_FOR_LLM", "value": "30"}
  ],
  "secrets": [
    {"name": "GEMINI_API_KEY",
     "valueFrom": "arn:aws:secretsmanager:eu-west-1:ID:secret:GEMINI_API_KEY"},
    {"name": "ANTHROPIC_API_KEY",
     "valueFrom": "arn:aws:secretsmanager:eu-west-1:ID:secret:ANTHROPIC_API_KEY"},
    {"name": "NVD_API_KEY",
     "valueFrom": "arn:aws:secretsmanager:eu-west-1:ID:secret:NVD_API_KEY"}
  ],
  "mountPoints": [{
    "sourceVolume": "drr_reports",
    "containerPath": "/tmp/drr_reports",
    "readOnly": false
  }
  ],
  "logConfiguration": {
    "logDriver": "awslogs",
    "options": {
      "awslogs-group": "/ecs/drr-backend",
      "awslogs-region": "eu-west-1",
      "awslogs-create-group": "true",
      "awslogs-stream-prefix": "ecs"
    }
  }
}],
"volumes": [{
  "name": "drr_reports",
  "host": {"sourcePath": "/tmp/drr_reports"}
}]
}

```

---

## A.6 Task Definition drr-frontend (JSON complet)

Listing A.6 – Task Definition drr-frontend — frontend React servi par Nginx

```
{
  "family": "drr-frontend",
  "networkMode": "bridge",
  "requiresCompatibilities": ["EC2"],
  "cpu": "256",
  "memory": "256",
  "containerDefinitions": [{
    "name": "drr-frontend",
    "image": "ID.dkr.ecr.eu-west-1.amazonaws.com/drr/frontend:latest",
    "essential": true,
    "portMappings": [{"containerPort": 80, "hostPort": 0}],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/drr-frontend",
        "awslogs-region": "eu-west-1",
        "awslogs-create-group": "true",
        "awslogs-stream-prefix": "ecs"
      }
    }
  ]
}
```

## A.7 Trust Policy du rôle GitHubActionsRole

La Trust Policy restreint l'assumption du rôle au seul repository du projet, via une condition `StringLike` sur le champ `sub` du token JWT émis par GitHub.

Listing A.7 – Trust Policy du rôle GitHubActionsRole — restriction par repository

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated":
        "arn:aws:iam::ID:oidc-provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {

```

---

```

        "token.actions.githubusercontent.com:sub":
          "repo:QuantumByte2007/DependancyRiskRadar:*"
      }
    }
  }
}

```

## A.8 Politique inline DRRSecretsManagerAccess

Cette politique est attachée au rôle `ecsTaskExecutionRole`. Elle accorde l'action `GetSecretValue` sur l'ensemble des secrets du compte dans la région `eu-west-1`.

Listing A.8 – Politique inline DRRSecretsManagerAccess attachée à `ecsTaskExecutionRole`

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["secretsmanager:GetSecretValue"],
    "Resource": [
      "arn:aws:secretsmanager:eu-west-1:ID:secret:*"
    ]
  }]
}

```

## A.9 Configuration Loki — schéma de stockage v13

Loki utilise le schéma `tsdb` (v13) recommandé depuis les versions récentes. La période d'index de 24 heures est adaptée au volume de logs généré par le pipeline DevSecOps de DRR.

Listing A.9 – Configuration Loki — schéma de stockage et provisioning datasource Grafana

```

# loki-config.yaml
auth_enabled: false

server:
  http_listen_port: 3100

ingester:
  lifecycler:
    address: 127.0.0.1
    ring:
      kvstore:
        store: inmemory
      replication_factor: 1
    final_sleep: 0s

```

---

```

    chunk_idle_period: 5m
    chunk_retain_period: 30s

schema_config:
  configs:
    - from: 2020-10-24
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h

storage_config:
  tsdb_shipper:
    active_index_directory: /loki/index
    cache_location: /loki/index_cache
  filesystem:
    directory: /loki/chunks

limits_config:
  reject_old_samples: true
  reject_old_samples_max_age: 168h

# grafana/provisioning/datasources/loki.yaml
apiVersion: 1
datasources:
  - name: Loki
    type: loki
    access: proxy
    url: http://loki:3100
    isDefault: true
    editable: true

```

## A.10 Smoke test post-déploiement

Le smoke test est exécuté à la fin du job `deploy` du pipeline CD. Il interroge l'endpoint `/health` du backend avec jusqu'à six tentatives espacées de dix secondes.

Listing A.10 – Smoke test post-déploiement — vérification de l'endpoint `/health`

```

for i in $(seq 1 6); do
  STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
    http://${ secrets.EC2_DRR_IP }:8000/api/v1/health || echo "000")
  if [ "$STATUS" = "200" ]; then
    echo "Smoke test passed (attempt $i)"
    break
  fi
fi

```



---

```
echo "Attempt_$i: got_$STATUS, waiting_10s ..."
sleep 10
if [ $i -eq 6 ]; then
    echo "Smoke_test_failed_after_6_attempts"
    exit 1
fi
done
```

## A.11 Requêtes LogQL du dashboard DevSecOps

Les requêtes suivantes alimentent les panels du dashboard Grafana *DRR DevSecOps Pipeline*.

Listing A.11 – Requêtes LogQL du dashboard Grafana DevSecOps

```
— Echecs CI des 24 dernieres heures
{pipeline="ci", status="failure"}

— Timeline des deploiements ECS reussis
count_over_time({ci_job="deploy-ecs", status="success"}[1h])

— Alertes DAST critiques uniquement
{ci_job="dast-zap"} | json | backend_high > 0

— Vue globale pipeline complet
{environment=~"ci|production"}
```